



OS-mate 机器人开发平台

基于 Ubuntu20.04 noetic

用户使用手册

User manual V1.0.2

目录

前言	2
重要安全信息	3
导航使用注意事项	4
导航功能	5
导航部件	6
导航拓扑	7
快速使用说明	9
终端命令操作	10
实验一 初识 ROS	13
实验二 底盘驱动及控制	16
实验三 IMU 数据处理	29
实验四 里程计融合	32
实验五 3D 激光雷达	39
实验六 2D 建图	49
实验七 3D 建图	56
实验八 Navigation 导航框架简介	70
实验九 Rviz 下点自主导航	72
实验十 自主导航 Node 的编程	84
实验十一 全局路径规划	90
实验十二 局部路径规划	92
实验十三 RGB-D 相机	96
实验十四 RGB-D 相机的三维点云	99
实验十五 障碍图层加入相机 LaserScan 数据	105
实验十六 回充	109
常见问题解决方法	111
附录一 远程控制	112
附录二 系统备份	120
附录三 安装 vscode	123

前言

ROS 是一个构建在 Ubuntu 上的机器人开发系统，由一系列的软件包和通讯架构组成，对于学生来说，Windows 可能用得比较多，相对 Ubuntu 比较陌生，所以在学习 ROS 的过程中，需要克服的困难更多一些。好在国内互联网行业发展迅猛，关于 Ubuntu 和 Linux 的书籍层出不穷，非常便于自学。对于 ROS 的学习，各位开发者也可以根据自身情况选择合适的方式，这里我们给出两个参考建议，希望能有帮助。

（1）对于时间不是很紧急，同时也对学习极具耐心的开发者，可以先通过书籍和网上教程学习 Ubuntu 的使用方法，然后去 ROS 的官方 Wiki 阅读了解 ROS 的详细介绍，理解里面的各种概念，再将 ROS 的官方 Wiki 上的 Tutorials 例程都亲手实现一遍，最后再回到我们这个教程。

（2）对于开发周期比较紧迫，对理论学习不是特别有耐心的开发者，可以直接按照本教程开始进行练习操作。先快速的将机器人运行起来，以避免枯燥的理论学习浇灭了心中学习的热情。经过本教程的操作练习，对 ROS 有了一个大体体验之后，再回过头查漏补缺的将理论知识补齐。

重要安全信息

- **定期检查和维护**
请定期检查机器人的各个部件，包括传感器、电池、电机和结构件，确保其正常运行和完好无损。及时更换磨损的部件，避免因机器人故障引发意外情况。
- **安全操作**
在操作 ROS 移动机器人之前，请确保您已经了解并熟悉了相关的操作手册和安全指南。遵循正确的操作步骤，避免不必要的风险。避免将机器人用于超出其设计和能力范围的任务。
- **保持周围环境清洁整洁**
确保机器人周围的工作区域清洁整洁，避免障碍物和杂物的存在。清理地面上的电线、绳索或其他可能导致机器人绊倒的物体。
- **安全隔离**
在进行机器人操作时，请确保没有人员进入机器人的工作区域，以防止意外伤害。根据需要，使用合适的安全隔离设备，例如栅栏或标识，来提醒他人不要靠近机器人。
- **紧急停止装置**
在任何时候，都要确保机器人配备了紧急停止装置，并且该装置易于访问。在紧急情况下，能够迅速切断机器人的电源以避免伤害。
- **数据备份与网络安全**
对于 ROS 移动机器人上的重要数据和配置文件，请定期进行备份，并采取适当的措施保护机器人与网络的安全。确保机器人的软件和固件始终是最新版本，以确保系统的可靠性和安全性。

导航使用注意事项

- 关于软件重启
使用“reboot”命令或者通过界面操作，工控机重启后，小车上的 CAN 卡是不能正常使用的，所以只能断电重启。
- Ubuntu 系统版本
煜禾森 ROS 机器人工控机安装的是 Ubuntu 20.04 的 noetic 版本。（**注意：如果小车 Ubuntu 界面弹出升级系统的弹窗时，关闭，不能更新系统版本**）
- 软件安装和卸载
进行软件安装和卸载时，要仔细查看终端打印的信息，可能牵扯到其他软件，必要时可以先备份系统（请看附录二）。
- 软件包
煜禾森机器人出厂标配的电脑里已经安装好 ROS、IDE 和煜禾森机器人的源码包，可以直接使用。
- 文件修改
建议对煜禾森提供的源码包中任何文件进行修改前，都做备份。
- 关于联网
本文档后面的所有操作（安装软件包除外）都不需要联网，联网方式建议使用手机 USB 共享网络。
- 连接屏幕
本说明文档上的所有终端操作，需要小车上工控机连接屏幕。
- 不连接屏幕操作方法
请看附录一。

导航功能

● 障碍物检测

传感器	功能
3D 激光雷达	检测动静态障碍物
深度相机	检测动静态障碍物
超声波	检测动静态障碍物

● 2D 建图

场景	功能
室内	扫描创建 2D 栅格地图

● 3D 建图

场景	功能
室外	扫描创建 3D 点云图

● 2D 定位

场景	功能
室内	使用 AMCL 定位

● 3D 定位

场景	功能
室内	使用 NDT 定位
室外	使用 NDT 定位

● 3D 定位导航

场景	功能
室内	单点导航
	多点导航
	录制路径
室外	单点导航
	多点导航
	录制路径

● 绕障

场景	功能
室内	导航过程绕障
室外	导航过程绕障

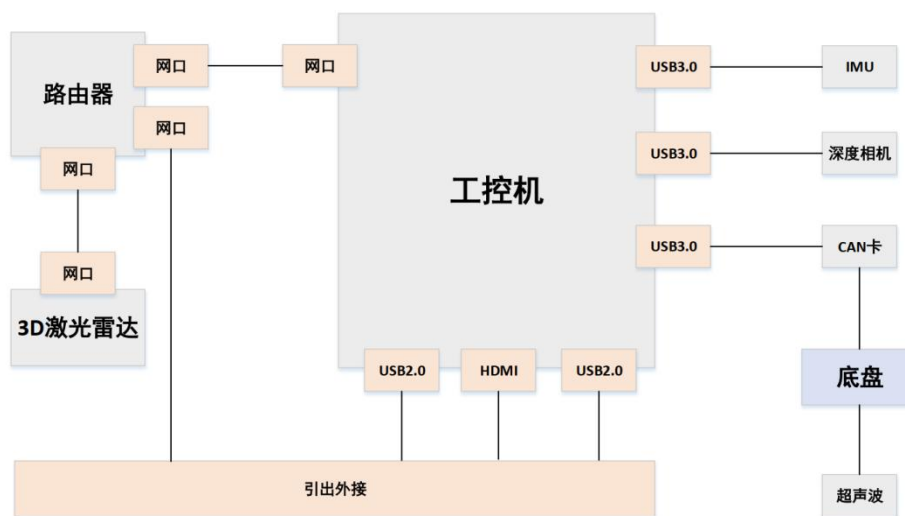
导航部件

导航部件及参数

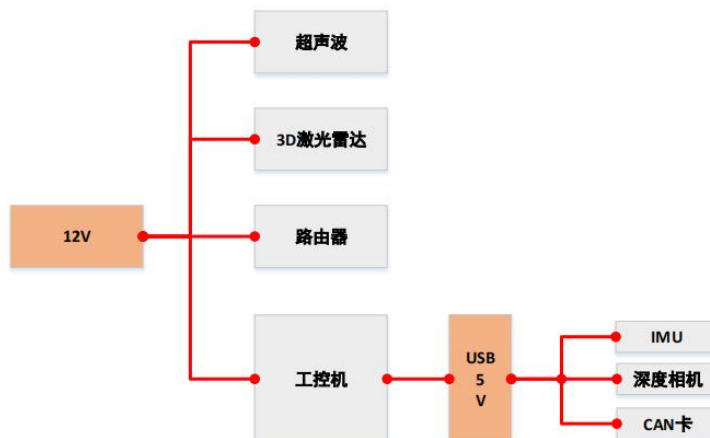
工控机	CPU: 酷睿 8 代 I5-8265U/1.6GHz
	内存: 8G DDR4
	硬盘: 128G
	USB: USB3.0*4 USB2.0*2
	网口: 千兆网口*2
	显示接口: HDMI
	串口: DB9 RS232*4
3D 激光雷达	测距范围: 0.2m~150m
	水平视场角: 360°
	垂直视场角: -15° ~ +15°
	测距精度: 2cm
	角度分辨率: 0.18° (10HZ)
	扫描频率: 10HZ
	通讯:网口
IMU	陀螺仪测量范围: $\pm 2000^{\circ} /s$
	加速度计: $\pm 8G(1G = 1x \text{ 重力加速度})$
	陀螺仪分辨率: $0.01^{\circ} /s$
	加速度计: 1uG
	陀螺仪零偏稳定性: $8^{\circ} /h$
深度相机	加速度计: 60uG
	通讯: USB 转串口
	工作范围深度: 0.2~4m
	工作温度: -10~50℃
	视场角深度: H73.8° × V58.8°
	RGB: H80.9° × V51.7°
CAN 卡	分辨率@帧率:
	640×480@30fps
	通讯: USB2.0
路由器	通讯: USB 转 CAN
	网络标准: 802.11ac
	无线网络支持频率: 2.4G/5G
	网口: 1900M 千兆端口*3

导航拓扑

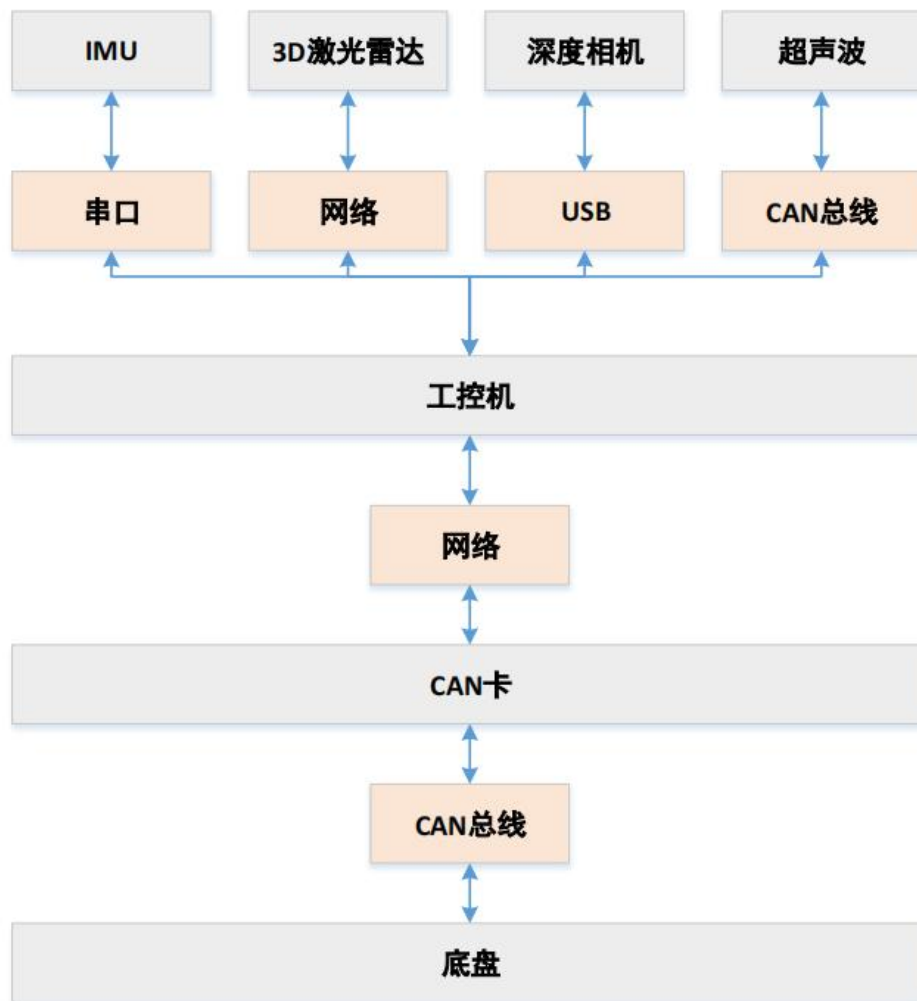
● 部件连接拓扑



● 电源连接拓扑



- 通讯连接拓扑



快速使用说明

- 路由器 wifi 名和密码

wifi 名	YHS-ROS-XX (XX 是车型名称)
wifi 密码、路由器登录密码	12345678

- 电脑用户名和密码

ubuntu 用户名	yhs
root 密码	123456
源码存放目录	/home/yhs/catkin_ws/src

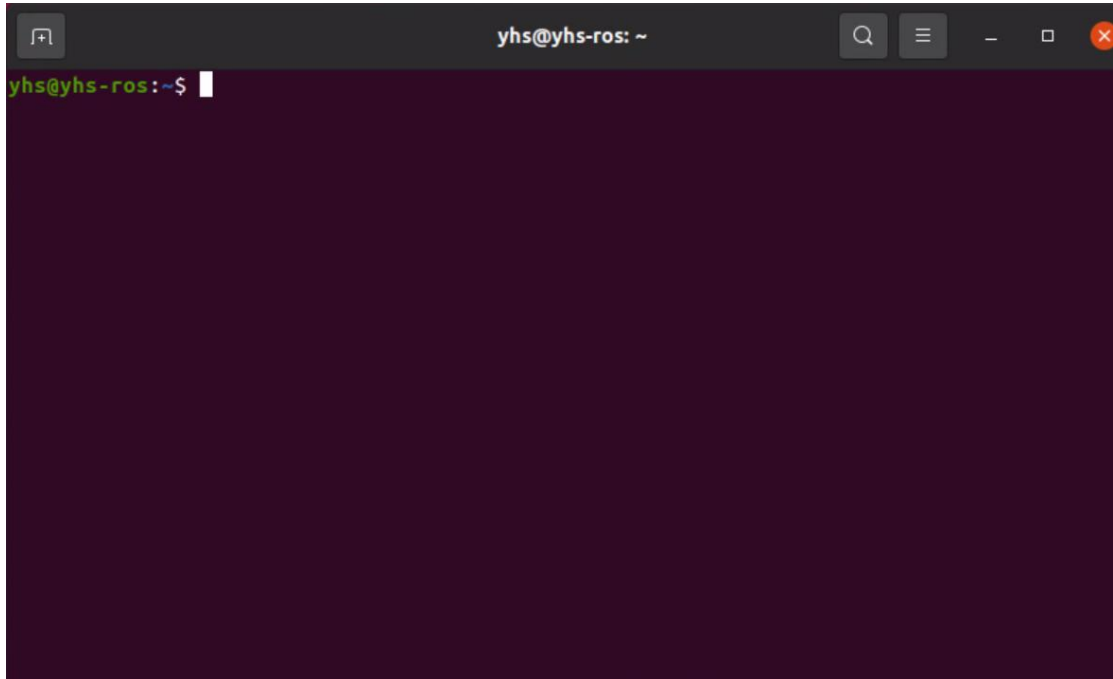
- 小车上电前，请仔细阅读随车携带的使用手册，了解遥控器的操作。
- 上电后，将小车遥控行走至较为空旷区域，避免在不太熟练情况下发生碰撞，建议将控制速度调制最低档。
- 如果对 Ubuntu 系统不太熟悉，可以阅读下一章节的终端操作介绍，按照步骤操作，较为熟悉后，再让小车自动实现自动驾驶功能。
- 让小车实现自动驾驶功能，如果小车上不接屏幕，则需要准备一台装有 ROS 系统的电脑，连上小车的 Wifi，按照附录一的操作说明，远程控制小车；然后跳过前面实验章节，直接按照实验六、实验七和实验九的说明进行操作。

终端命令操作

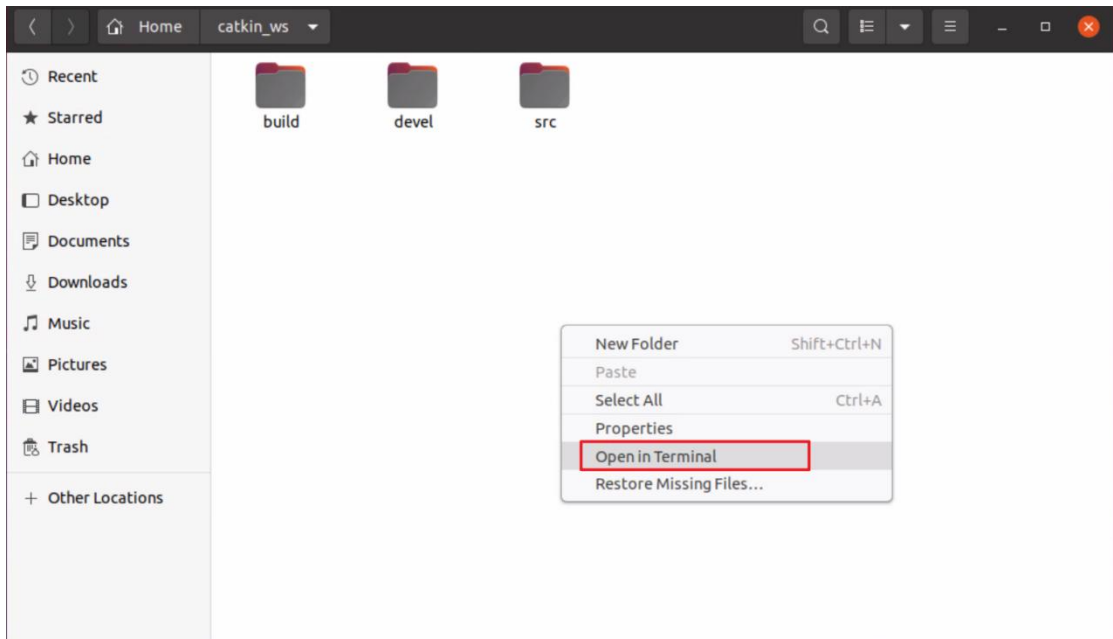
本文档接下来章节可能涉及到的终端命令操作说明。

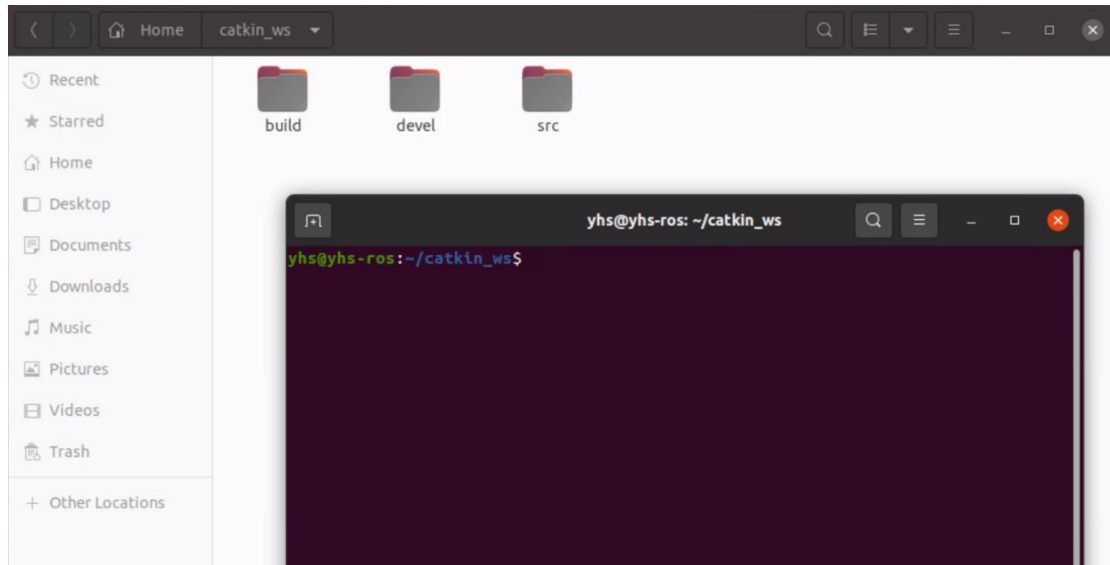
1、打开终端

(1) 按快捷键 “ctrl + alt + t”，默认路径为 “home” 目录



(2) 在文件内打开，路径为文件所在路径





2、“Tab”键补全命令

(1) 在终端运行程序时，有些命令比较长或者复杂时，就可以用“Tab”键来补全，完整的命令如下

```
roslaunch yhs_nav yhs_nav.launch
```

(2) 输入“roslaunch yhs_nav”后，按“Tab”键，可能要按几次，出现四个选项，如果有更多，都会显示出来，所以我们要尽可能输入更多的信息，这样按“Tab”键，列出来的选项会更少，根据列出的选项，接着输入，继续按“Tab”键补全。如果列出来的所有选项不包含目标文件/目标命令，检查已输入的命令是否正确、目标文件/目标命令是否存在、环境变量是否正确。

```
yhs@yhs-ros:~/catkin_ws$ roslaunch yhs_nav  
cartographer.launch gmapping.launch yhs_nav_2d.launch yhs_nav.launch  
yhs@yhs-ros:~/catkin_ws$ roslaunch yhs_nav yhs_nav.launch
```

(3) 输入以下命令，补全的选项

```
roslaunch yhs_
```

```
yhs@yhs-ros:~/catkin_ws$ roslaunch yhs_  
yhs_can_control yhs_msgs yhs_nav  
yhs@yhs-ros:~/catkin_ws$ roslaunch yhs_
```

(4) 快速进入 Package 中文件夹的命令，输入“roscd yhs_nav”后，继续按几次“Tab”，会显示 Package 中的所有文件夹

```
roslaunch yhs_nav2
```

```
yhs@yhs-ros:~/catkin_ws$ roscd yhs_nav/  
yhs_nav/include/ yhs_nav/launch/ yhs_nav/map/ yhs_nav/param/ yhs_nav/src/  
yhs@yhs-ros:~/catkin_ws$ roscd yhs_nav/
```

3、终端终止程序命令

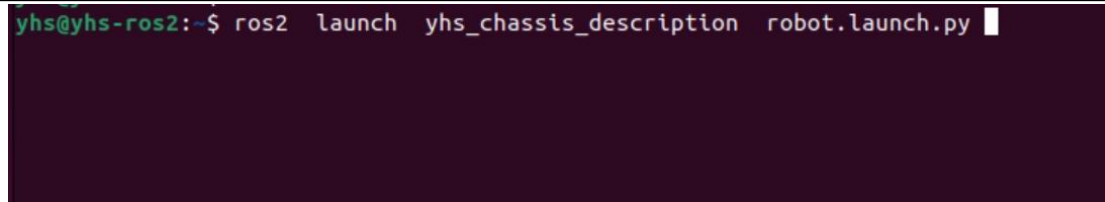
(1) 假设某个终端执行着激光雷达驱动程序，接着要执行建图程序命令，建图命令会启动激光雷达，这时就会报错，所以在执行建图命令前要终止激光雷达程序。在执行激光雷达驱动程序的终端按“Ctrl + C”后，等待程序终止，进行下一个实验前把本次实验的启动的程序全部终止

```
^C[pointcloud_to_laserscan-4] killing on exit  
[laser_link_to_base_link-3] killing on exit  
[rslidar_sdk_node-2] killing on exit  
RoboSense-LiDAR-Driver is stopping.....  
terminate called after throwing an instance of 'std::system_error'  
  what(): Invalid argument  
[rosout-1] killing on exit  
[master] killing on exit  
shutting down processing monitor...  
... shutting down processing monitor complete  
done
```

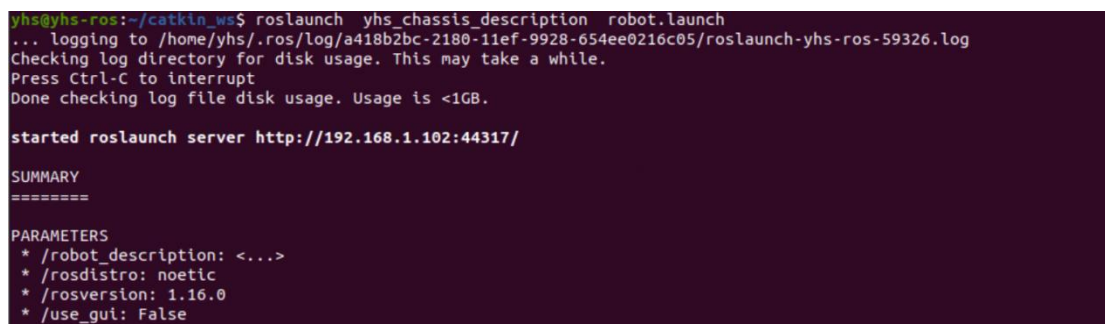
实验一 初识 ROS

1、打开一个终端，输入

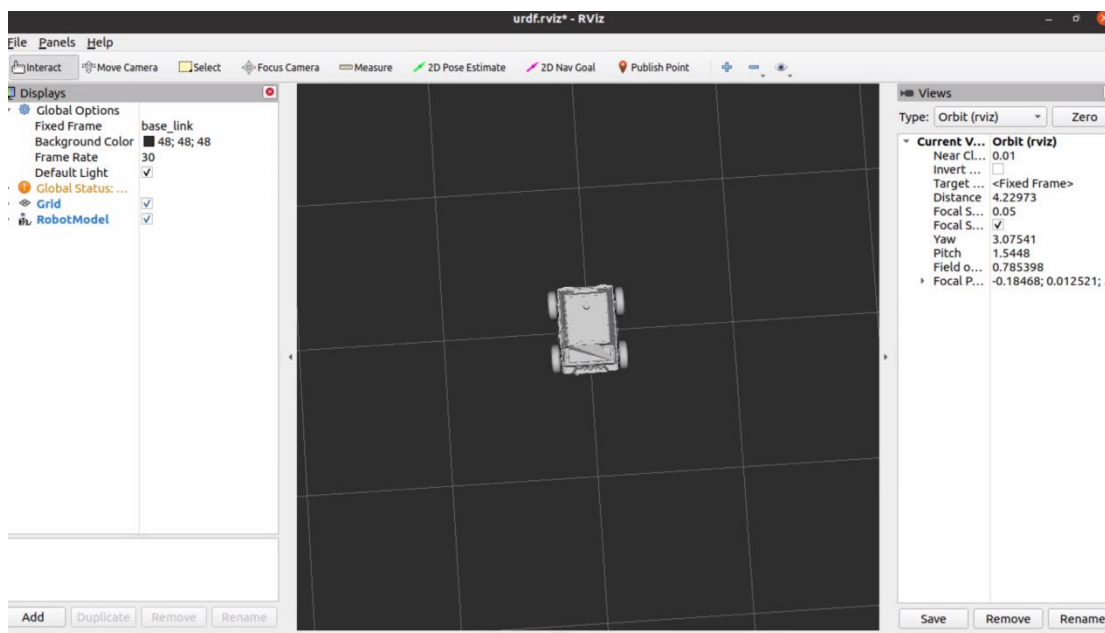
```
roslaunch yhs_chassis_description robot.launch
```



指令输入后按下回车，同时桌面上能看到一个“Rviz”的启动 Logo。经过片刻的初始化工作，一个图形窗体出现在桌面上，这就是 ROS 里最常用的图形化显示界面 Rviz，所有用到的传感器以及算法产生的结果都可以很直观的显示在这个界面里，为机器人编程调试提供了很大的方便。



通常 Rviz 的主界面会分为左右两栏，左边一栏是“Displays”，右边是主显示区“View”。其中右边“View”是一个三维世界的显示，默认情况下可以看到栅格状的基准地面，可使用鼠标拖动来调整三维视角。左边的“Displays”则用于配置“View”里显示的信息种类和数量。



2、回到刚才我们输入的指令（`roslaunch yhs_chassis_description robot.launch`），是用 `roslaunch` 工具软件，启动了一个名为“`yhs_chassis_description`”的软件包里的 `robot.launch` 启动文件。Launch 文件是 ROS 里对多个程序节点（Node）进行批量启动的文本描述文件，下面我们来看看 `robot.launch` 这个文件的内容：

```
<launch>

  <arg name="gui" default="false" />

  <param name="robot_description" textfile="$(find yhs_chassis_description)/urdf/robot.urdf"
/>
  <param name="use_gui" value="$(arg gui)"/>

  <node name="joint_state_publisher" pkg="joint_state_publisher"
type="joint_state_publisher" />

  <node name="robot_state_publisher" pkg="robot_state_publisher"
type="robot_state_publisher" />

  <node name="rviz" pkg="rviz" type="rviz" args="-d $(find
yhs_chassis_description)/rviz/urdf.rviz" />

</launch>
```

这是一个 xml 格式的文本文件，我们简要了解其内容：

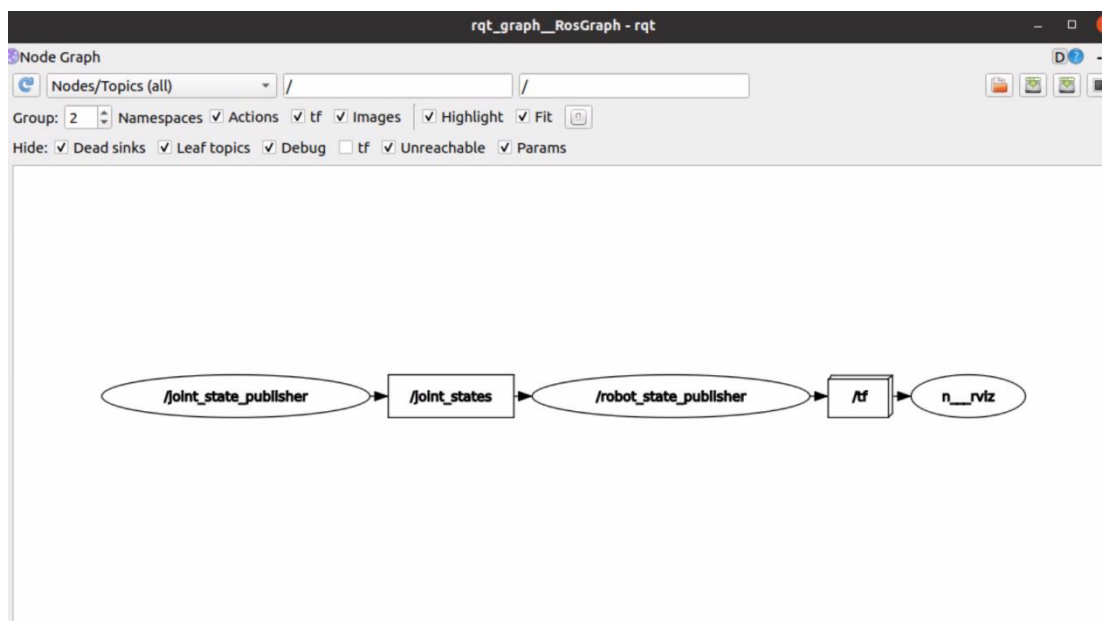
- （1）以“arg”开头的是设置静态参数，gui 是控制界面显示标记；
- （2）以“param”开头的是设置动态参数，robot_description 是解析机器人 urdf 描述的指令，use_gui 是控制界面显示标记；
- （3）以“node”开头的是 launch 文件的重点，每一条会启动一个程序节点，其中 name 是节点启动后在 ROS 里显示的名称（这个其实可以随便起名），pkg 是节点所从属的软件包名称，用来定位节点程序位置，type 是节点的真实名字，args 是启动参数。

2、看完 launch 文件内容，我们来看看它运行的幕后是什么情况。维持本节开头“robot.launch”启动的 Rviz 界面别关闭。打开一个新的终端，输入一条新指令

```
rqt_graph
```



按下回车键，会弹出一个框图



这个框图就是目前运行起来的节点及相互之间的数据关系图，椭圆的是节点，带箭头的曲线是数据流，矩形方框是主题，节点之间是通过“主题”这个渠道来传递数据的。可以看到图中的三个节点和刚才 launch 里的<node>项一一对应。这里再简要了解一下：

- (1) joint_state_publisher 节点从 robot_description 参数指令解析的机器人 urdf 里读取机器人各关节值，通过/joint_states 主题传递给 robot_state_publisher 节点；
- (2) robot_state_publisher 节点将获得的机器人关节值转换成 tf 格式数据，发布到/tf 主题；
- (3) rviz 节点（就是我们看到的图形界面）从/tf 主题读取数据，实时更新其显示的三维模型。

launch 文件是大部分 ROS 软件包的入口。在 ROS 学习初期，从网上找到一个 ROS 软件包，看着成堆的目录会比较迷惑，不知从何入手。这时可以先点开里面的 launch 文件夹，逐个阅读其 launch 文件，就能大概知道它启动的哪个 pkg 里的哪个 node，然后再到相应的 pkg 的 src 目录下去找这个 node 的 cpp 或者 py 类型文件，了解他们的组合关系，这样就能慢慢的把这个软件包的结构和功能都弄明白了。

实验二 底盘驱动及控制

煜禾森机器人底盘采用的是 CAN 通讯方式,电脑不能直接底盘通讯,用 USB 转 CAN 工具,也就是 CAN 卡,根据底盘通讯协议和 CAN 卡的使用方式,需要一个专门的驱动程序包。

1、CAN 卡设置,小车上配套的 CAN 卡,连接到工控机或者笔记本上,ubuntu 系统,要在终端执行两条命令后才能正常使用(配套的工控机开机过程已设置好)

```
sudo ip link set can0 type can bitrate 500000
sudo ifconfig can0 up
```

```
yhs@yhs-ros:~$ sudo ip link set can0 type can bitrate 500000
yhs@yhs-ros:~$ sudo ip link set can0 up
yhs@yhs-ros:~$
```

2、CAN 卡读取打印数据,安装“can-utils”软件工具包后,在终端用命令读取查看或者发送数据

```
sudo apt install can-utils
candump can0
```

```
yhs@yhs-ros2:~$ candump can0
can0 18C4D1EF [8] 00 00 00 00 00 00 00 00
can0 18C4D7EF [8] 00 00 F3 FF FF FF 00 0C
can0 18C4D8EF [8] 00 00 01 00 00 00 00 01
can0 18C4EAEF [8] E3 B0 00 31 01 00 50 33
can0 18C4E2EF [8] 64 01 00 40 10 FA 80 4F
can0 18C4D1EF [8] 00 00 00 00 00 00 10 10
can0 18C4D7EF [8] 00 00 F3 FF FF FF 10 1C
can0 18C4D8EF [8] 00 00 01 00 00 00 10 11
can0 18C4EAEF [8] E3 B0 00 31 01 00 60 03
can0 18C4DAEF [8] 00 10 00 00 00 00 00 10
can0 18C4E1EF [8] 90 0A 00 00 AC 03 80 B5
can0 18C4D1EF [8] 00 00 00 00 00 00 20 20
can0 18C4D7EF [8] 00 00 F3 FF FF FF 20 2C
can0 18C4D8EF [8] 00 00 01 00 00 00 20 21
can0 18C4D1EF [8] 00 00 00 00 00 00 30 30
can0 18C4D7EF [8] 00 00 F3 FF FF FF 30 3C
can0 18C4D8EF [8] 00 00 01 00 00 00 30 31
can0 18C4EAEF [8] E3 B0 00 31 01 00 70 13
can0 18C4D1EF [8] 00 00 00 00 00 00 40 40
can0 18C4D7EF [8] 00 00 F3 FF FF FF 40 4C
can0 18C4D8EF [8] 00 00 01 00 00 00 40 41
```

3、读取或者发送拓展帧数据,can 地址最高位要置 1,否则不能正常发送和接收数据,所以驱动程序里面的地址开头都会从“0x18”变成“0x98”,下面是头文件“/usr/include/linux/can.h”中关于地址的说明

```
/*
 * Controller Area Network Identifier structure
 *
 * bit 0-28 : CAN identifier (11/29 bit)
 * bit 29 : error message frame flag (0 = data frame, 1 = error message)
 * bit 30 : remote transmission request flag (1 = rtr frame)
 * bit 31 : frame format flag (0 = standard 11 bit, 1 = extended 29 bit)
 */
```

4、了解了 CAN 卡的设置和使用，接下来运行底盘驱动程序，驱动程序调用 CAN 卡读取底盘信息和发送命令控制底盘

(1) 指令敲回车，可以看到底盘类型，节点被成功初始化

```
roslaunch yhs_can_control yhs_can_control.launch.py
```

```
yhs@yhs-ros:~$ roslaunch yhs_can_control yhs_can_control.launch
... logging to /home/yhs/.ros/log/51b1dc26-218d-11ef-9928-654ee0216c05/roslaunch-yhs-ros-60638.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:37901/
```

```
NODES
/
  yhs_can_control_node (yhs_can_control/yhs_can_control_node)

auto-starting new master
process[roscpp-1]: started with pid [60647]
ROS_MASTER_URI=http://192.168.1.102:11311

setting /run_id to 51b1dc26-218d-11ef-9928-654ee0216c05
process[roscpp-2]: started with pid [60657]
started core service [/roscpp]
process[yhs_can_control_node-2]: started with pid [60664]
[ INFO] [1717407632.225772820]: Chassis type: FW
[ INFO] [1717407632.226339177]: yhs_can_control_node initialized successfully
```

(2) 如果 CAN 设置不成功，或者 CAN 线连接异常，会出现警告

```
setting /run_id to d56f9d6e-218d-11ef-9928-654ee0216c05
process[roscpp-1]: started with pid [60744]
started core service [/roscpp]
process[yhs_can_control_node-2]: started with pid [60751]
[ INFO] [1717407853.253432985]: Chassis type: FW
[ INFO] [1717407853.254001539]: yhs_can_control_node initialized successfully
[ WARN] [1717407853.304268209]: Timeout waiting for CAN frame! Please check whether the can0 setting is correct, whether the c
an line is connected correctly, and whether the chassis is powered on.
[ WARN] [1717407853.354499353]: Timeout waiting for CAN frame! Please check whether the can0 setting is correct, whether the c
an line is connected correctly, and whether the chassis is powered on.
[ WARN] [1717407853.404736517]: Timeout waiting for CAN frame! Please check whether the can0 setting is correct, whether the c
an line is connected correctly, and whether the chassis is powered on.
[ WARN] [1717407853.454966327]: Timeout waiting for CAN frame! Please check whether the can0 setting is correct, whether the c
an line is connected correctly, and whether the chassis is powered on.
[ WARN] [1717407853.505214884]: Timeout waiting for CAN frame! Please check whether the can0 setting is correct, whether the c
an line is connected correctly, and whether the chassis is powered on.
```

5、驱动程序运行起来后，要查看底盘的数据，或者要下发命令控制底盘，要通过什么方式跟驱动程序通讯呢，通常是通过“topic”的订阅和发布，那么在另外一个终端输入下面指令查看所有的“topic”

```
rostopic list
```

```
yhs@yhs-ros:~$ rostopic list
/angular
/chassis_info_fb
/fw_ctrl_cmd
/imu_data
/io_cmd
/linear
/odom
/rosout
/rosout_agg
/scan3
/smoothother_cmd_vel
/tf
/ultrasonic
yhs@yhs-ros:~$
```

要查看底盘的所有信息，可以输入下面指令，同样的，要查看其他话题的数据，使用“rostopic echo /话题名”即可

```
rostopic echo /chassis_info_fb
```

```
yhs@yhs-ros:~$ rostopic echo /chassis_info_fb
```

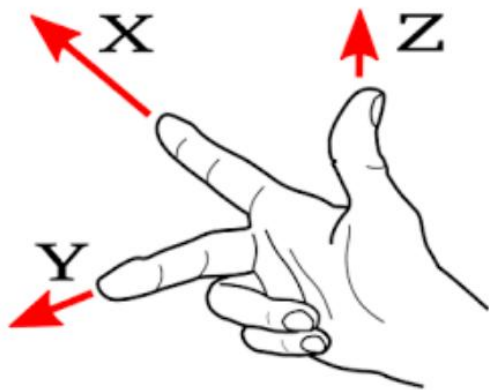
```
header:
  seq: 864
  stamp:
    secs: 1717462475
    nsecs: 699478829
  frame_id: ''
ctrl_fb:
  ctrl_fb_gear: 1
  ctrl_fb_linear: 0.0
  ctrl_fb_angular: 0.0
  ctrl_fb_slipangle: 0.0
io_fb:
  io_fb_lamp_ctrl: False
  io_fb_unlock: False
  io_fb_lower_beam_headlamp: True
  io_fb_upper_beam_headlamp: False
  io_fb_turn_lamp: 0
  io_fb_braking_lamp: True
  io_fb_clearance_lamp: False
  io_fb_fog_lamp: False
  io_fb_speaker: False
  io_fb_fl_impact_sensor: False
  io_fb_fm_impact_sensor: False
  io_fb_fr_impact_sensor: False
  io_fb_rl_impact_sensor: False
  io_fb_rm_impact_sensor: False
  io_fb_rr_impact_sensor: False
  io_fb_fl_drop_sensor: False
  io_fb_fm_drop_sensor: False
  io_fb_fr_drop_sensor: False
  io_fb_rl_drop_sensor: False
  io_fb_rm_drop_sensor: False
  io_fb_rr_drop_sensor: False
  io_fb_estop: False
  io_fb_joypad_ctrl: False
  io_fb_charge_state: False
```

```
bms_flag_fb:
  bms_flag_fb_soc: 76
  bms_flag_fb_single_ov: False
  bms_flag_fb_single_uv: False
  bms_flag_fb_ov: False
  bms_flag_fb_uv: False
  bms_flag_fb_charge_ot: False
  bms_flag_fb_charge_ut: False
  bms_flag_fb_discharge_ot: False
  bms_flag_fb_discharge_ut: False
  bms_flag_fb_charge_oc: False
  bms_flag_fb_discharge_oc: False
  bms_flag_fb_short: False
  bms_flag_fb_ic_error: False
  bms_flag_fb_lock_mos: False
  bms_flag_fb_charge_flag: False
  bms_flag_fb_hight_temperature: 25.0
  bms_flag_fb_low_temperature: 25.0
```

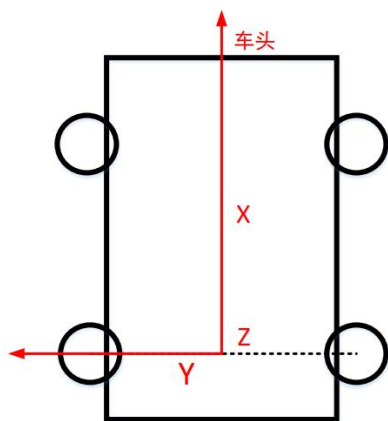
不同的底盘，“chassis_info_fb”话题反馈的内容是不一样的

6、在控制底盘运动之前，我们先来了解下 ROS 系统的坐标体系：

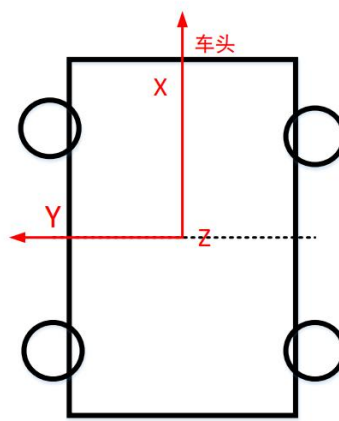
(1) ROS 坐标系使用右手定义



对应到小车底盘上，就是车头为 X 轴正方向，左侧为 Y 轴正方向，垂直向上为 Z 正方向

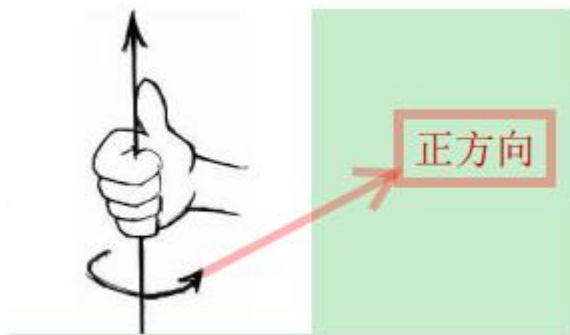


阿克曼车型



差速/四转四驱车型

(2) 旋转轴方向也采用右手定则，向左旋转为正方向，向右旋转为负方向



7、了解了小车底盘的坐标系后，接下来尝试发送命令控制底盘，从第“5”步可以看到存在“/smoother_cmd_vel”话题，通过此话题可以向驱动程序传递控制底盘运动的数据，先看下“/smoother_cmd_vel”话题的消息类型和消息的内容

(1) 打开终端，输入下面指令后敲回车，可以看到有关该话题的详细信息：数据类型、发布者和订阅者的数量等，同样的，查看其他话题的信息，使用“rostopic info /话题名”即可

```
rostopic info /smoother_cmd_vel
```

```
yhs@yhs-ros:~$ rostopic info /smoother_cmd_vel
Type: geometry_msgs/Twist

Publishers: None

Subscribers:
 * /yhs_can_control_node (http://192.168.1.102:34467/)
```

(2) 查看“geometry_msgs/Twist”消息内容,输入下面指令后敲回车，可以看到该消息的所有内容，同样的，查看其消息的内容，使用“rosmmsg show 消息类型”即可

```
rosmmsg show geometry_msgs/Twist
```

```
yhs@yhs-ros:~$ rosmmsg show geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

(3) 消息的部分内容含义如下

geometry_msgs/Vector3 linear		线速度
float64 x	x 轴平移速度，单位为 m/s	
float64 y		
float64 z		
geometry_msgs/Vector3 angular		角速度
float64 x		
float64 y		
float64 z	z 轴旋转速度，单位为 rad/s (如果为阿克曼车型，仅当做前轮转向角度)	

(4) 通过“rqt_publisher”软件工具下发速度，打开终端，输入下面指令敲回车，弹出软件界面后，按照图片上的步骤操作即可，测试时注意线速度不要设置过大，将遥控器切换到指令模式后，底盘按照设置的线速度和角速度运动

```
roslaunch rqt_publisher rqt_publisher
```

```
yhs@yhs-ros:~$ roslaunch rqt_publisher rqt_publisher
```



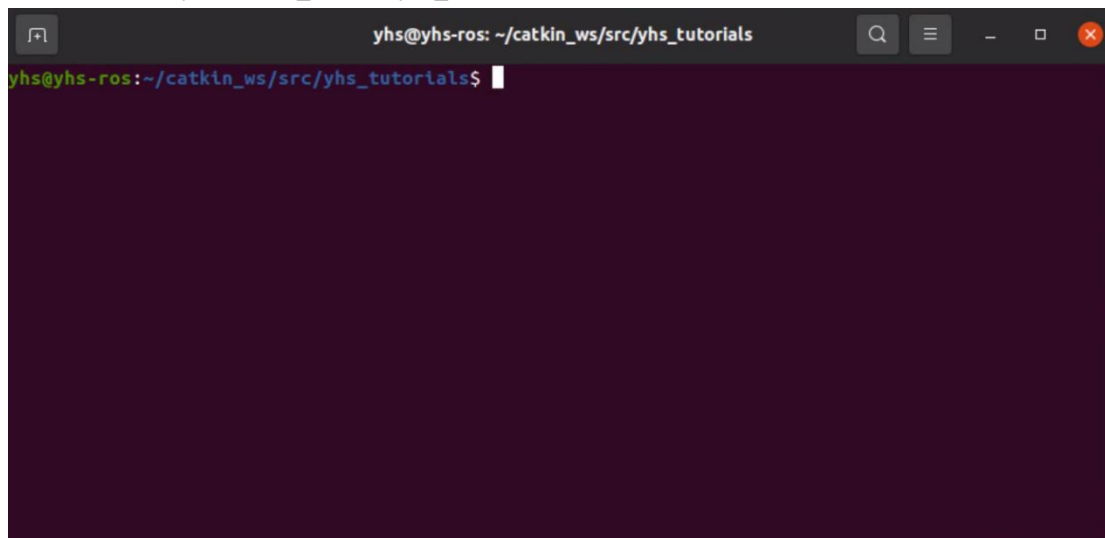
(5) 在终端直接发布速度，打开一个终端，输入“rostopic pub -r 50 /smoother_cmd_vel”指令后，按几次“Tab”键，会自动补全

```
yhs@yhs-ros:~$ rostopic pub -r 50 /smoother_cmd_vel geometry_msgs/Twist "linear:
x: 0.0
y: 0.0
z: 0.0
angular:
x: 0.0
y: 0.0
z: 0.0"
```

补全后，按键盘上的左右箭头移动光标，填入线速度和角速度后，敲回车发布，按“ctrl + c”停止发布

8、通过程序节点下发速度

- (1) 首先，需要新建一个 ROS 源码包 Package。打开终端进入“/home/yhs/catkin_ws/src/yhs_tutorials”目录



- (2) 然后输入如下指令新建一个 ROS 源码包

```
catkin_create_pkg vel_pub roscpp geometry_msgs
```

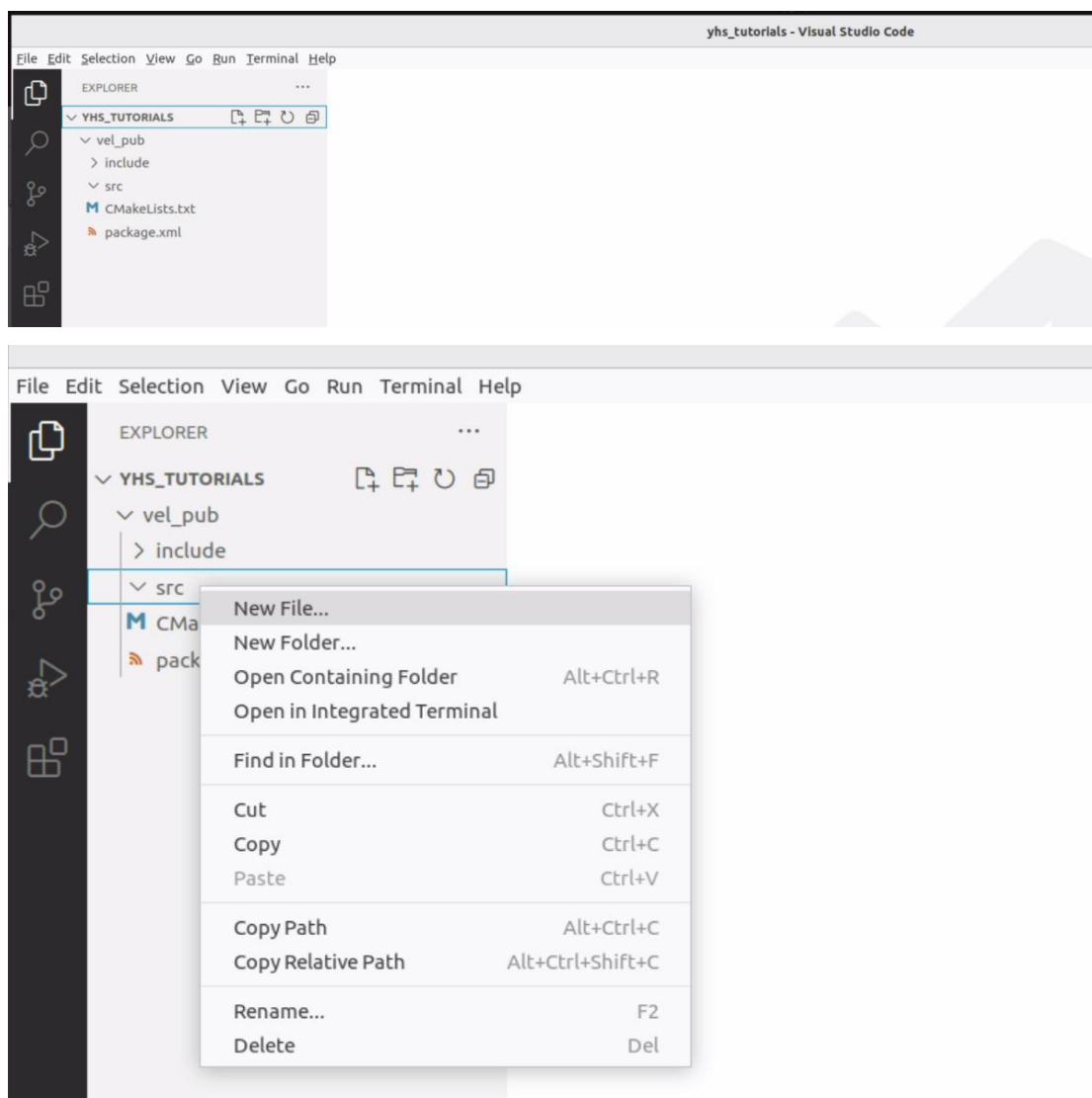
A terminal window showing the execution of the command 'catkin_create_pkg vel_pub roscpp geometry_msgs'. The output shows the creation of package.xml, CMakeLists.txt, and the src directory, followed by a success message.

```
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$ catkin_create_pkg vel_pub roscpp geometry_msgs
Created file vel_pub/package.xml
Created file vel_pub/CMakeLists.txt
Created folder vel_pub/include/vel_pub
Created folder vel_pub/src
Successfully created files in /home/yhs/catkin_ws/src/yhs_tutorials/vel_pub. Please adjust the values in package.xml.
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$
```

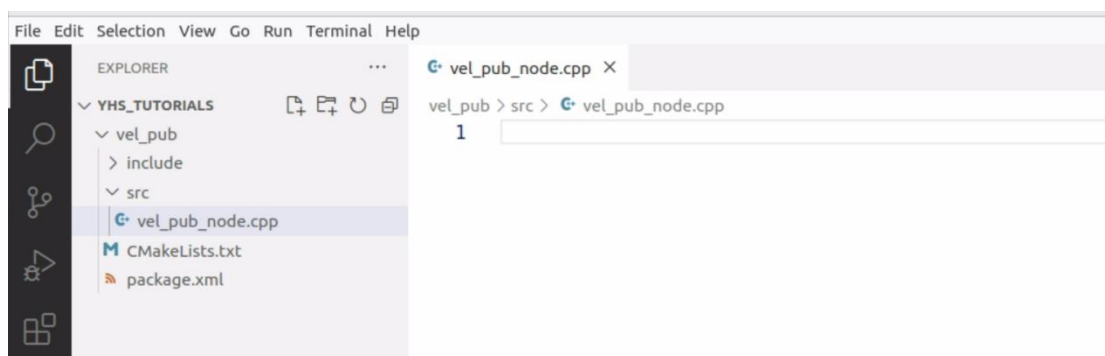
这条指令的具体含义是：

指令	含义
catkin_create_pkg	创建 ROS 源码包（package）的指令
vel_pub	新建的 ROS 源码包命名
roscpp	C++依赖项，本例程使用 C++编写，所以需要这个依赖项
geometry_msgs	包含机器人移动速度消息包格式文件的包名称

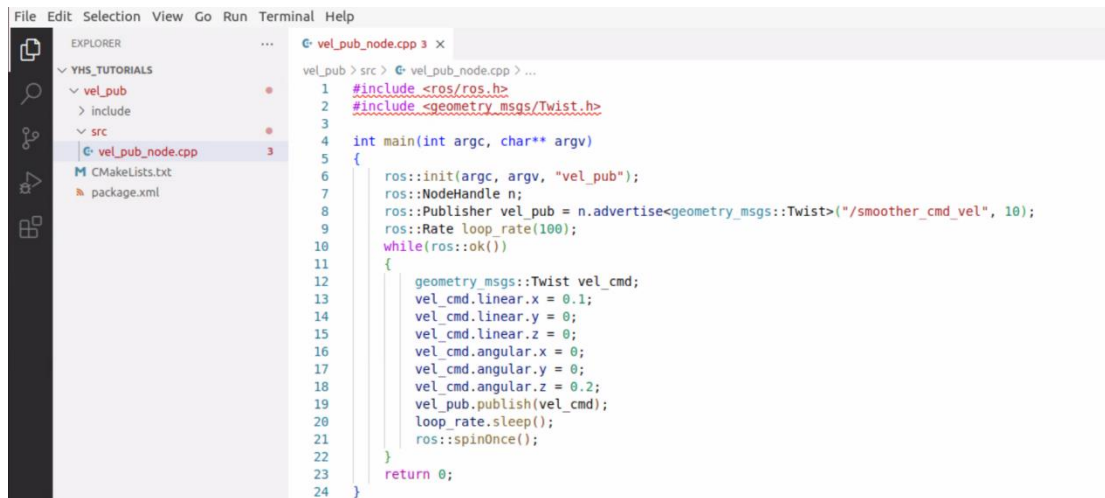
(3) 创建“vel_pub”包后，用 IDE 打开“yhs_tutorials”，可以看到“yhs_tutorials”目录下多了一个“vel_pub”文件夹，在其“src”子文件夹上右键点击鼠标，选择“New File”新建一个代码文件。（这里我们用的是 visual studio code，也可以使用 vim、Text Editor 等其他文本编辑工具打开或创建）



(4) 新建的代码文件，我们为其命名为“vel_pub_node.cpp”。



(5) 命名完毕后，在 IDE 的右侧可以开始编写 “vel_pub_node.cpp” 的代码，其内容下：



```
1 #include <ros/ros.h>
2 #include <geometry_msgs/Twist.h>
3
4 int main(int argc, char** argv)
5 {
6     ros::init(argc, argv, "vel_pub");
7     ros::NodeHandle n;
8     ros::Publisher vel_pub = n.advertise<geometry_msgs::Twist>("/smoother_cmd_vel", 10);
9     ros::Rate loop_rate(100);
10    while(ros::ok())
11    {
12        geometry_msgs::Twist vel_cmd;
13        vel_cmd.linear.x = 0.1;
14        vel_cmd.linear.y = 0;
15        vel_cmd.linear.z = 0;
16        vel_cmd.angular.x = 0;
17        vel_cmd.angular.y = 0;
18        vel_cmd.angular.z = 0.2;
19        vel_pub.publish(vel_cmd);
20        loop_rate.sleep();
21        ros::spinOnce();
22    }
23    return 0;
24 }
```

```
#include <ros/ros.h>
#include <geometry_msgs/Twist.h>

int main(int argc, char** argv)
{
    ros::init(argc, argv, "vel_pub_node");
    ros::NodeHandle n;
    ros::Publisher vel_pub = n.advertise<geometry_msgs::Twist>("/smoother_cmd_vel", 10);
    ros::Rate loop_rate(100);
    while(ros::ok())
    {
        geometry_msgs::Twist vel_cmd;
        vel_cmd.linear.x = 0.1;
        vel_cmd.linear.y = 0;
        vel_cmd.linear.z = 0;
        vel_cmd.angular.x = 0;
        vel_cmd.angular.y = 0;
        vel_cmd.angular.z = 0.2;
        vel_pub.publish(vel_cmd);
        loop_rate.sleep();
        ros::spinOnce();
    }
    return 0;
}
```

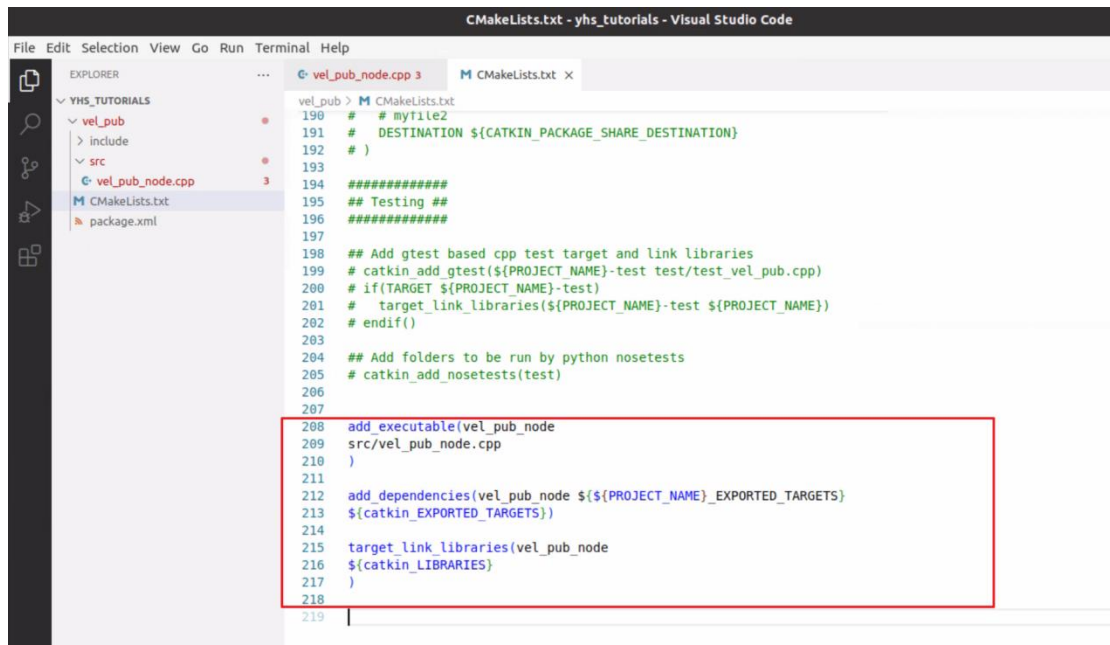
- 1) 代码的开始部分，先 `include` 了两个头文件，一个是 `ros` 的系统头文件，另一个是运动速度结构体类型 `geometry_msgs::Twist` 的定义文件；
- 2) `ROS` 节点的主体函数是 `int main(int argc, char** argv)`，其参数定义和其他 `C++` 程序一样；
- 3) `main` 函数里，首先调用 `ros::init(argc, argv, "vel_pub_node");` 进行该节点的初始化操作，函数的第三个参数是节点名称；
- 4) 接下来声明一个 `ros::NodeHandle` 对象 `n`，并用 `n` 生成一个广播对象 `vel_pub`，调用的参数里指明了 `vel_pub` 将会在主题 `"/smoother_cmd_vel"` 里广播 `geometry_msgs::Twist` 类型的数据。我们对机器人的控制，就是通过这个广播形式实现的；
- 5) 为了连续不断的发送速度，使用一个 `while(ros::ok())` 循环，以 `ros::ok()` 返回值作为循环结束条件可以让循环在程序关闭时正常退出；
- 6) 为了发送速度值，声明一个 `geometry_msgs::Twist` 类型的对象 `vel_cmd`，并将速度值赋值到这个对象里。
- 7) `vel_cmd.linear.x` 是机器人前后平移运动速度，正值往前，负值往后，单位是 `"m/s"`；
- 8) `vel_cmd.angular.z`（注意 `angular`）是机器人自转速度，正值左转，负值右转，单位是 `"rad/s"`；
- 9) `vel_cmd` 赋值完毕后，使用广播对象 `vel_pub` 将其发布到主题 `"/smoother_cmd_vel"` 上去。机器人的底盘驱动节点会从这个主题接收我们发过去的速度值，并转发到硬件底盘去执行；
- 10) 调用 `ros::spinOnce()` 函数给其他回调函数得以执行（本例程未使用回调函数）。
- 11) `vel_cmd` 赋值完毕后，使用广播对象 `publisher` 将其发布到主题 `"/smoother_cmd_vel"` 上去。机器人的底盘驱动节点会从这个主题接收我们发过去的速度值，并通过 `CAN` 卡转发到底盘去执行。

(6) 代码编写完毕，需要将文件名添加到编译文件里才能进行编译。
编译文件在“vel_pub”的目录下，文件名为“CMakeLists.txt”，在 IDE 界面左侧点击该文件，右侧会显示文件内容。在“CMakeLists.txt”文件末尾，为“vel_pub_node.cpp”添加新的编译规则。内容如下：

```
add_executable(vel_pub_node
src/vel_pub_node.cpp
)

add_dependencies(vel_pub_node ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

target_link_libraries(vel_pub_node
${catkin_LIBRARIES}
)
```



(7) 同样，修改完需要按下键盘快捷键 **Ctrl+S** 进行保存，代码上方的文件名右侧的小黑点会变成“X”，说明保存文件成功。下面开始进行代码文件的编译操作，启动一个终端程序，键入如下命令进入 ROS 的工作空间

```
cd /home/yhs/catkin_ws/
```

```
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws/
yhs@yhs-ros:~/catkin_ws$
```

(8) 然后再执行如下命令开始编译

```
catkin_make
```

```
yhs@yhs-ros:~/catkin_ws$ catkin_make
Base path: /home/yhs/catkin_ws
Source space: /home/yhs/catkin_ws/src
Build space: /home/yhs/catkin_ws/build
Devel space: /home/yhs/catkin_ws/devel
Install space: /home/yhs/catkin_ws/install
####
### Running command: "cmake /home/yhs/catkin_ws/src -DCATKIN_DEVEL_PREFIX=/home/yhs/catkin_ws/devel -DCMAKE_INSTALL_PREFIX=/home/yhs/catkin_ws/install -G Unix Makefiles" in "/home/yhs/catkin_ws/build"
###
```

执行这条指令后，会出现滚动的编译信息，直到出现“Linking CXX executable /home/yhs/catkin_ws/devel/lib/vel_pub/vel_pub_node”信息，说明新的 vel_pub_node 节点已经编译成功

```
[ 95%] Built target ukf_localization_node
[ 95%] Built target ekf_localization_node
[ 95%] Built target ekf_localization_nodelet
[ 96%] Built target teb_local_planner
[ 98%] Built target carrot_planner
[ 98%] Built target ukf_localization_nodelet
[ 98%] Built target robot_localization_listener_node
[100%] Built target dwa_local_planner
[100%] Built target planner
[100%] Built target test_optim_node
[100%] Built target move_base
[100%] Built target move_base_node
[100%] Linking CXX executable /home/yhs/catkin_ws/devel/lib/vel_pub/vel_pub_node
[100%] Built target vel_pub_node
yhs@yhs-ros:~/catkin_ws$
```

(9) 因为我们前面已经启动了底盘 CAN 控制节点，所以直接启动 vel_pub_node 节点。启动终端。输入以下指令

```
roslaunch vel_pub vel_pub_node
```

```
yhs@yhs-ros:~/catkin_ws$ roslaunch vel_pub vel_pub_node
```

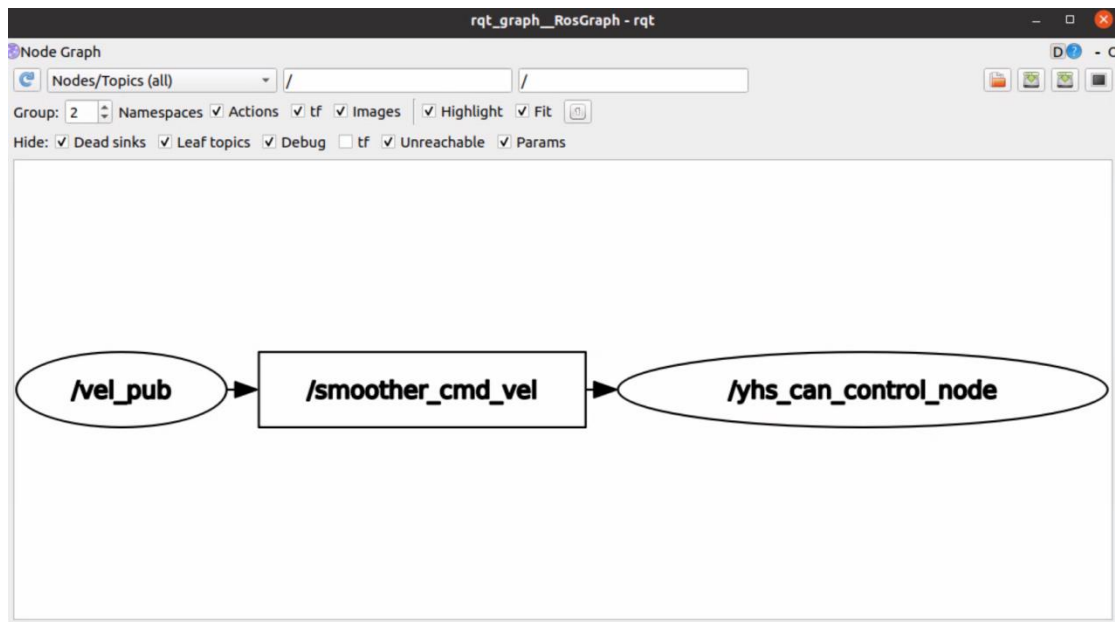
注意这里我们使用的是“roslaunch”命令而不是之前的“roslaunch”。“roslaunch”是启动单个 ROS 节点的指令。按下回车键后，可以看到机器人以 0.1m/s 的线速度和 0.2rad/s 的角速度缓慢运动。

(10) 我们可以通过指令查看 ROS 的节点网络状况，打开终端，输入指令

```
rqt_graph
```

```
yhs@yhs-ros:~$ rqt_graph
```

按下回车键，便会弹出一个窗口，显示当前 ROS 系统里的节点网络情况。



可以看到，我们编写的 `vel_pub_node` 节点，通过主题 `“/smoother_cmd_vel”` 向底盘控制节点发送速度消息。`yhs_can_control_node` 节点获得速度消息后，通过 CAN 通讯方式将其发送到底盘的 VCU，控制底盘运动。

实验三 IMU 数据处理

在小车运动过程中，如果仅仅靠底盘轮子编码上传数据来计算旋转的角度，累计误差会越来越大，所以需要一个小精度的 IMU 来提供角度、加速度等信息。

1、运行 IMU 驱动程序包，输入指令敲回车

```
roslaunch serial_imu imu_msg.launch
```

```
yhs@yhs-ros:~$ roslaunch serial_imu imu_msg.launch
... logging to /home/yhs/.ros/log/9f2b8dec-221b-11ef-8593-6da3ed79c16a/roslaunch-yhs-ros-17949.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:36493/

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.16.0

NODES
/
  HI226 (serial_imu/serial_imu)
  imu_broadcaster (tf2_ros/static_transform_publisher)

auto-starting new master
process[master]: started with pid [17957]
ROS_MASTER_URI=http://192.168.1.102:11311

setting /run_id to 9f2b8dec-221b-11ef-8593-6da3ed79c16a
process[rosout-1]: started with pid [17967]
started core service [/rosout]
process[HI226-2]: started with pid [17970]
process[imu_broadcaster-3]: started with pid [17972]
[ INFO] [1717468750.654202551]: /dev/imu is opened.
```

2、查看 IMU 节点的话题，输入指令后敲回车，可以看到 IMU 数据话题名为 “/imu_data”

```
rostopic list
```

```
yhs@yhs-ros:~$ rostopic list
/imu_data
/rosout
/rosout_agg
/rpy
/tf_static
yhs@yhs-ros:~$
```

3、查看 “/imu_data” 话题信息

(1) 输入指令后敲回车，可以看到 “/imu_data” 话题的消息类型为 “sensor_msgs/Imu”

```
rostopic info /imu_data
```

```
yhs@yhs-ros:~$ rostopic info /imu_data
Type: sensor_msgs/Imu

Publishers:
* /HI226 (http://192.168.1.102:42825/)

Subscribers: None

yhs@yhs-ros:~$
```

(2) 查看 “sensor_msgs/Imu” 消息的内容，输入指令后回车

```
rosmmsg show sensor_msgs/Imu
```

```
yhs@yhs-ros:~$ rosmmsg show sensor_msgs/Imu
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
geometry_msgs/Quaternion orientation
  float64 x
  float64 y
  float64 z
  float64 w
float64[9] orientation_covariance
geometry_msgs/Vector3 angular_velocity
  float64 x
  float64 y
  float64 z
float64[9] angular_velocity_covariance
geometry_msgs/Vector3 linear_acceleration
  float64 x
  float64 y
  float64 z
float64[9] linear_acceleration_covariance
```

部分内容的含义如下：

std_msgs/Header header	
uint32 seq	
time stamp	
string frame_id	
geometry_msgs/Quaternion orientation	四元数
float64 x	
float64 y	
float64 z	
float64 w	
float64[9] orientation_covariance	四元数协方差
geometry_msgs/Vector3 angular_velocity	角速度
float64 x	
float64 y	
float64 z	
float64[9] angular_velocity_covariance	角速度协方差
geometry_msgs/Vector3 linear_acceleration	线加速度
float64 x	
float64 y	
float64 z	
float64[9] linear_acceleration_covariance	线加速度协方差

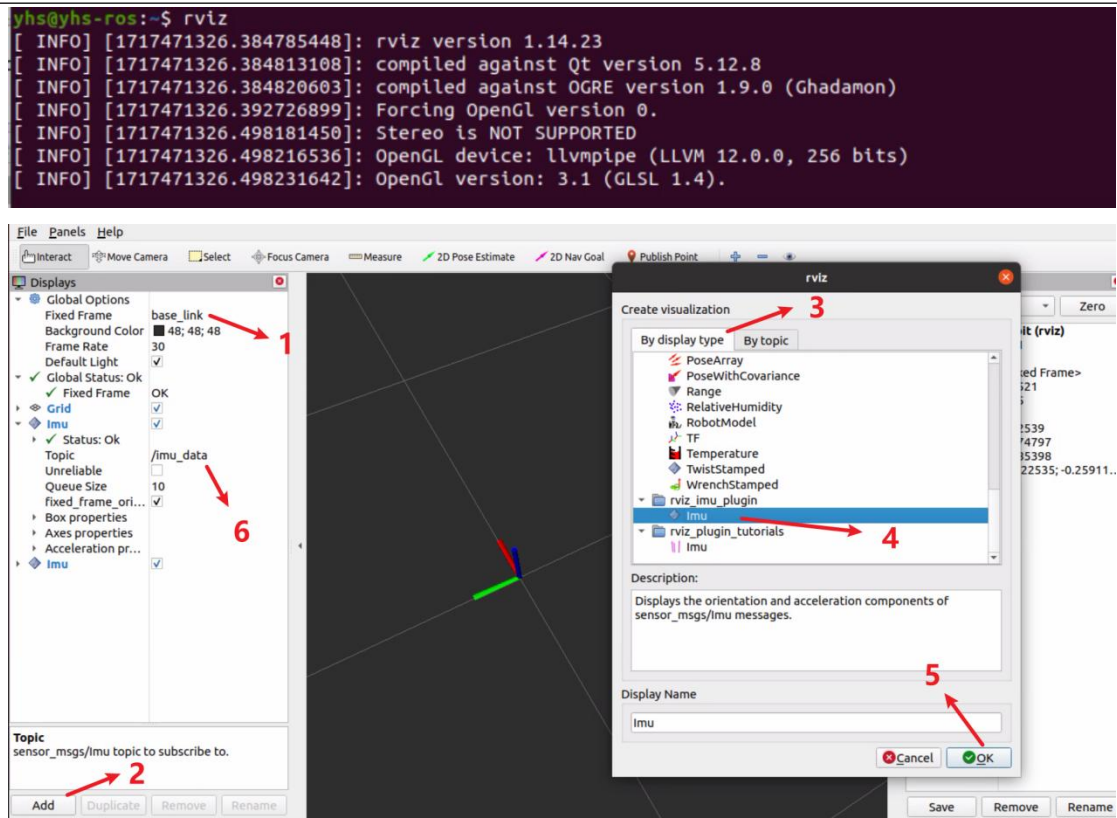
(3) 输入指令查看 “/imu_data” 话题，小车水平放置静止时候的数据，可以看到 IMU 的三个轴的角速度角速度都是零，X 轴和 Y 轴的角速度接近于零，Z 轴加速度约为 9.7 m/s²

```
rostopic echo /imu_data
```

```
yhs@yhs-ros:~$ rostopic echo /imu_data
header:
  seq: 114110
  stamp:
    secs: 1717469893
    nsecs: 329065760
  frame_id: "imu_link"
orientation:
  x: -0.00255478429608047
  y: 0.0059074051678180695
  z: -9.647007391322404e-05
  w: 0.9999792575836182
orientation_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
angular_velocity:
  x: -0.0017453290260074287
  y: 0.0017453290260074287
  z: 0.0
angular_velocity_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
linear_acceleration:
  x: -0.15680000744760036
  y: -0.11760000102221967
  z: 9.731399714946747
linear_acceleration_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

(4) IMU 数据可视化查看，打开另外一个输入以下指令后敲回车，在 Rviz 上按照下图中的步骤操作，添加 IMU 插件，添加完成后在右边黑色方格内可以看到坐标轴，鼠标的滚轮可以放大缩小

rviz



黑色网格上的三个轴，红色为 x 轴，绿色为 y 轴，蓝色为 z 轴，然后我们遥控小车原地向左旋转 90 度，再向右旋转 90 度，观察三个轴的方向是否跟初始一样，以此来简单判断 IMU 的旋转精度。

实验四 里程计融合

Gmapping 和 Cartographer 建图，地图精度很大程度上依赖里程计，所以输出高精度的里程计供建图程序订阅，是建出大面积且闭环地图的关键。

1、先了解里程计的消息类型，在前面的底盘控制实验中，我们可以看到“odom”话题，这个就是根据底盘上传的线速度结合 IMU 的角度计算并发布出来的里程计数据

(1) 打开终端，运行底盘驱动程序

```
roslaunch yhs_can_control yhs_can_control.launch
```

```
NODES
/
  yhs_can_control_node (yhs_can_control/yhs_can_control_node)

auto-starting new master
process[roscpp-1]: started with pid [18485]
ROS_MASTER_URI=http://192.168.1.102:11311

setting /run_id to b499840a-2233-11ef-8593-6da3ed79c16a
process[roscpp-1]: started with pid [18495]
started core service [/roscpp]
process[yhs_can_control_node-2]: started with pid [18502]
[ INFO] [1717479094.617996846]: Chassis type: FW
[ INFO] [1717479094.618557145]: yhs_can_control_node initialized successfully
```

(2) 打开另外一个终端，列举所有话题，看到“/odom”

```
rostopic list
```

```
yhs@yhs-ros:~$ rostopic list
/angular
/chassis_info_fb
/fw_ctrl_cmd
/imu_data
/io_cmd
/linear
/odom
/roscpp
/roscpp_agg
/scan3
/smooth_cmd_vel
/tf
/ultrasonic
yhs@yhs-ros:~$
```

(3) 查看话题消息类型，可以看到消息类型为“nav_msgs/Odometry”

```
rostopic info /odom
```

```
yhs@yhs-ros:~$ rostopic info /odom
Type: nav_msgs/Odometry

Publishers:
 * /yhs_can_control_node (http://192.168.1.102:45897/)

Subscribers: None
```

(4) 查看消息的内容

```
rosmmsg show nav_msgs/Odometry
```

```
yhs@yhs-ros:~$ rosmmsg show nav_msgs/Odometry
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
string child_frame_id
geometry_msgs/PoseWithCovariance pose
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
  float64[36] covariance
geometry_msgs/TwistWithCovariance twist
  geometry_msgs/Twist twist
    geometry_msgs/Vector3 linear
      float64 x
      float64 y
      float64 z
    geometry_msgs/Vector3 angular
      float64 x
      float64 y
      float64 z
  float64[36] covariance
```

“nav_msgs/msg/Odometry” 消息的部分内容含义如下：

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
string child_frame_id
geometry_msgs/PoseWithCovariance pose
  geometry_msgs/Pose pose          # 机器人实时坐标，只用到 x,y 轴
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation  # 机器人实时方向
      float64 x
      float64 y
      float64 z
      float64 w
  float64[36] covariance          # 协方差
geometry_msgs/TwistWithCovariance twist
  geometry_msgs/Twist twist
    geometry_msgs/Vector3 linear      # 机器人实时线速度
      float64 x
      float64 y
      float64 z
    geometry_msgs/Vector3 angular      # 机器人实时角速度
```

float64 x	
float64 y	
float64 z	
float64[36] covariance	# 协方差

(5) 在调试建图或者导航时可能要查看“odom”数据，输入指令敲回车

```
rostopic echo /odom
```

```
yhs@yhs-ros:~$ rostopic echo /odom
header:
  seq: 76855
  stamp:
    secs: 1717479863
    nsecs: 317833234
  frame_id: "odom"
child_frame_id: "base_link"
pose:
  pose:
    position:
      x: 0.0
      y: 0.0
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.0
      w: 1.0
  covariance: [0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 10000000000.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 10000000000.0, 0.0,
0.0, 0.0, 0.0, 0.0, 10000000000.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.2]
twist:
  twist:
    linear:
      x: 0.0
      y: 0.0
      z: 0.0
    angular:
      x: 0.0
      y: 0.0
      z: 0.0
  covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

- 2、知道了里程计“odom”的消息类型，现在通过程序来发布该话题数据，下面是“/home/yhs/catkin_ws/src/yhs_can_control/src/yhs_XX_can_control_node.cpp”文件中发布里程计的函数代码，这里直接把 IMU 的航向角度用作机器人的航向角，所以 IMU 的航向角精度必须要高

```
void FwCanControl::OdomPub(const float linear, const float angular, const unsigned char gear,
const float slipangle)
{
    static double x = 0.0;
    static double y = 0.0;
    static double th = 0.0;

    static double lastYaw = 0;

    static tf2_ros::TransformBroadcaster odom_broadcaster;

    static ros::Time last_time = ros::Time::now();
    ros::Time current_time;

    double vx = linear;
    double vth = angular;

    current_time = ros::Time::now();
    double dt = (current_time - last_time).toSec();
```

```
static bool imu_published = false;
if(!imu_published)
{
    imu_published = hasPublisher("/imu_data");
}
else
{
    ROS_INFO_ONCE("Get imu_data");
}

if(gear == 6)
{
    x += vx * cos(th) * dt;
    y += vx * sin(th) * dt;

    if (!imu_published)
    {
        th += vth * dt;
    }
    else
    {
        std::lock_guard<std::mutex> lock(mutex_);
        th = imu_yaw_;
    }
}

else if(gear == 7)
{
    double theta = th + slipangle;
    if(vx >= 0)
    {
        x += vx * cos(theta) * dt;
        y += vx * sin(theta) * dt;
    }
    else
    {
        theta = th + slipangle + M_PI;
        x += -vx * cos(theta) * dt;
        y += -vx * sin(theta) * dt;
    }

    if (!imu_published)
    {}
}
```

```
else
{
    std::lock_guard<std::mutex> lock(mutex_);
    th = imu_yaw_;
}
}

tf2::Quaternion quat;
quat.setRPY(imu_roll_, imu_pitch_, th);
geometry_msgs::Quaternion odom_quat = tf2::toMsg(quat);

geometry_msgs::TransformStamped odom_trans;
odom_trans.header.stamp = current_time;
odom_trans.header.frame_id = odomFrame_;
odom_trans.child_frame_id = baseFrame_;

odom_trans.transform.translation.x = x;
odom_trans.transform.translation.y = y;
odom_trans.transform.translation.z = 0.0;
odom_trans.transform.rotation = odom_quat;

if (tfUsed_)
    odom_broadcaster.sendTransform(odom_trans);

nav_msgs::Odometry odom;
odom.header.stamp = current_time;
odom.header.frame_id = odomFrame_;

odom.pose.pose.position.x = x;
odom.pose.pose.position.y = y;
odom.pose.pose.position.z = 0.0;
odom.pose.pose.orientation = odom_quat;

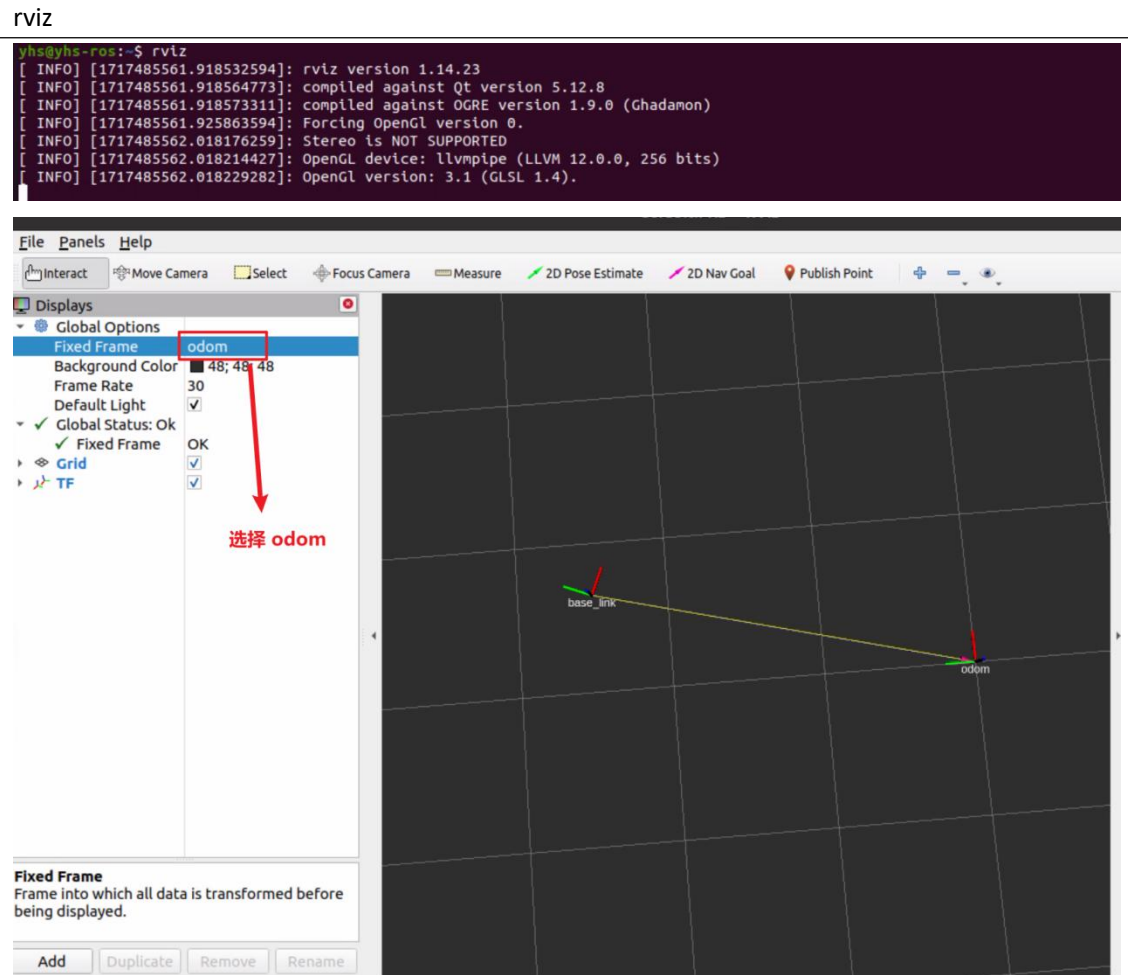
odom.child_frame_id = baseFrame_;
odom.twist.twist.linear.x = vx;
odom.twist.twist.linear.y = 0.0;
odom.twist.twist.angular.z = vth;

odom_pub_.publish(odom);

last_time = current_time;
}
```

3、对比测试，“odom”里程计话题数据使用 IMU 航向角和不使用航向角的精度

(1) 不使用 IMU 航向角，使用机器人的角速度计算机器人的航向角，不用运行 IMU 节点，用遥控器控制底盘行走，过程中尽可能的旋转，一段时间后，回到起点，机器人（base_link）相对于起点（odom）的位置和方向，可以看到回到起点后，在 Rviz 上“base_link”与“odom”的距离和方向相差较大



(2) 把 IMU 航向角当做机器人的航向角，先把底盘控制节点关闭再运行，运行 IMU 节点，用遥控器控制底盘行走，过程中尽可能的旋转，一段时间后，回到起点，机器人 (base_link) 相对于起点 (odom) 的位置和方向，可以看到回到起点后，在 Rviz 上 “base_link” 与 “odom” 的方向相差很小，X 和 Y 轴位置偏差较小，可用于 gmapping 和 cartographer 建图

```
roslaunch serial_imu imu_spec_msg.launch
```

```
yhs@yhs-ros:~$ roslaunch serial_imu imu_spec_msg.launch
... logging to /home/yhs/.ros/log/847e60f0-2243-11ef-8593-6da3ed79c16a/roslaunch-yhs-ros-50276.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

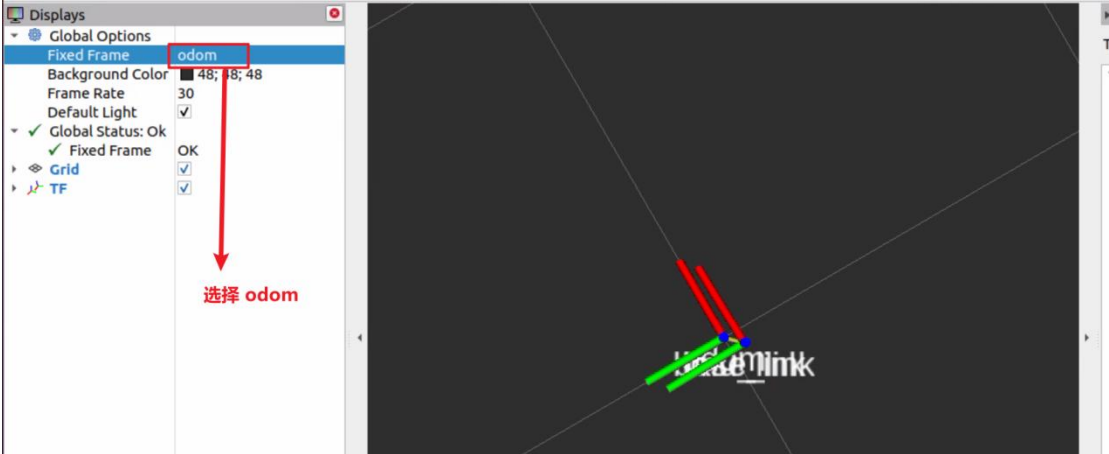
started roslaunch server http://192.168.1.102:39097/

SUMMARY
=====
PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.16.0
NODES
/
  HI226 (serial_imu/serial_imu)
  imu_broadcaster (tf2_ros/static_transform_publisher)

auto-starting new master
process[master]: started with pid [50284]
ROS_MASTER_URI=http://192.168.1.102:11311

setting /run_id to 847e60f0-2243-11ef-8593-6da3ed79c16a
process[rosout-1]: started with pid [50294]
started core service [/rosout]
process[HI226-2]: started with pid [50297]
process[imu_broadcaster-3]: started with pid [50299]
[ INFO] [1717485885.772724968]: /dev/imu is opened.

started core service [/rosout]
process[yhs_can_control_node-2]: started with pid [50197]
[ INFO] [1717485556.581152452]: Chassis type: FW
[ INFO] [1717485556.581685066]: yhs can control_node initialized successfully
^C[yhs_can_control_node-2] killing on exit
yhs_can_control_node stopped
[rosout-1] killing on exit
[master] killing on exit
shutting down processing monitor...
... shutting down processing monitor complete
done
yhs@yhs-ros:~/catkin_ws$
yhs@yhs-ros:~/catkin_ws$ roslaunch yhs_can_control yhs_can_control.launch
... logging to /home/yhs/.ros/log/847e60f0-2243-11ef-8593-6da3ed79c16a/roslaunch-yhs-ros-50313.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.
```

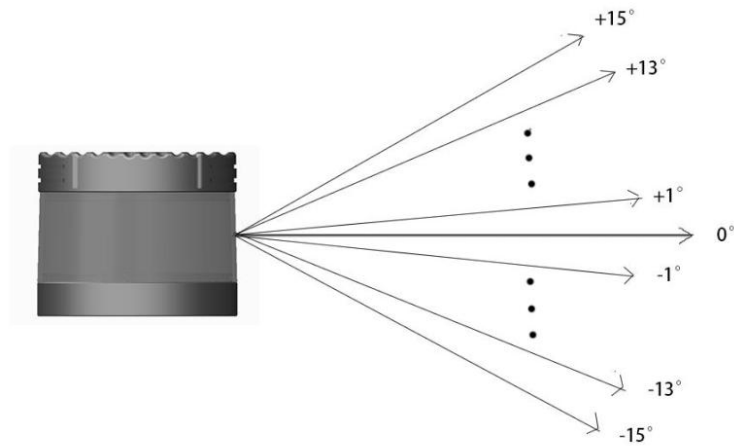


4、这里是直接把 IMU 的航向角当做机器人的航向角，还可以用 “robot_localization” 来融合计算里程计，感兴趣可以参考

“https://navigation.ros.org/setup_guides/odom/setup_odom.html”。

实验五 3D 激光雷达

激光雷达是地面移动机器人常用的一种传感器，我们采用的 16 线激光雷达是使用脉冲飞行时间法（Pulsed ToF）进行距离测量，16 线激光雷达装有 16 对激光发射接收模组电机一般以 10hz 转速驱动进行 360 度扫描。



1、首先我们通过一个简单实验大概体验激光雷达的传感器特性。

(1) 打开一个终端，输入指令后回车启动激光雷达节点

```
roslaunch timoo_pointcloud TM16.launch
```

```
yhs@yhs-ros:~$ roslaunch timoo_pointcloud TM16.launch
... logging to /home/yhs/.ros/log/0fc226f4-224b-11ef-b9bf-e7da4213ad17/roslaunch-yhs-ros-14111.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

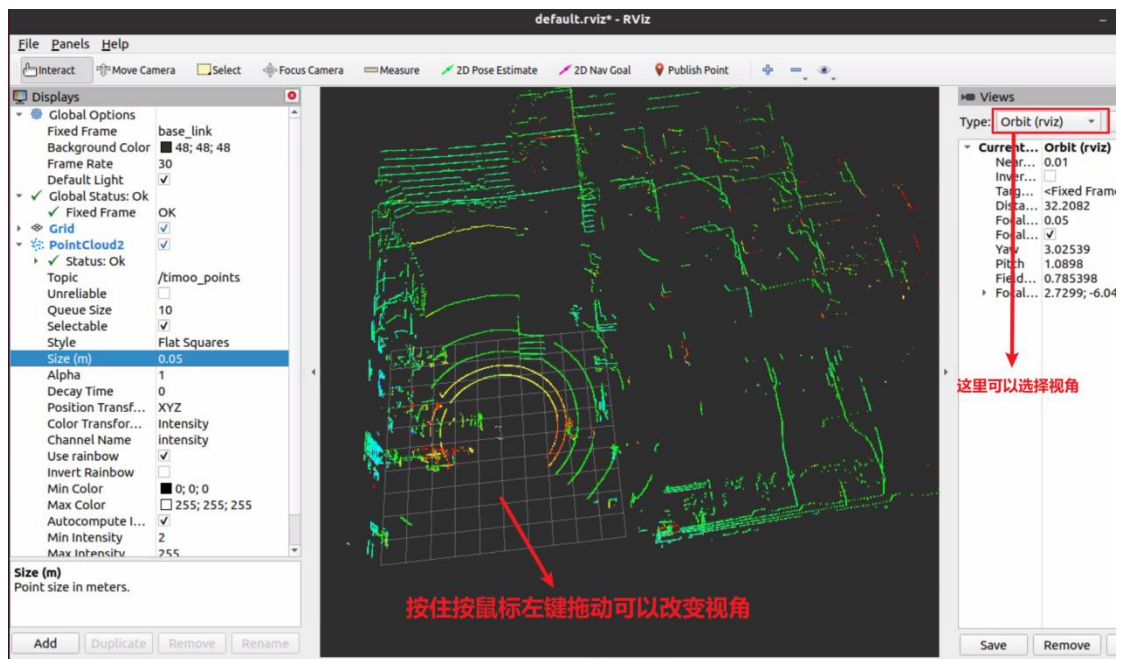
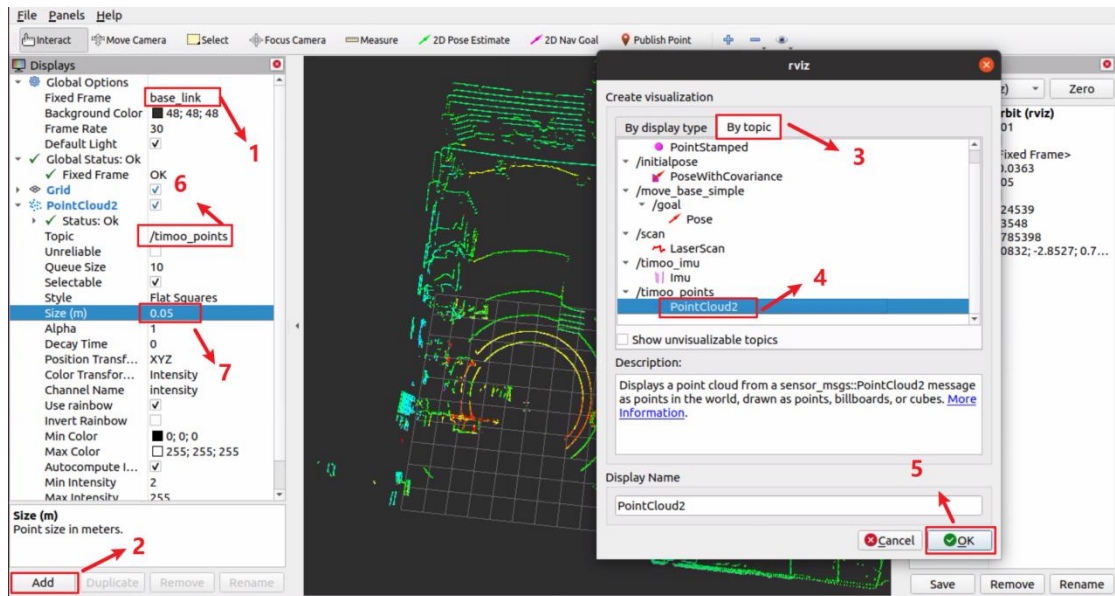
started roslaunch server http://192.168.1.102:39371/

SUMMARY
=====

setting /run_id to 0fc226f4-224b-11ef-b9bf-e7da4213ad17
process[rosout-1]: started with pid [14131]
started core service [/rosout]
process[laser_link_nodelet_manager-2]: started with pid [14138]
process[laser_link_nodelet_manager_driver-3]: started with pid [14139]
process[laser_link_nodelet_manager_cloud-4]: started with pid [14140]
process[laser_link_to_base_link-5]: started with pid [14141]
process[pointcloud_to_laserscan-6]: started with pid [14147]
[ INFO] [1717489126.266070277]: Initializing nodelet with 8 worker threads.

[ INFO] [1717489126.288291901]: correction angles: /home/yhs/catkin_ws/src/hardware/timoo/timoo_pointcloud/params/tm16.yaml
[ INFO] [1717489126.290532814]: Number of lasers: 16.
[ INFO] [1717489126.292434218]: Initialized container with min_range: 0.15, max_range: 50, target_frame: timoo, fixed_frame: laser_link
, init_width: 0, init_height: 1, is_dense: 1, scans_per_packet: 384
[ INFO] [1717489126.317811633]: Reconfigure Request
[ INFO] [1717489126.317869732]: Initialized container with min_range: 0.15, max_range: 50, target_frame: laser_link, fixed_frame: laser
_link, init_width: 0, init_height: 1, is_dense: 1, scans_per_packet: 384
[ INFO] [1717489126.347265708]: timoo TM-C16 rotating at 600 RPM
[ INFO] [1717489126.347534780]: publishing 85 packets per scan
[ INFO] [1717489126.348039683]: Setting timoo scan start time to timestamp of first packet
[ INFO] [1717489126.348855130]: Cut at specific angle feature deactivated.
[ INFO] [1717489126.353214425]: Reconfigure Request
[ INFO] [1717489126.353866888]: expected frequency: 9.976 (Hz)
[ INFO] [1717489126.355418195]: Opening UDP socket: port 2368
[ INFO] [1717489126.356681784]: Opening UDP socket: port 8603
```

(2) 在另外一个终端上打开 Rviz，可以在 Rviz 中看到机器人周围彩色的点云，这就是激光雷达采集到的障碍点数据。Rviz 里的地面基准是一个栅格图案，每一格代表 1 米距离，我们可以试着从这个点集里观察它们和周围障碍物的关系。因为传感器通常都存在测量误差，所以这些点云会在一个很小的范围内来回跳动，可以绕机器人走动，观察激光雷达点云的变化情况。



(3) 查看激光雷达节点发布的“topic”，输入命令后敲回车，找到“/tmlidar_points”激光雷达话题

```
rostopic list
```

```
yhs@yhs-ros:~$ rostopic list
/diagnostics
/laser_link_nodelet_manager/bond
/laser_link_nodelet_manager_cloud/parameter_descriptions
/laser_link_nodelet_manager_cloud/parameter_updates
/laser_link_nodelet_manager_driver/parameter_descriptions
/laser_link_nodelet_manager_driver/parameter_updates
/rosout
/rosout_agg
/scan
/tf
/tf_static
/timoo_imu
/timoo_packets
/timoo_points
/timoo_status
```

(4) 查看“/tmlidar_points”话题的信息，输入指令后敲回车，可以看到“/tmlidar_points”话题的消息类型是“sensor_msgs/PointCloud2”

```
rostopic info /tmlidar_points
```

```
yhs@yhs-ros:~$ rostopic info /timoo_points
Type: sensor_msgs/PointCloud2

Publishers:
 * /laser_link_nodelet_manager (http://192.168.1.102:43475/)

Subscribers: None
```

(5) 查看“sensor_msgs/msg/PointCloud2”的内容，输入指令后敲回车

```
rosmmsg show sensor_msgs/PointCloud2
```

```
yhs@yhs-ros:~$ rosmmsg show sensor_msgs/PointCloud2
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
uint32 height
uint32 width
sensor_msgs/PointField[] fields
  uint8 INT8=1
  uint8 UINT8=2
  uint8 INT16=3
  uint8 UINT16=4
  uint8 INT32=5
  uint8 UINT32=6
  uint8 FLOAT32=7
  uint8 FLOAT64=8
  string name
  uint32 offset
  uint8 datatype
  uint32 count
bool is_bigendian
uint32 point_step
uint32 row_step
uint8[] data
bool is_dense
```

2、下面我们通过编写代码，获取激光雷达的具体测量数值

(1) 打开一个终端程序，输入指令进入 “/home/yhs/catkin_ws/src/yhs_tutorials” 目录

```
cd /home/yhs/catkin_ws/src/yhs_tutorials
```

```
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws/src/yhs_tutorials/
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$
```

(2) 使用如下指令创建一个新的 ROS 软件包 (Package)

```
catkin_create_pkg tm_lidar_pkg roscpp std_msgs sensor_msgs pcl_ros
```

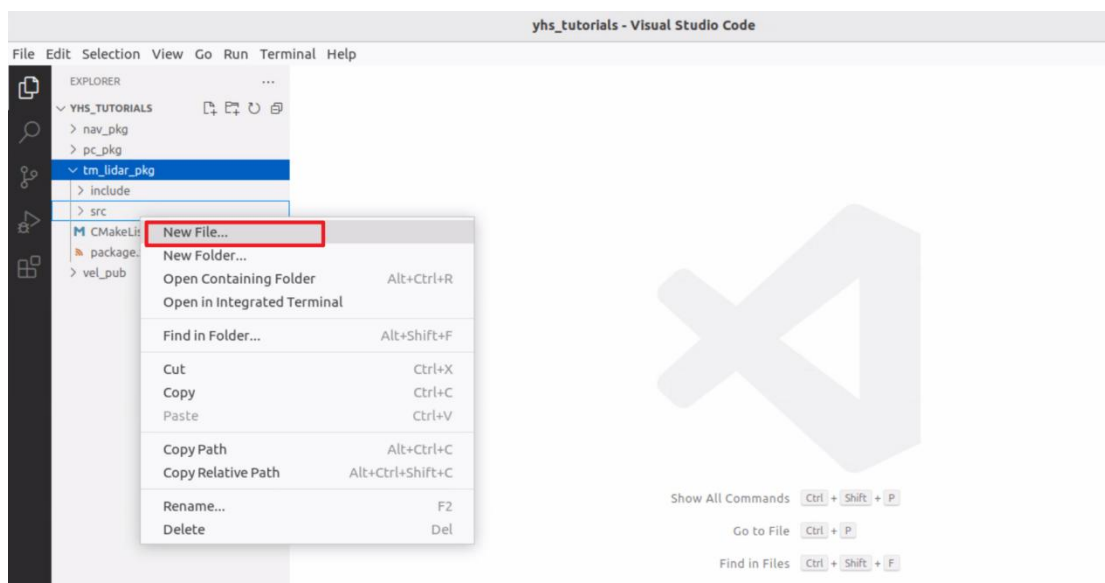
```
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$ catkin_create_pkg tm_lidar_pkg roscpp std_msgs sensor_msgs pcl_ros
Created file tm_lidar_pkg/package.xml
Created file tm_lidar_pkg/CMakeLists.txt
Created folder tm_lidar_pkg/include/tm_lidar_pkg
Created folder tm_lidar_pkg/src
Successfully created files in /home/yhs/catkin_ws/src/yhs_tutorials/tm_lidar_pkg. Please adjust the values in package.xml.
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$
```

这条指令的具体含义是：

指令	含义
catkin_create_pkg	创建 ROS 源码包 (package) 的指令
tm_lidar_pkg	新建的 ROS 源码包命名
roscpp	C++依赖项，本例程使用 C++编写，所以需要这个依赖项
std_msgs	标准消息依赖项，需要里面的 String 格式做文字输出
sensor_msgs	传感器消息依赖项，激光雷达数据格式需要此项
pcl_ros	ROS 里的开源点云库 PCL 的依赖项

(3) 创建 “tm_lidar_pkg” 包后，用 IDE 打开 “yhs_tutorials”，可以看到 “yhs_tutorials” 目录下多了一个 “tm_lidar_pkg” 文件夹，在其 “src” 子文件夹上右键点击鼠标，选择 “New File” 新建一个代码文件。（这里我们用的是 visual studio code，也可以使用 vim、Text Editor 等其他文本编辑工具打开或创建）

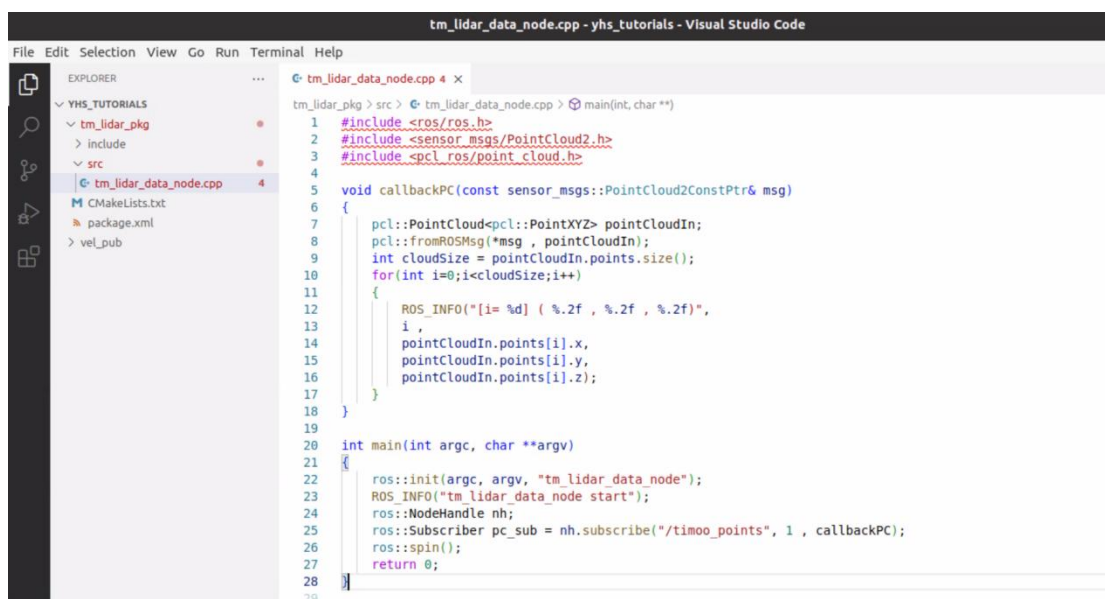




(4) 新建的代码文件，我们为其命名为“tm_lidar_data_node.cpp”。



(5) 命名完毕后，在 IDE 的右侧可以开始编写“tm_lidar_data_node.cpp”的代码，其内容如下：



```
#include <ros/ros.h>
#include <sensor_msgs/PointCloud2.h>
#include <pcl_ros/point_cloud.h>

void callbackPC(const sensor_msgs::PointCloud2ConstPtr& msg)
{
    pcl::PointCloud<pcl::PointXYZ> pointCloudIn;
    pcl::fromROSMsg(*msg, pointCloudIn);
    int cloudSize = pointCloudIn.points.size();
    for(int i=0;i<cloudSize;i++)
    {
        ROS_INFO("[i= %d] ( %.2f , %.2f , %.2f)",
            i,
            pointCloudIn.points[i].x,
            pointCloudIn.points[i].y,
            pointCloudIn.points[i].z);
    }
}

int main(int argc, char **argv)
{
    ros::init(argc, argv, "tm_lidar_data_node");
    ROS_INFO("tm_lidar_data_node start");
    ros::NodeHandle nh;
    ros::Subscriber pc_sub = nh.subscribe("/timoo_points", 1, callbackPC);
    ros::spin();
    return 0;
}
```

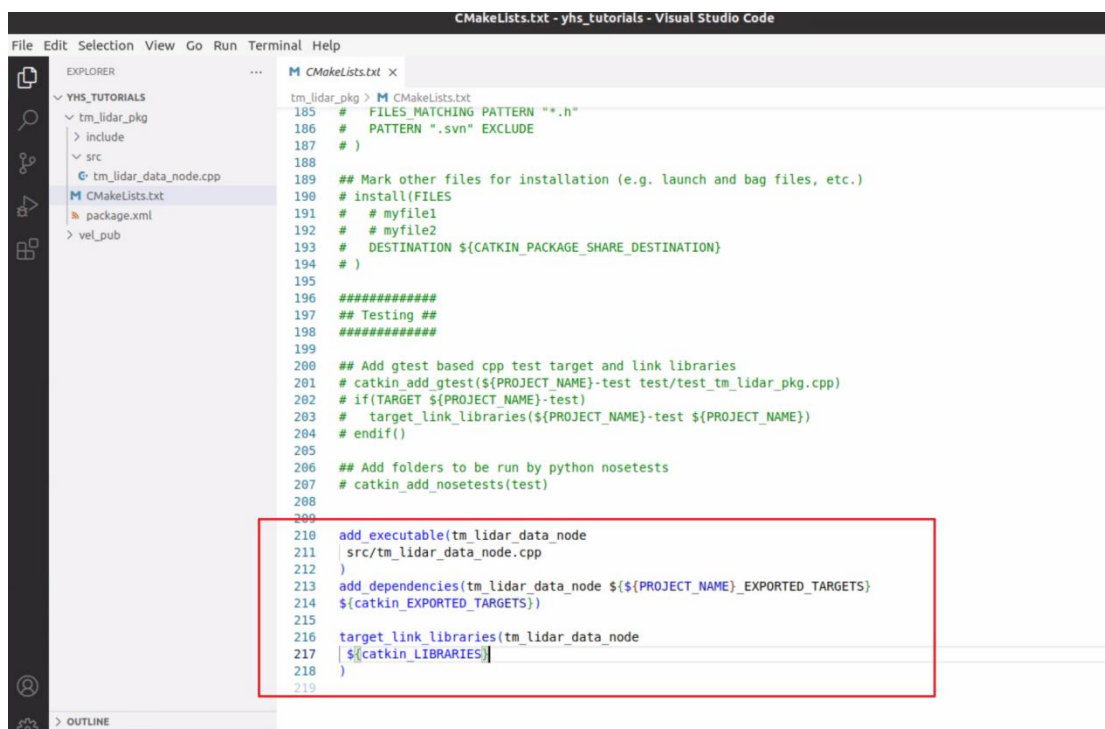
- 1) 代码的开头 include 了三个头文件：ros.h 是 ROS 系统头文件；sensor_msgs/PointCloud2.h 是 ROS 里的点云类型头文件；pcl_ros/point_cloud.h 是 PCL 里的点云类型头文件；
- 2) 定义一个回调函数 void callbackPC()，用来处理三维点云数据。ROS 每接收到一帧点云数据，自动调用一次这个回调函数。三维点云数据会以参数的形式传递到这个回调函数里；
- 3) 回调函数 void callbackPC()的参数 msg 是一个 sensor_msgs::PointCloud2 格式指针，其指向的内存区域就是存放三维点云的内存空间。在实际开发中，通常不会直接使用这个格式的点云，而是将其转换成 PCL 的点云格式，这样就可以使用丰富的 PCL 函数来处理点云数据；

- 4) 在回调函数 `void callbackPC()` 中, 定义一个 `pcl::PointXYZ` 格式的点云容器 `pointCloudIn`, 调用 `pcl::fromROSMsg()` 函数将参数里的 ROS 格式点云数据转换成 PCL 格式的点云数据, 存放在容器 `pointCloudIn` 里;
- 5) 获取转换后的点云数组 `pointCloudIn.points` 的三维点数量, 存储在一个变量 `cloudSize` 里。使用一个 `for` 循环, 把的 `pointCloudIn.points` 里所有点的 `x`、`y` 和 `z` 三个值, 通过 `ROS_INFO()` 显示在终端程序里。通常来说, `pointCloudIn.points` 里的原始坐标值不会直接拿来使用, 而是需要转换到机器人坐标系之后, 再用 PCL 点云库的函数来进行处理;
- 6) 在主函数 `main()` 中, 调用 `ros::init()`, 对这个节点进行初始化;
- 7) 调用 `ROS_INFO()` 向终端程序输出字符串信息, 以表明节点正常启动了;
- 8) 定义一个 `ros::NodeHandle` 节点句柄 `nh`, 并使用这个句柄向 ROS 核心节点订阅 Topic 主题 “`/timoo_points`” 的数据, 回调函数设置为之前定义的 `callbackPC()`。这个 “`/timoo_points`” 是激光雷达的 ROS 节点发布三维点云的主题名, 激光雷达采集到的三维点云会发送到这个主题里, 我们自己写的这个节点 `tm_lidar_node` 只需要订阅它就能收到采集到的三维点云;
- 9) 调用 `ros::spin()` 对 `main()` 函数进行阻塞, 保持这个节点程序不会结束退出。

(6) 代码编写完毕, 需要将文件名添加到编译文件里才能进行编译。编译文件在 `tm_lidar_pkg` 的目录下, 文件名为 “`CMakeLists.txt`”, 在 IDE 界面左侧点击该文件, 右侧会显示文件内容。在 “`CMakeLists.txt`” 文件末尾, 为 `tm_lidar_data_node.cpp` 添加新的编译规则。内容如下:

```
add_executable(tm_lidar_data_node
  src/tm_lidar_data_node.cpp
)
add_dependencies(tm_lidar_data_node ${${PROJECT_NAME}_EXPORTED_TARGETS}
  ${catkin_EXPORTED_TARGETS})

target_link_libraries(tm_lidar_data_node
  ${catkin_LIBRARIES}
)
```



(7) 同样，修改完需要按下键盘快捷键 **Ctrl+S** 进行保存，代码上方的文件名右侧的小黑点会变成“X”，说明保存文件成功。下面开始进行代码文件的编译操作，启动一个终端程序，键入如下命令进入 ROS 的工作空间

```
cd /home/yhs/catkin_ws
```

```
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws/
yhs@yhs-ros:~/catkin_ws$
```

(8) 然后再执行如下命令开始编译

```
catkin_make
```

```
yhs@yhs-ros:~/catkin_ws$ catkin_make
Base path: /home/yhs/catkin_ws
Source space: /home/yhs/catkin_ws/src
Build space: /home/yhs/catkin_ws/build
Devel space: /home/yhs/catkin_ws/devel
Install space: /home/yhs/catkin_ws/install
```

执行这条指令后，会出现滚动的编译信息，直到出现“Linking CXX executable /home/yhs/catkin_ws/devel/lib/tm_lidar_pkg/tm_lidar_data_node”信息，说明新的 tm_lidar_data_node 节点已经编译成功

```

[ 96%] Built target trajectory_planner_ros
[ 96%] Built target planner
[ 96%] Built target carrot_planner
[ 96%] Built target rotate_recovery
[ 97%] Built target dwa_local_planner
[ 98%] Built target teb_local_planner
[100%] Built target test_optim_node
[100%] Built target move_base
[100%] Built target move_base
[100%] Linking CXX executable /home/yhs/catkin_ws/devel/lib/tm_lidar_pkg/tm_lidar_data_node
[100%] Built target tm_lidar_data_node
yhs@yhs-ros:~/catkin_ws$

```

4、运行“tm_lidar_data_node”节点

(1) 先运行 3D 激光雷达节点，打开终端，输入指令敲回车，启动相机节点

```
roslaunch timoo_pointcloud TM16.launch
```

```
yhs@yhs-ros:~/catkin_ws$ roslaunch timoo_pointcloud TM16.launch

[ INFO] [1717492513.028524696]: correction angles: /home/yhs/catkin_ws/src/hardware/timoo/timoo_pointcloud/params/tm16.yaml
[ INFO] [1717492513.033759816]: Number of lasers: 16.
[ INFO] [1717492513.035232678]: Initialized container with min_range: 0.15, max_range: 50, target_frame: timoo, fixed_frame: laser_link
, init_width: 0, init_height: 1, is_dense: 1, scans_per_packet: 384
[ INFO] [1717492513.063362733]: Reconfigure Request
[ INFO] [1717492513.063431259]: Initialized container with min_range: 0.15, max_range: 50, target_frame: laser_link, fixed_frame: laser
link, init_width: 0, init_height: 1, is_dense: 1, scans_per_packet: 384
[ INFO] [1717492513.082118100]: timoo TM-C16 rotating at 600 RPM
[ INFO] [1717492513.082558545]: publishing 85 packets per scan
[ INFO] [1717492513.083287027]: Setting timoo scan start time to timestamp of first packet
[ INFO] [1717492513.084802338]: Cut at specific angle feature deactivated.
[ INFO] [1717492513.089325606]: Reconfigure Request
[ INFO] [1717492513.089973979]: expected frequency: 9.976 (Hz)
[ INFO] [1717492513.091540377]: Opening UDP socket: port 2368
[ INFO] [1717492513.092375572]: Opening UDP socket: port 8603
```

(2) 此时我们可以运行刚才编写的 tm_lidar_data_node，打开另外一个终端，输入指令，敲回车

```
roslaunch tm_lidar_pkg tm_lidar_data_node
```

```
yhs@yhs-ros:~/catkin_ws$ roslaunch tm_lidar_pkg tm_lidar_data_node
```

这条指令会启动我们编写的 tm_lidar_data_node。按照程序逻辑，会从 3D 激光雷达的三维点云主题“/tmlidar_points”主题里不断获取点云数据包，并把 ROS 格式的三维点云转换成 PCL 格式，然后把所有点的 x、y、z 坐标值显示在终端上

终端里显示“tm_lidar_data_node start”提示 tm_lidar_data_node 启动成功，接着不停的刷新显示点云所有点位的坐标值

```
yhs@yhs-ros:~/catkin_ws$ roslaunch tm_lidar_pkg tm_lidar_data_node
[ INFO] [1717492910.461252681]: tm_lidar_data_node start
```

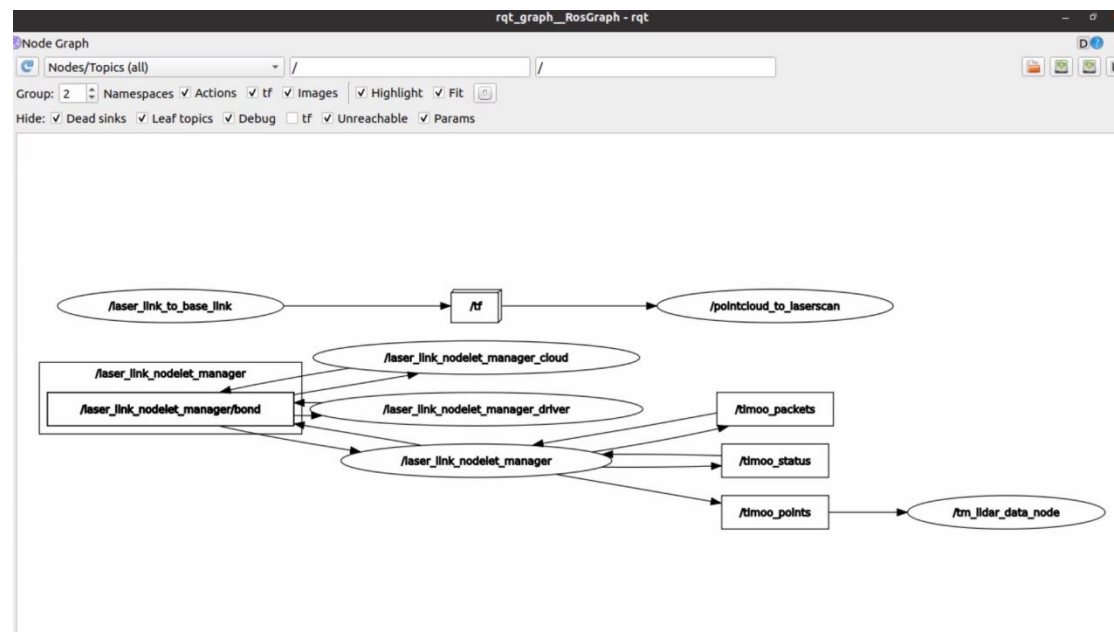
```
[ INFO] [1717492950.545794576]: [i= 3977] ( 3.87 , 4.20 , -0.49)
[ INFO] [1717492950.545819407]: [i= 3978] ( 4.75 , 5.16 , 1.33)
[ INFO] [1717492950.545842898]: [i= 3979] ( 4.77 , 5.17 , 1.58)
[ INFO] [1717492950.545866496]: [i= 3980] ( 3.99 , 4.32 , -0.10)
[ INFO] [1717492950.545890055]: [i= 3981] ( 4.44 , 4.81 , 1.71)
[ INFO] [1717492950.545914643]: [i= 3982] ( 1.92 , 2.08 , -0.74)
[ INFO] [1717492950.545938512]: [i= 3983] ( 2.27 , 2.46 , -0.75)
[ INFO] [1717492950.545961641]: [i= 3984] ( 4.77 , 5.16 , 0.36)
[ INFO] [1717492950.545984772]: [i= 3985] ( 2.77 , 2.99 , -0.78)
[ INFO] [1717492950.546007970]: [i= 3986] ( 3.63 , 3.92 , -0.82)
[ INFO] [1717492950.546031383]: [i= 3987] ( 4.51 , 4.87 , 0.79)
[ INFO] [1717492950.546054747]: [i= 3988] ( 3.42 , 3.69 , 0.78)
[ INFO] [1717492950.546078193]: [i= 3989] ( 3.87 , 4.17 , -0.49)
[ INFO] [1717492950.546101671]: [i= 3990] ( 4.78 , 5.15 , 1.33)
[ INFO] [1717492950.546124783]: [i= 3991] ( 4.79 , 5.16 , 1.58)
[ INFO] [1717492950.546148314]: [i= 3992] ( 4.00 , 4.31 , -0.10)
[ INFO] [1717492950.546171596]: [i= 3993] ( 4.46 , 4.80 , 1.71)
[ INFO] [1717492950.546195215]: [i= 3994] ( 1.93 , 2.08 , -0.74)
[ INFO] [1717492950.546219875]: [i= 3995] ( 2.28 , 2.45 , -0.76)
```

2、保持上面的终端程序持续运行，我们可以查看此时 ROS 系统里的节点网络关系。再启动一个终端程序，输入如下指令：

```
rqt_graph
```

```
yhs@yhs-ros:~$ rqt_graph
```

按下回车键之后，就会弹出一个新的窗体，里面显示了一系列连线框图即为现在 ROS 系统中的节点网络关系。

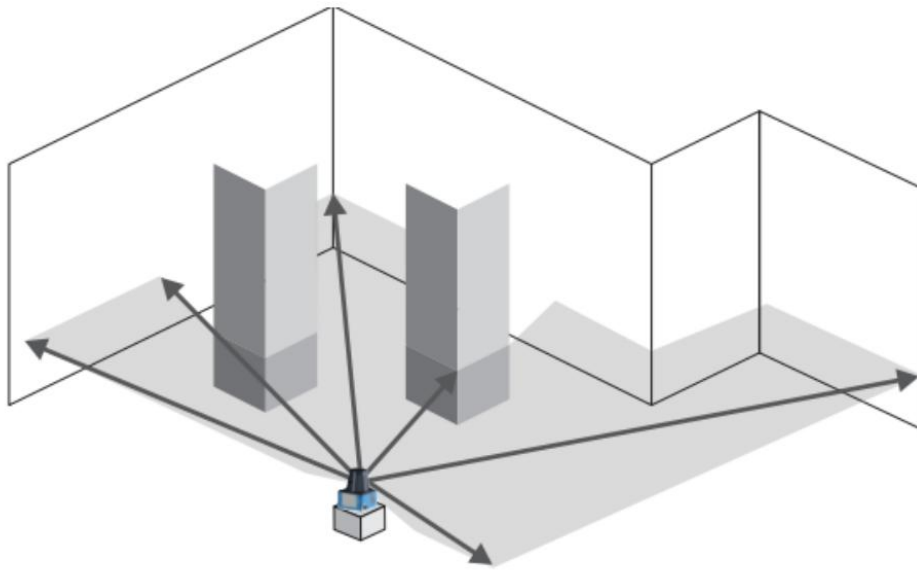


实验六 2D 建图

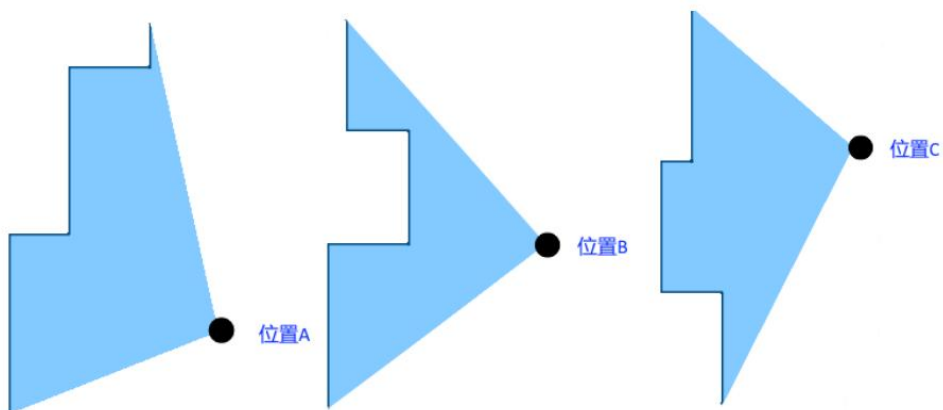
通过“pointcloud_to_laserscan”节点可以将 3D 激光点云数据转换为 2D 格式激光数据，用 2D 建图。

下面通过 2D 建图的过程，对建图有个初步的了解。

看过激光雷达的数据形式，这节将会开始进行激光雷达数据的应用。这里接触到一个名词“SLAM”，英文全称是“Simultaneous Localization And Mapping”，翻译过来就是“即时定位与地图构建”。SLAM 最早由 Smith、Self 和 Cheeseman 于 1988 年提出，由于其重要的理论与应用价值，被很多学者认为是实现真正全自主移动机器人的关键。要理解 SLAM，先得理解激光雷达的数据特点，激光雷达的扫描数据可以理解为一个障碍物分布的切面图，其反映的是在一个特定高度上，障碍物面向雷达的面的边缘形状和分布位置。

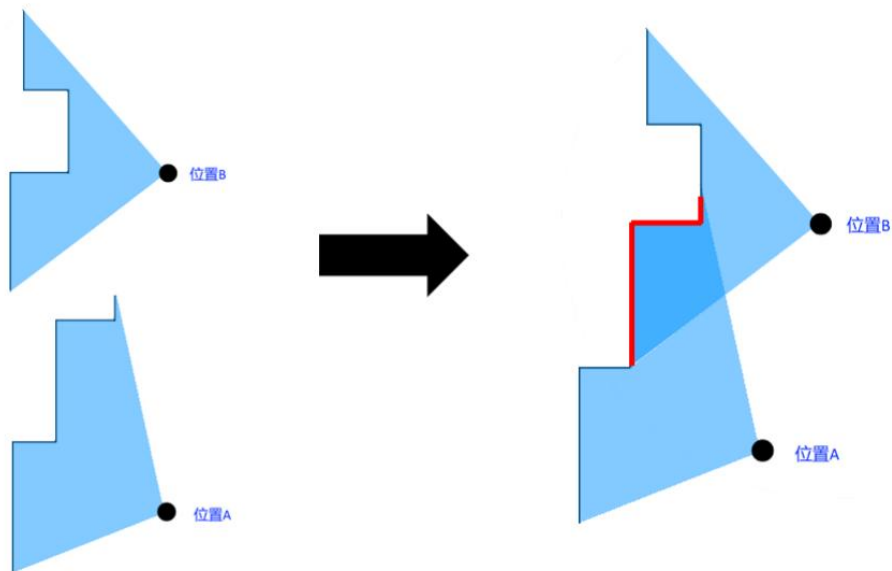


所以，当携带激光雷达的机器人在环境中运动时，它在某一个时刻，只能得到有限范围内的障碍物的部分轮廓和其在机器人本体坐标系里的相对位置。比如在下图中，反映了一个机器人在相邻比较近得 A、B、C 三个位置激光雷达扫描到的障碍物轮廓。

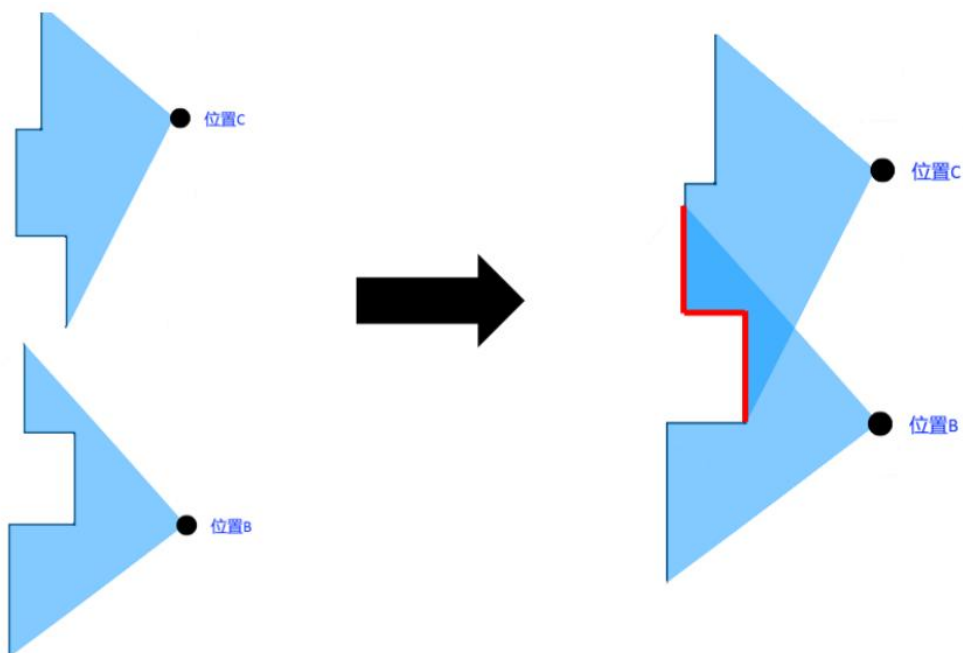


虽然此时我们还不知道位置 A、B、C 的相互关系，但是通过仔细观察，可以发现在 A、B、C 三个位置所扫描到的障碍物轮廓，某一些部分，是可以匹配重合的。因为这三个位置离得比较近，我们就假设扫描到的障碍物轮廓的相似部分就是同一个障碍物，这样就可以试

着将相似部分的障碍物轮廓叠加重合在一起，得到一个更大的障碍物轮廓图案。比如位置 A 和位置 B 的障碍物轮廓叠加后如下：

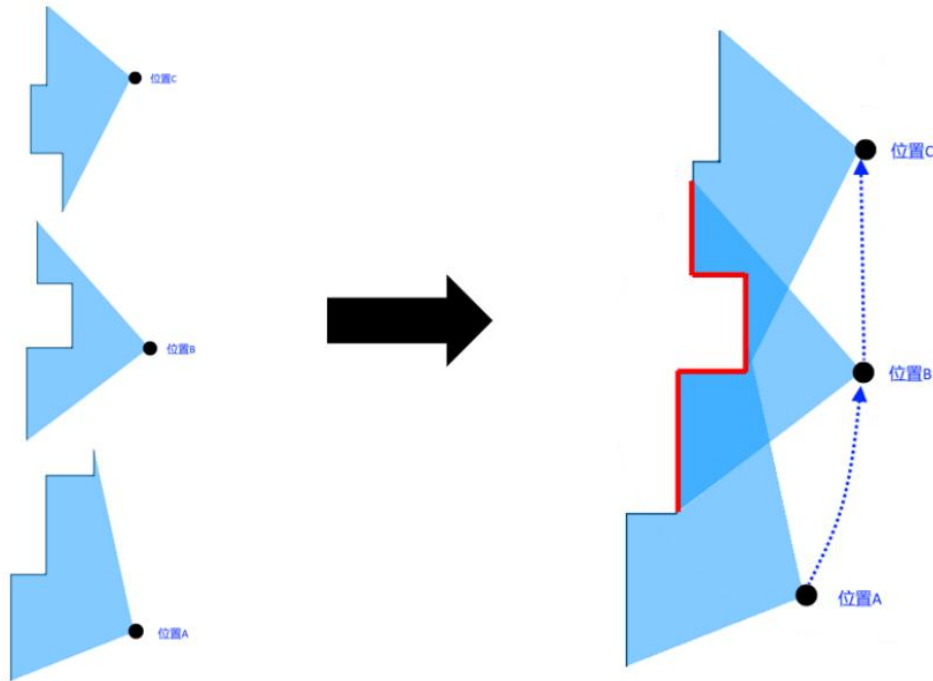


再比如，位置 B 和位置 C 的障碍物轮廓叠加后如下：



按照上述的方法，将连续的多个位置激光雷达扫描到的障碍物轮廓拼合在一起，就能形成一个比较完整的平面地图。这个地图是一个二维平面上的地图，其反映的是在激光雷达的扫描面上，整个环境里的障碍物轮廓和分布情况。在构建地图的过程中，还可以根据障碍物轮廓的重合关系，反推出机器人所走过的这几个位置之间的相互关系以及机器人在地图中所处的位置，这就同时完成了地图构建和机器人的自身实时定位这两项功能，这也就是“SLAM”全称“Simultaneous Localization And Mapping”的由来。同样以前面的 A、B、C 三个位置为

例，将三个位置的激光雷达扫描轮廓拼合在一起，就能得到一个相对更完整的平面地图，同时也得出 A、B、C 三个位置在这个地图中的位置：



接下来使用的是 Gmapping 和 Cartographer。

Gmapping 建图思想是基于 PBRF 粒子滤波算法，每个粒子都携带一张地图。**Gmapping** 可以很好的利用里程计的信息来降低对使用的激光雷达频率的要求。机器人可以通过里程计把编码器的数据根据机器人当前的位姿计算求解出下一个时刻的位姿，然后再将每个粒子的激光传感器数据与建好的地图特征进行对比，选出与地图特征最好的粒子的位姿作为当前的位姿，如此反复构建地图。

Cartographer 是一种基于图优化的 slam 算法，建图思想是 GraphSLAM。使用图优化计算在高建图精度的前提下效率还快，处理数据的方式和滤波的方法不同，它不是在线的纠正位姿，而是把所有数据记下来，最后一次性计算。并且 **Cartographer** 带有闭环检测。

1、Gmapping 建图

(1) 打开终端，输入指令后敲回车，出现“Registering Scans:Done”则表示启动 Gmapping 建图成功。

```
roslaunch yhs_nav gmapping.launch
```

```
yhs@yhs-ros2:~$ ros2 launch yhs_nav2 gmapping.launch.py
```

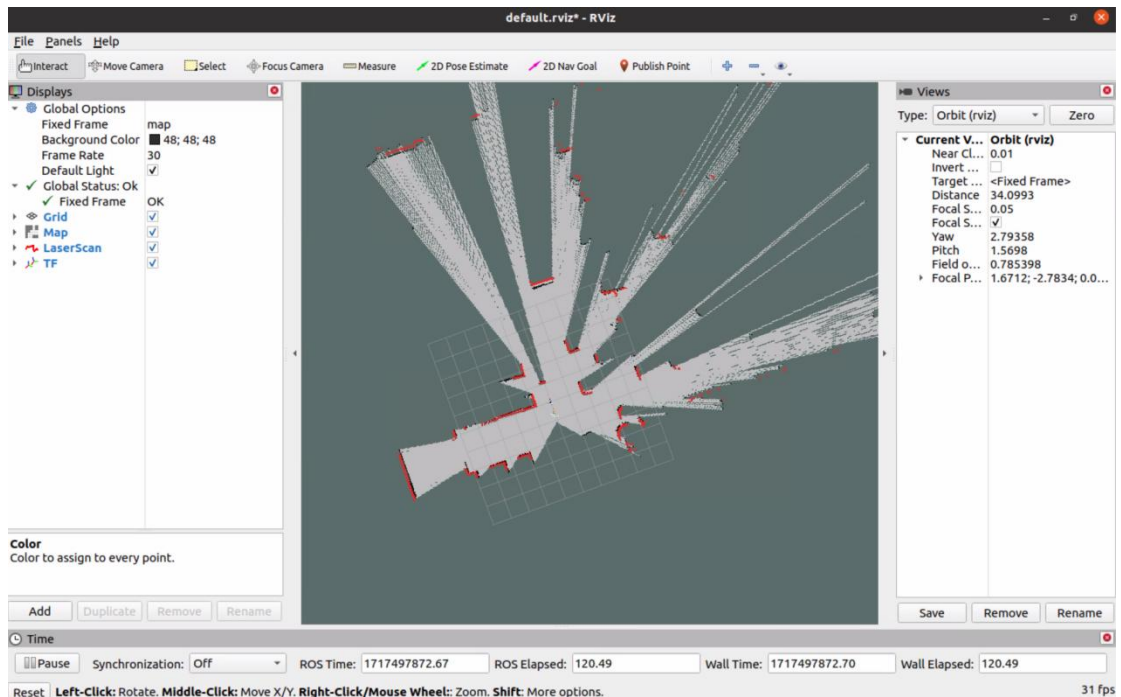
```
yhs@yhs-ros:~$ roslaunch yhs_nav gmapping.launch
... logging to /home/yhs/.ros/log/edd40aee-225e-11ef-8593-6da3ed79c16a/roslaunch-yhs-ros-51315.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:33919/

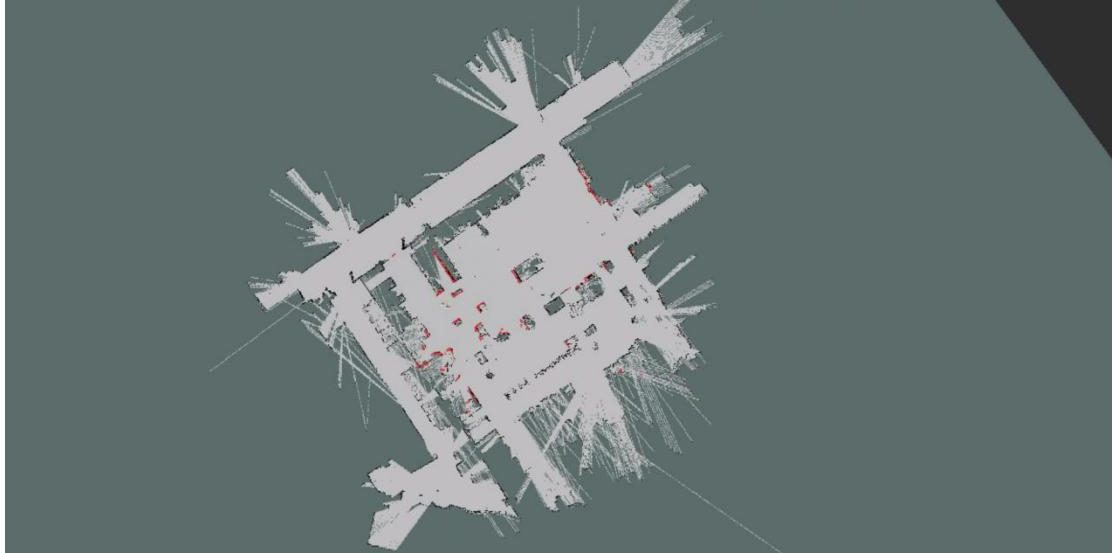
SUMMARY
=====

[ INFO] [1717497659.713489679]: Laser is mounted upwards.
-maxUrange 49.99 -maxUrange 49.99 -sigma 0.05 -kernelSize 3 -lstep 0.05 -lobGain 3 -astep 0.05
-srr 0.001 -srt 0.002 -str 0.001 -stt 0.002
-linearUpdate 0.1 -angularUpdate 0.1 -resampleThreshold 0.5
-xmin -50 -xmax 50 -ymin -50 -ymax 50 -delta 0.05 -particles 8
[ INFO] [1717497659.723625027]: Initialization complete
update frame 0
update ld=0 ad=0
Laser Pose= 0.14 0 0
m_count 0
Registering First Scan
```

(2) 此时在 Rviz 上看到机器人周围的地面基准变成了深灰色，而机器人脚下则出现一片白色图案。这个图案，是由很多条线段叠加而成，这些线段是机器人本体中心地面投影和每一个激光雷达红色障碍点的连线，是测距激光飞行的轨迹，表示这条线段内部没有障碍物



(3) 此时就可以使用遥控器控制底盘行走建图了，建图过程应遵循以下两条原则：
不要过快的旋转；
在有走廊和房间的场景下，先建好房间的轮廓再到走廊；
建图过程可以实时观察 Rviz 上的地图，如果地图与实际场景相差过大，终止建图，按照上面两条原则重新开始建图



(4) 用遥控器遥控底盘走完想建图的区域，并且建图的效果达到要求，这时就可以保存地图了，打开另外一个终端，进入“/home/yhs/catkin_ws/src/yhs_nav/map”目录下，输入命令保存地图，命令末尾“g1”是地图名字，若取相同名字，会覆盖上次保存的地图

```
roscd yhs_nav/map
```

```
roslaunch map_server map_saver -f g1
```

```
yhs@yhs-ros:~$ roscd yhs_nav/map/
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/map$

yhs@yhs-ros:~/catkin_ws/src/yhs_nav/map$ roslaunch map_server map_saver -f g1
[ INFO] [1717498230.490113350]: Waiting for the map
[ INFO] [1717498230.734770835]: Received a 1984 X 1984 map @ 0.050 m/pix
[ INFO] [1717498230.734796133]: Writing map occupancy data to g1.pgm
[ INFO] [1717498230.834033929]: Writing map occupancy data to g1.yaml
[ INFO] [1717498230.834138973]: Done
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/map$
```

2、Cartographer 建图

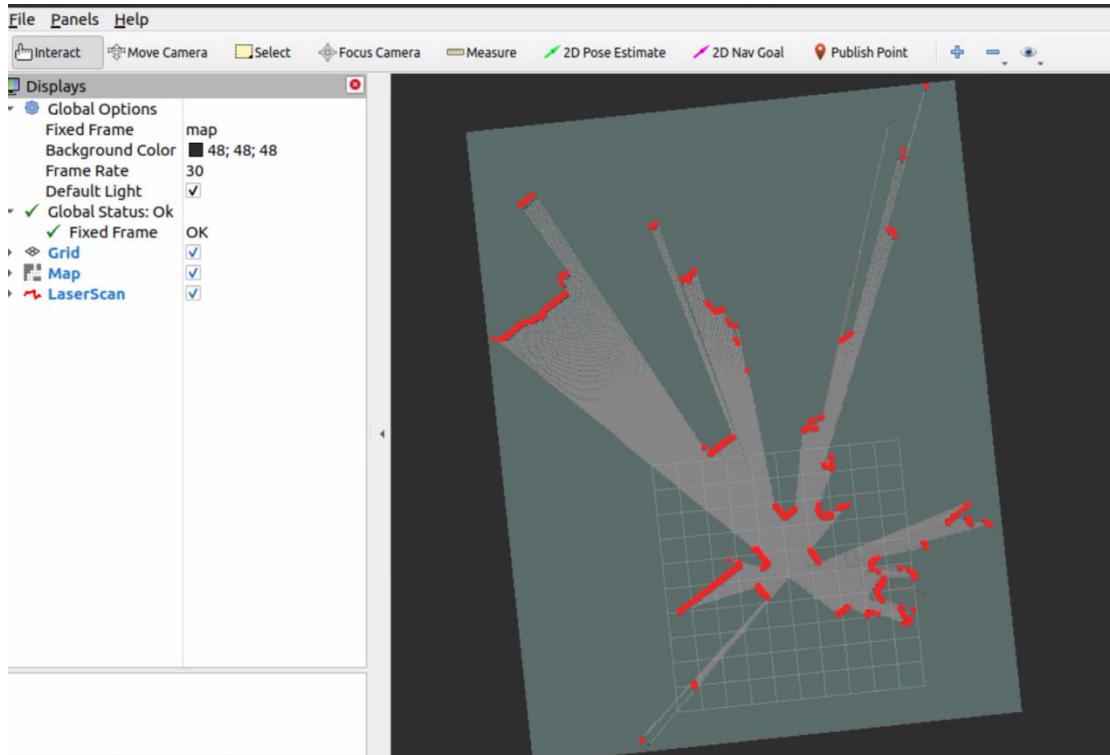
(1) 打开终端，输入指令后敲回车，看到 “Inserted submap (0, 0).” 输出后，启动成功

```
roslaunch yhs_nav cartographer.launch
```

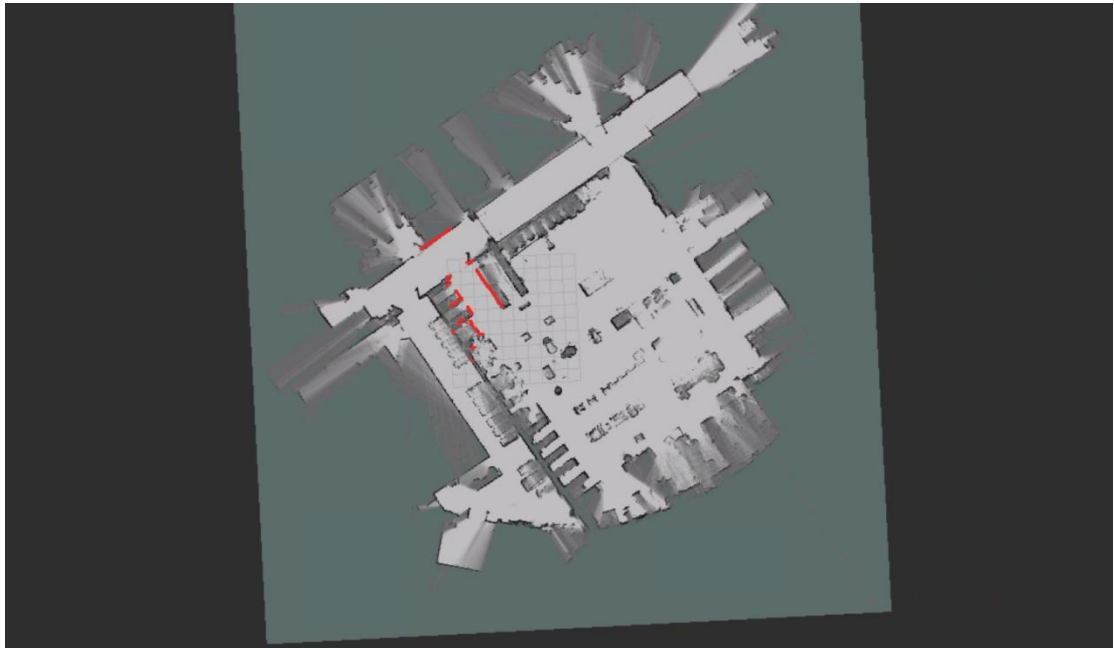
```
yhs@yhs-ros:~$ roslaunch yhs_nav cartographer.launch

[ INFO] [1717498908.910931803]: I0604 19:01:48.000000 52414 configuration_file_resolver.cc:41] Found '/usr/local/share/cartographer/configuration_file
s/trajectory_builder.lua' for 'trajectory_builder.lua'.
[ INFO] [1717498908.910978108]: I0604 19:01:48.000000 52414 configuration_file_resolver.cc:41] Found '/usr/local/share/cartographer/configuration_file
s/trajectory_builder_2d.lua' for 'trajectory_builder_2d.lua'.
[ INFO] [1717498908.911005807]: I0604 19:01:48.000000 52414 configuration_file_resolver.cc:41] Found '/usr/local/share/cartographer/configuration_file
s/trajectory_builder_2d.lua' for 'trajectory_builder_2d.lua'.
[ INFO] [1717498908.91123195]: I0604 19:01:48.000000 52414 configuration_file_resolver.cc:41] Found '/usr/local/share/cartographer/configuration_file
s/trajectory_builder_3d.lua' for 'trajectory_builder_3d.lua'.
[ INFO] [1717498908.91150609]: I0604 19:01:48.000000 52414 configuration_file_resolver.cc:41] Found '/usr/local/share/cartographer/configuration_file
s/trajectory_builder_3d.lua' for 'trajectory_builder_3d.lua'.
[ INFO] [1717498908.921400126]: I0604 19:01:48.000000 52414 map_builder_bridge.cc:135] Added trajectory with ID '0'.
[ INFO] [1717498909.609256916]: I0604 19:01:49.000000 52414 ordered_multi_queue.cc:172] All sensor data for trajectory 0 is available starting at '638
530957096040609'.
[ INFO] [1717498909.613003185]: I0604 19:01:49.000000 52414 local_trajectory_builder_2d.cc:312] Extrapolator not yet initialized.
[ WARN] [1717498909.613837935]: W0604 19:01:49.000000 52414 pose_extrapolator.cc:108] Queue too short for velocity estimation. Queue duration: 0 s
[ INFO] [1717498909.623200509]: I0604 19:01:49.000000 52414 pose_graph_2d.cc:148] Inserted submap (0, 0).
```

(2) 此时在 viz 上看到机器人周围的地面基准变成了深灰色，而机器人脚下则出现一片白色图案。这个图案，是由很多条线段叠加而成，这些线段是机器人本体中心地面投影和每一个激光雷达红色障碍点的连线，是测距激光飞行的轨迹，表示这条线段内部没有障碍物



(3) 此时就可以使用遥控器控制底盘行走建图了，建图过程应使底盘不要过快的旋转，可以实时观察 Rviz 上的地图，如果地图与实际场景相差过大，终止建图，按照不要过快的旋转的原则重新开始建图



(4) 用遥控器遥控底盘走完想建图的区域并回到起点，停下来，在 Rviz 上观察地图，如果地图有重影，建图程序会进行地图的优化修正，此时耐心等待建图优化完成，效果达到要求后，这时就可以保存地图了，打开另外一个终端，进入“/home/yhs/catkin_ws/src/yhs_nav/map”目录下，输入命令保存地图，命令末尾“c1”是地图名字，若取相同名字，会覆盖上次保存的地图

```
roscd yhs_nav/map
```

```
roslaunch map_server map_saver -f c1
```

```
yhs@yhs-ros:~$ roscd yhs_nav/map
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/map$ roslaunch map_server map_saver -f c1
[ INFO] [1717499543.256024204]: Waiting for the map
[ INFO] [1717499543.496792205]: Received a 980 X 1141 map @ 0.050 m/pix
[ INFO] [1717499543.496817277]: Writing map occupancy data to c1.pgm
[ INFO] [1717499543.526357116]: Writing map occupancy data to c1.yaml
[ INFO] [1717499543.526459604]: Done
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/map$
```

实验七 3D 建图

使用 LIO-SAM，一种通过平滑和映射进行紧密耦合的激光雷达惯性里程计框架，该框架可实现高精度、实时的移动机器人轨迹估计和地图构建。LIO-SAM 在因子图上制定激光雷达惯性里程计，允许将来自不同来源的大量相对和绝对测量值（包括环闭合）作为因子整合到系统中。惯性测量单元（IMU）预积分的估计运动会消除点云的偏差，并为激光雷达里程计优化生成初步猜测。利用得到的激光雷达里程计解来估计 IMU 的偏差。为了确保实时的高性能，将旧的激光雷达扫描边缘化以进行姿态优化，而不是将激光雷达扫描与全球地图相匹配。在局部规模而不是全局规模上进行扫描匹配可以显著提高系统的实时性能，选择性地引入关键帧，以及将新关键帧注册到固定大小的先前“子关键帧”集的高效滑动窗口方法。

1、建图过程不使用 GNSS:

(1) 打开终端，运行 3D 激光雷达驱动程序

```
roslaunch timoo_pointcloud TM16.launch
```

```
yhs@yhs-ros:~$ roslaunch timoo_pointcloud TM16.launch
... logging to /home/yhs/.ros/log/d801c9a6-22d4-11ef-baaa-6fd2ae4fa70b/roslaunch-yhs-ros-4254.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:45935/

SUMMARY
=====

setting /run_id to d801c9a6-22d4-11ef-baaa-6fd2ae4fa70b
process[rosout-1]: started with pid [4272]
started core service [/rosout]
process[laser_link_nodelet_manager-2]: started with pid [4279]
process[laser_link_nodelet_manager_driver-3]: started with pid [4280]
process[laser_link_nodelet_manager_ccloud-4]: started with pid [4281]
process[laser_link_to_base_link-5]: started with pid [4282]
process[pointcloud_to_laserscan-6]: started with pid [4287]
[ INFO] [1717548303.216993360]: Initializing nodelet with 8 worker threads.
[ INFO] [1717548303.234090769]: timoo TM-C16 rotating at 600 RPM
[ INFO] [1717548303.234965334]: publishing 85 packets per scan
[ INFO] [1717548303.235623938]: Setting timoo scan start time to timestamp of first packet
[ INFO] [1717548303.236671846]: Cut at specific angle feature deactivated.
[ INFO] [1717548303.242768977]: Reconfigure Request
[ INFO] [1717548303.243981386]: expected frequency: 9.976 (Hz)
[ INFO] [1717548303.246134102]: Opening UDP socket: port 2368
[ INFO] [1717548303.247167652]: Opening UDP socket: port 8603
[ INFO] [1717548303.270811135]: correction angles: /home/yhs/catkin_ws/src/hardware/timoo/timoo_pointcloud/params/tm16.yaml
[ INFO] [1717548303.273771755]: Number of lasers: 16.
[ INFO] [1717548303.275621319]: Initialized container with min_range: 0.15, max_range: 50, target_frame: timoo, fixed_frame: laser_link, init_wld: 0
[ INFO] [1717548303.275621319]: init_height: 1, is_dense: 1, scans_per_packet: 384
[ INFO] [1717548303.357685326]: Reconfigure Request
[ INFO] [1717548303.357911959]: Initialized container with min_range: 0.15, max_range: 50, target_frame: laser_link, fixed_frame: laser_link, init_wld: 0, init_height: 1, is_dense: 1, scans_per_packet: 384
```

(2) 打开另外一个终端，运行 imu 驱动程序

```
roslaunch serial_imu imu_msg.launch
```

```
yhs@yhs-ros:~$ roslaunch serial_imu imu_msg.launch
... logging to /home/yhs/.ros/log/d801c9a6-22d4-11ef-baaa-6fd2ae4fa70b/roslaunch-yhs-ros-4550.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:46005/

SUMMARY
=====

PARAMETERS
* /roscdistro: noetic
* /rosversion: 1.16.0

NODES
/
  HI226 (serial_imu/serial_imu)
  imu_broadcaster (tf2_ros/static_transform_publisher)

ROS_MASTER_URI=http://192.168.1.102:11311

process[HI226-1]: started with pid [4564]
process[imu_broadcaster-2]: started with pid [4565]
[ INFO] [1717548428.668664134]: /dev/imu is opened.
```

(3) 再打开一个终端，进入“3dbags”目录

```
cd /home/yhs/3dbags
```

```
yhs@yhs-ros:~$ cd 3dbags/
yhs@yhs-ros:~/3dbags$
```

(4) 查看话题，确认 3D 激光点云话题“/tmlidar_points”和 imu 数据话题“/imu_data”有数据输出

```
rostopic list
```

```
rostopic echo /imu_data
```

```
rostopic echo /tmlidar_points
```

```
yhs@yhs-ros:~/3dbags$ rostopic list
/diagnostics
/imu_data
/laser_link_nodelet_manager/bond
/laser_link_nodelet_manager_cloud/parameter_descriptions
/laser_link_nodelet_manager_cloud/parameter_updates
/laser_link_nodelet_manager_driver/parameter_descriptions
/laser_link_nodelet_manager_driver/parameter_updates
/rosout
/rosout_agg
/rpy
/scan
/tf
/tf_static
/timoo_imu
/timoo_packets
/timoo_points
/timoo_status
yhs@yhs-ros:~/3dbags$
```

```
yhs@yhs-ros:~/3dbags$ rostopic echo /imu_data
header:
  seq: 16204
  stamp:
    secs: 1717548591
    nsecs: 859930004
  frame_id: "imu_link"
orientation:
  x: 0.006324083544313908
  y: -0.010295684449374676
  z: 1.0933740668406244e-05
  w: 0.9999271035194397
orientation_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
angular_velocity:
  x: 0.0017453290260074287
  y: 0.0
  z: 0.0
angular_velocity_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
linear_acceleration:
  x: 0.22540000043809416
  y: 0.12740000262856485
  z: 9.84899995326996
linear_acceleration_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

```
yhs@yhs-ros:~/3dbags$ rostopic echo /timoo_points
```

```
88, 144, 63, 214, 114, 17, 191, 0, 0, 248, 66, 4, 0, 224, 114, 202, 61, 240, 217, 95, 65, 242, 153, 91, 64, 179, 234, 13, 64, 0, 0,
5, 116, 202, 61, 153, 252, 154, 64, 243, 237, 151, 63, 171, 50, 219, 190, 0, 0, 250, 66, 5, 0, 25, 118, 202, 61, 211, 179, 95, 65,
4, 86, 46, 64, 0, 0, 248, 66, 13, 0, 181, 119, 202, 61, 169, 94, 164, 64, 91, 226, 160, 63, 67, 87, 138, 190, 0, 0, 244, 66, 6, 0,
98, 69, 96, 65, 204, 89, 91, 64, 246, 148, 79, 64, 0, 0, 242, 66, 14, 0, 238, 122, 202, 61, 153, 95, 85, 65, 113, 136, 80, 64, 67,
, 204, 66, 7, 0, 138, 124, 202, 61, 190, 230, 47, 65, 229, 199, 43, 64, 17, 210, 60, 64, 0, 0, 2, 67, 15, 0, 39, 126, 202, 61, 177,
55, 40, 63, 61, 199, 58, 191, 0, 0, 80, 66, 0, 0, 195, 127, 202, 61, 137, 189, 87, 65, 61, 53, 82, 64, 198, 160, 110, 62, 0, 0, 206
```

(5) 开始录制数据，在“/home/yhs/3dbags”目录下执行

```
rosbag record /imu_data /tmlidar_points -O in1
```

```
yhs@yhs-ros:~/3dbags$ rosbag record /imu_data /timoo_points -O in1
[ INFO] [1717549141.453994558]: Subscribing to /imu_data
[ INFO] [1717549141.456447997]: Subscribing to /timoo_points
[ INFO] [1717549141.459015094]: Recording to 'in1.bag'.
```

“in1”是存放数据的文件夹名，可以随便取（相同名字会覆盖），录制开始后，就可以遥控底盘绕需建图区域行走，行走过程中不要原地、过快旋转，最后回到起点，一定要按“ctrl + c”终止录制

```
yhs@yhs-ros:~/3dbags$ rosbag record /imu_data /timoo_points -O in1
[ INFO] [1717549141.453994558]: Subscribing to /imu_data
[ INFO] [1717549141.456447997]: Subscribing to /timoo_points
[ INFO] [1717549141.459015094]: Recording to 'in1.bag'.
^C
yhs@yhs-ros:~/3dbags$
```

(6) 参数修改，在室内平地、长走廊场景下，将“/home/yhs/catkin_ws/src/LIO-SAM/config/params.yaml”文件中的三个参数改为下面的值，其他场景不用需改，使用默认值

z_tolerance	0.0
rotation_tolerance	0.0
historyKeyframeFitnessScore	0.1

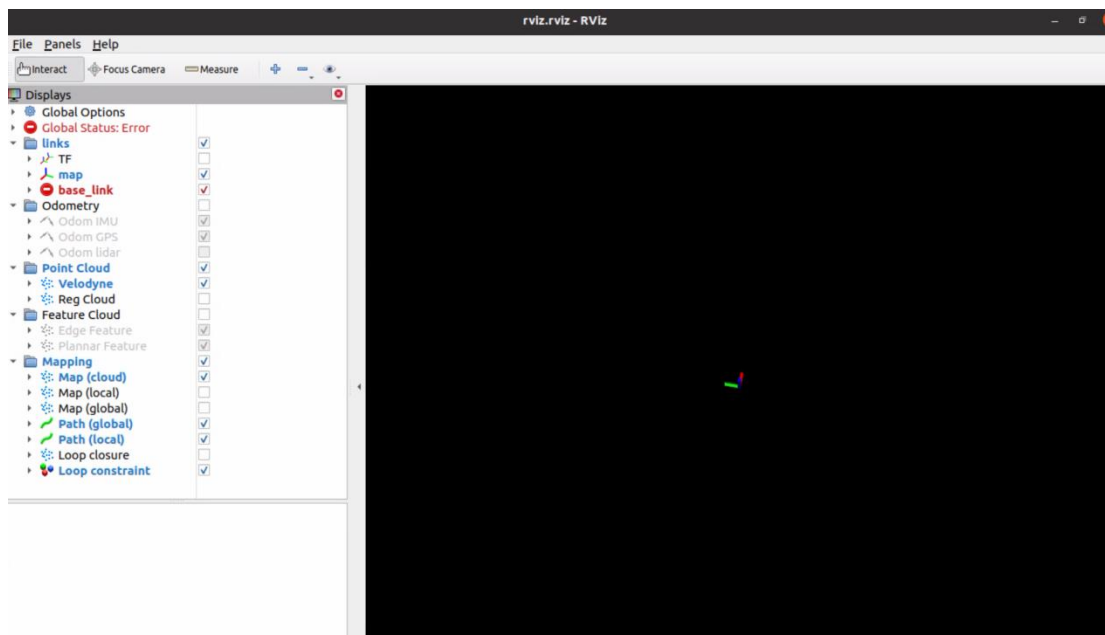
(7) 运行 lio-sam 节点，先关闭之前打开的 3D 激光和 imu 驱动程序节点，输入指令

```
roslaunch lio_sam run.launch
```

```
yhs@yhs-ros:~$ roslaunch lio_sam run.launch
... logging to /home/yhs/.ros/log/98f47238-22d7-11ef-baaa-6fd2ae4fa70b/roslaunch-yhs-ros-5662.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.
```

```
setting /run_id to 98f47238-22d7-11ef-baaa-6fd2ae4fa70b
process[rosout-1]: started with pid [5682]
started core service [/rosout]
process[lio_sam_imuPreintegration-2]: started with pid [5689]
process[lio_sam_imageProjection-3]: started with pid [5690]
process[lio_sam_featureExtraction-4]: started with pid [5691]
process[lio_sam_mapOptimization-5]: started with pid [5692]
process[robot_state_publisher-6]: started with pid [5694]
process[ekf_gps-7]: started with pid [5707]
process[navsat-8]: started with pid [5710]
process[lio_sam_rviz-9]: started with pid [5715]
[ INFO] [1717549486.212563721]: ----> Feature Extraction Started.
[ INFO] [1717549486.235468734]: ----> Image Projection Started.
[ INFO] [1717549486.308258658]: ----> Map Optimization Started.
[ INFO] [1717549486.319291597]: ----> IMU Preintegration Started.
```

指令执行后，会自动弹出 Rviz 界面



(8) 运行 lio-sam 节点后，播放录制好的“bag”，打开终端进入“/home/yhs/3dbags”目录

```
cd /home/yhs/3dbags
```

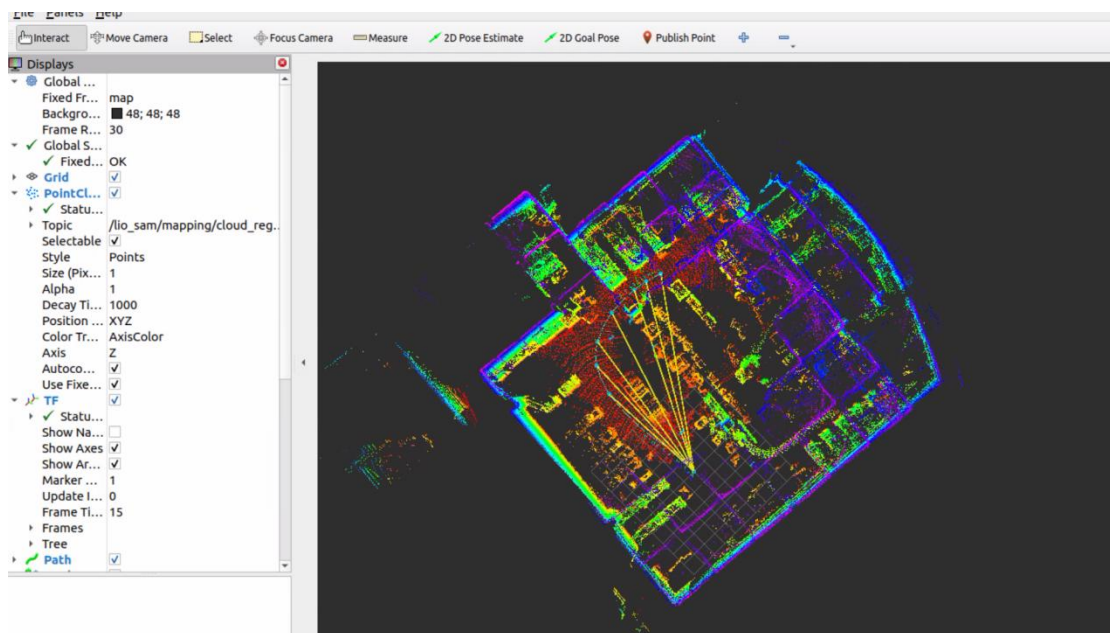
```
yhs@yhs-ros:~$ cd 3dbags/  
yhs@yhs-ros:~/3dbags$
```

输入指令以下指令，其中“-r”后面接的是播放的倍数，一般以 3~4 倍速度播放，播放过程中可以按“空格键”暂停，再按继续

```
roslaunch play -r 3 in1.bag
```

```
yhs@yhs-ros:~/3dbags$ roslaunch play -r 3 in1.bag  
[ INFO ] [1717550749.919580674]: Opening in1.bag  
  
Waiting 0.2 seconds after advertising topics... done.  
  
Hit space to toggle paused, or 's' to step.  
[RUNNING] Bag Time: 1717549152.631826 Duration: 10.965746 / 199.015749
```

此时，在 Rviz 上就可以看到点云、行走轨迹等数据了



等待播放完成，在运行 lio-sam 节点终端按“ctrl + c”保存地图

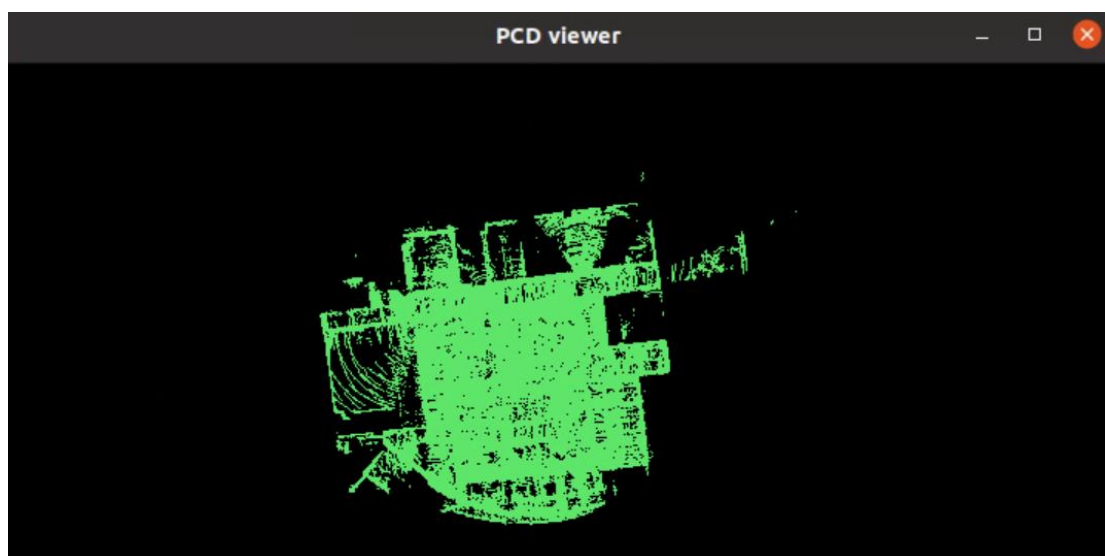
```
^C[lio_sam_rviz-9] killing on exit  
[navsat-8] killing on exit  
[ekf_gps-7] killing on exit  
[robot_state_publisher-6] killing on exit  
[lio_sam_featureExtraction-4] killing on exit  
[lio_sam_imageProjection-3] killing on exit  
[lio_sam_imuPreIntegration-2] killing on exit  
[lio_sam_mapOptimization-5] killing on exit  
*****  
Saving map to pcd files ...  
Save destination: /home/yhs/Downloads/LOAM/  
Processing feature cloud 79 of 80 ...  
Saving map to pcd files completed  
  
[rosout-1] killing on exit  
[master] killing on exit  
shutting down processing monitor...  
... shutting down processing monitor complete  
done
```

(9) 查看建好的 pcd 点云图，地图默认保存在“/home/yhs/Downloads/LOAM”目录下，需要注意的是，每次保存都会覆盖掉前一次建好的地图，进入“/home/yhs/Downloads/LOAM”目录，打开终端，使用“pcl_viewer”查看“cloudGlobal.pcd”地图

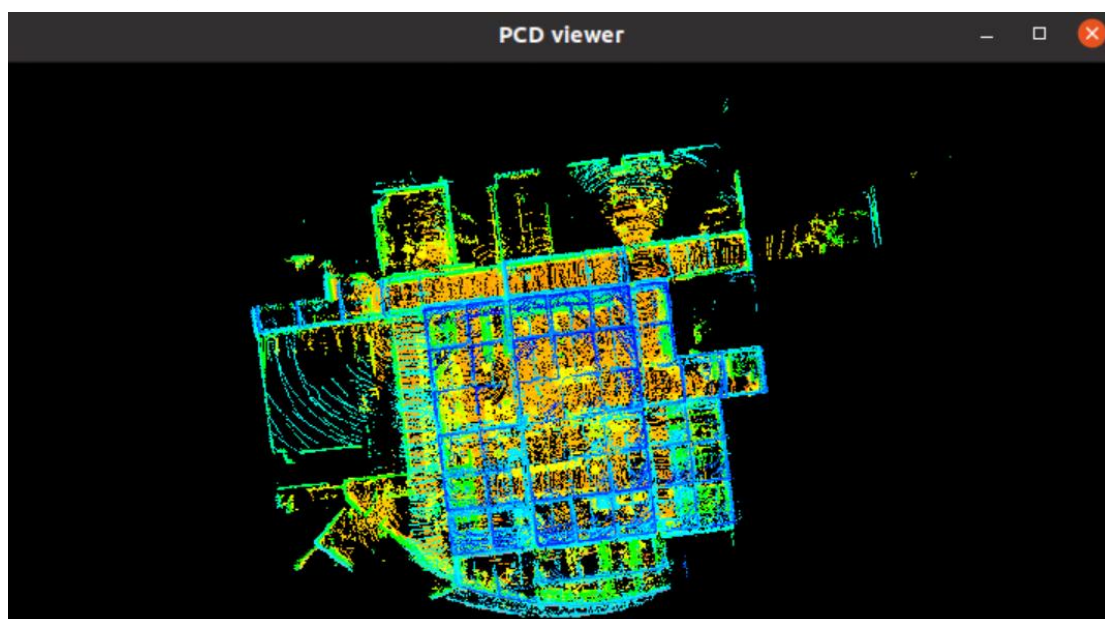
```
pcl_viewer GlobalMap.pcd
```

```
yhs@yhs-ros:~/Downloads/LOAM$ pcl_viewer GlobalMap.pcd
The viewer window provides interactive commands; for help, press 'h' or 'H' from within the window.
> Loading GlobalMap.pcd [PCLVisualizer::setUseVbos] Has no effect when OpenGL version is ≥ 2
[done, 118.595 ms : 393332 points]
Available dimensions: x y z intensity
```

弹出“PCD viewer”界面



滚动鼠标滚轮放大缩小，按住鼠标滚轮拖动，键盘数字“1、2、3、4、5”可以改变颜色，键盘“f”键放大鼠标所指位置，观察点云图是否是闭环的，如果没有闭环，调整参数再次建图，看完即可关闭



(10) 转换 PCD 点云图为栅格地图并保存，输入指令

```
roslaunch pcl_csfc run.launch
```

```
yhs@yhs-ros:~$ roslaunch pcl_csfc run.launch
... logging to /home/yhs/.ros/log/86ef5c70-22db-11ef-bc55-55fb98a1cfcb/roslaunch-yhs-ros-5645.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:40981/

setting /run_id to 86ef5c70-22db-11ef-bc55-55fb98a1cfcb
process[rosout-1]: started with pid [5663]
started core service [/rosout]
process[pcl_csfc_node-2]: started with pid [5670]
[ INFO] [1717551173.819553326]: bSloopSmooth: true
[ INFO] [1717551173.820740419]: cloth_resolution: 0.500000
[ INFO] [1717551173.820778917]: rigidness: 3
[ INFO] [1717551173.820815506]: time_step: 0.650000
[ INFO] [1717551173.820832190]: class_threshold: 0.010000
[ INFO] [1717551173.820859372]: interations: 500
[ INFO] [1717551173.820896989]: pcd_path: /home/yhs/Downloads/LOAM/GlobalMap.pcd
[ INFO] [1717551173.820939494]: resolution: 0.050000
[ INFO] [1717551173.820980777]: z_min: -2.000000
[ INFO] [1717551173.821022255]: z_max: 5.000000
[ INFO] [1717551173.821055339]: MeanK: 60
[ INFO] [1717551173.821096289]: StddevMulThresh: 0.100000
正在加载 /home/yhs/Downloads/LOAM/GlobalMap.pcd
加载完成，进行转换，pcd地图越大，转换时间越长，请等待转换完成。。。
[0] Configuring terrain...
[0] Configuring cloth...
[0] - width: 116 height: 114
[0] Rasterizing...
[0] Simulating...
[0] - post handle...
转换栅格地图完成，可以保存地图了。。。

```

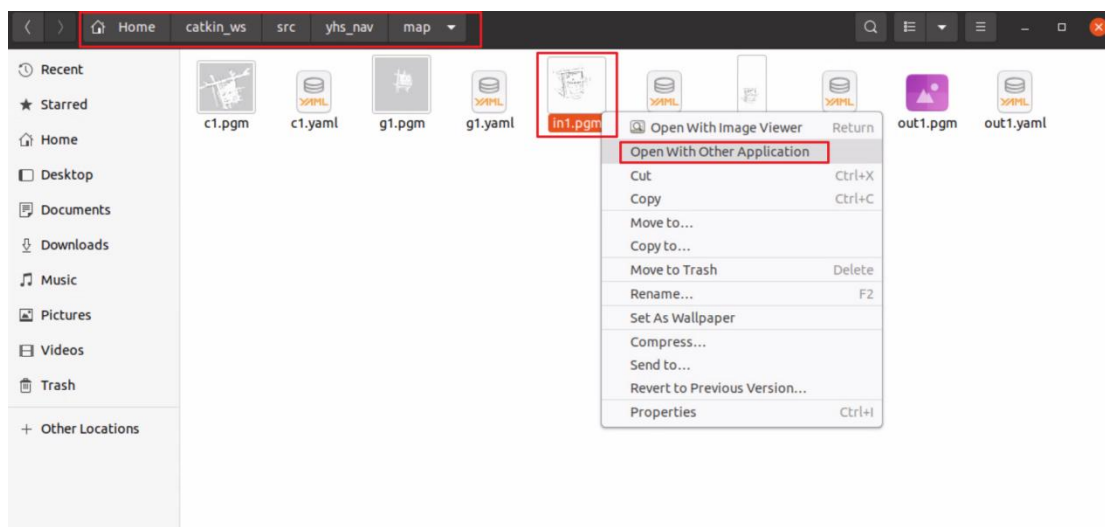
等待转换完成，地图越大，转换时间越长，转换完成后，先不要关闭转换节点，打开另外一个终端，进入“/home/yhs/catkin_ws/src/yhs_nav/map/”目录，输入以下指令保存地图，其中“in1”是地图名字，可以随便取，若名字相同，则会覆盖掉之前的地图

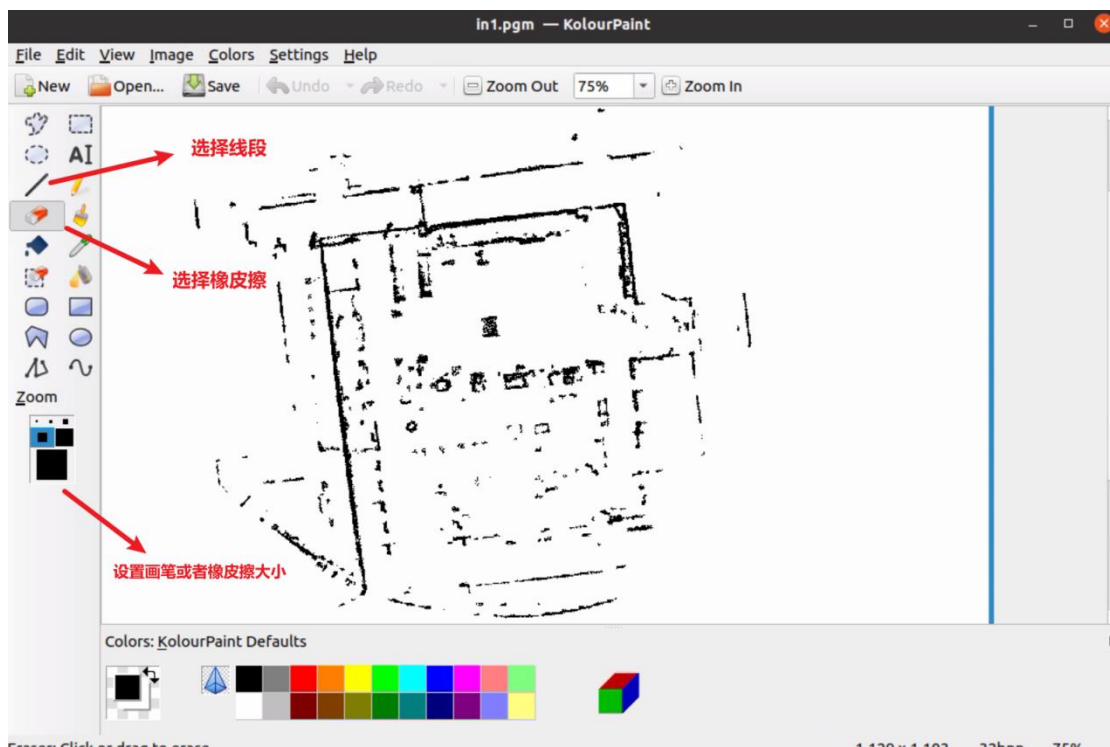
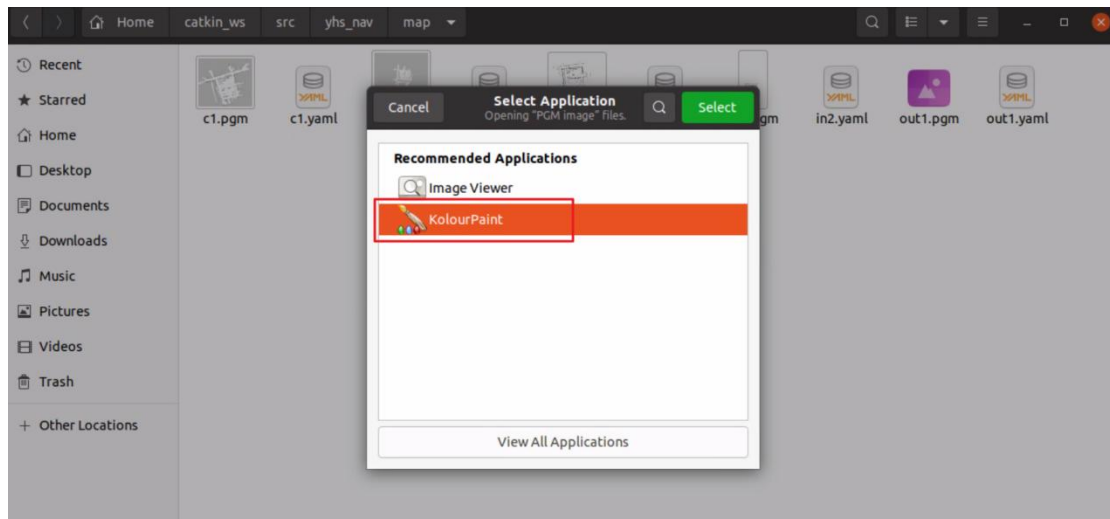
```
roslaunch map_server map_saver -f in1
```

```
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/map$ roslaunch map_server map_saver -f in1
[ INFO] [1717551253.070819411]: Waiting for the map
[ INFO] [1717551253.346515918]: Received a 1129 X 1103 map @ 0.050 m/pix
[ INFO] [1717551253.346548169]: Writing map occupancy data to in1.pgm
[ INFO] [1717551253.380468771]: Writing map occupancy data to in1.yaml
[ INFO] [1717551253.380641355]: Done
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/map$

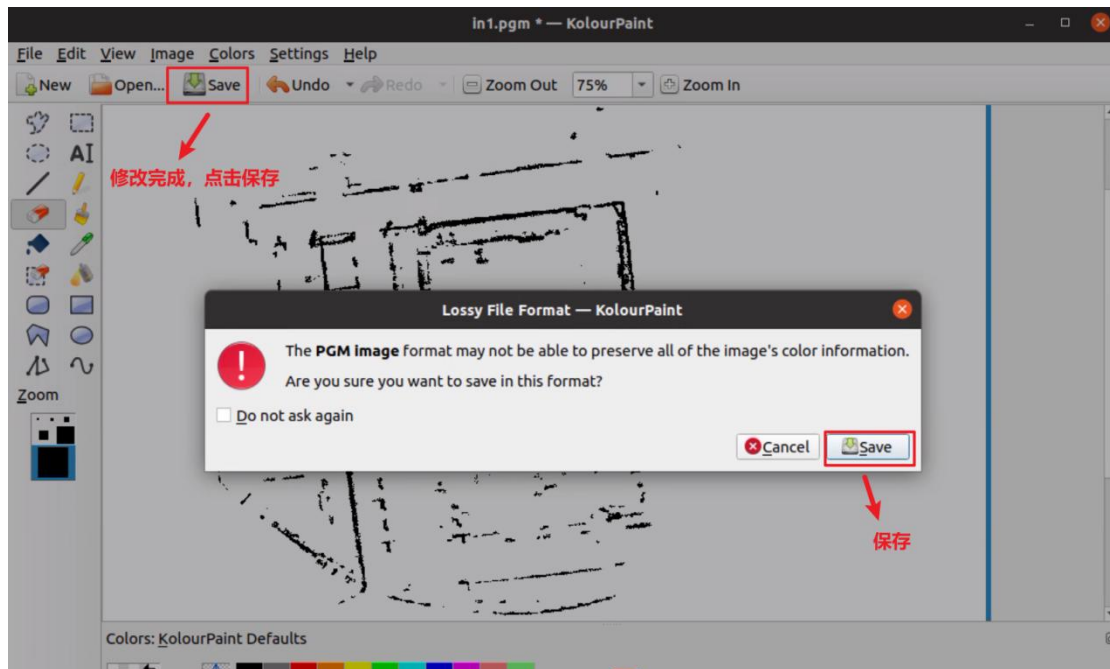
```

(11) 栅格地图编辑，得到栅格地图后，可能要对地图中的某些区域进行清除或者添加障碍物，进入“/home/yhs/catkin_ws/src/yhs_nav/map/”目录，找到要编辑的地图，右键选择“KolourPaint”软件打开





按住鼠标左键画直线和擦除，编辑完成后点击保存，地图较大时，保存时间较长，等待保存完成，保存完成关闭软件即可



2、建图过程使用 GNSS:

要在室外空旷的地方

(1) 打开终端，输入指令运行运行底盘驱动、IMU、激光雷达和 GNSS 节点，因为要运行的节点较多，将相应的 launch 放在底盘驱动的 “yhs_can_control_bringup.launch” 中，值需要运行

```
roslaunch yhs_can_control yhs_can_control_bringup.launch
```

```
yhs@yhs-ros:~$ roslaunch yhs_can_control yhs_can_control_bringup.launch
... logging to /home/yhs/.ros/log/343631f8-23e8-11ef-bb0c-fff83ab0b749/roslaunch-yhs-ros-132575.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:46131/

SUMMARY
=====
```

(2) GNSS 节点运行起来后，就要查看状态，输入指令敲回车，找到 “/gps/fixed” 话题，打印数据

```
rostopic list
```

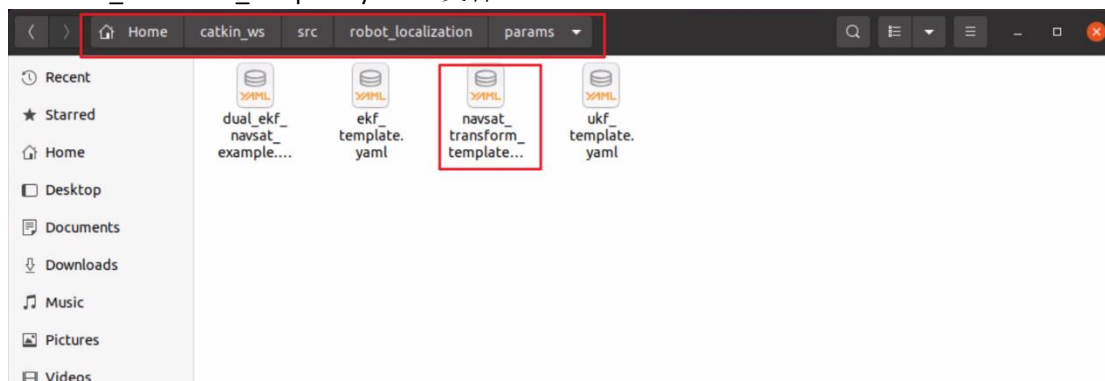
```
rostopic echo /gps/fixed
```

```
yhs@yhs-ros:~$ rostopic list
/gps/fixed
/gps/point
/gps/rpy
/nmea_msg
/rosout
/rosout_agg
/tf_static

yhs@yhs-ros:~$ rostopic echo /gps/fixed
header:
  seq: 340
  stamp:
    secs: 1717664858
    nsecs: 361438302
  frame_id: "gps_link"
status:
  status: 4
  service: 1
latitude: 22.69034314
longitude: 114.300877405
altitude: 58.1671
position_covariance: [0.000144, 0.0, 0.0, 0.0, 0.000144, 0.0, 0.0, 0.0, 0.002304]
position_covariance_type: 1
```

要 >= 4 才能录制 bag

(3) 进入 “/home/yhs/catkin_ws/src/robot_localization/params” 目录，可以看到有 “navsat_transform_template.yaml” 文件



打开该文件，找到 “wait_for_datum” 参数，改为 “false”，改完保存

```
40 # If true, will retrieve the datum from the 'datum' parameter below, if available. If no 'datum' parameter exists,
41 # navsat_transform_node will wait until the user calls the 'datum' service with the SetDatum service message.
42 wait_for_datum: false #建图为 false 导航时为 true
43
```

(4) 打开另外一个终端，输入指令运行 robot_localization 节点

```
roslaunch robot_localization navsat_transform_template.launch
```

```
yhs@yhs-ros:~$ roslaunch robot_localization navsat_transform_template.launch
... logging to /home/yhs/.ros/log/d3059e9a-23cf-11ef-bb0c-fff83ab0b749/roslaunch-yhs-ros-77116.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:33075/

process[navsat_transform_node-1]: started with pid [131202]
process[odom_to_base_link-2]: started with pid [131203]
[ INFO] [1717664900.226600851]: Waiting for valid clock time...
[ INFO] [1717664900.228041887]: Valid clock time received. Starting node.
[ INFO] [1717664903.247088902]: Initial odometry pose is Origin: (0.32999999804374058332 -3.5932314562114888953e-05 0)
Rotation (RPY): (0.025235594645456410623, -0.003586315846486050167, -0.00010888580191853894992)
[ INFO] [1717664903.247387870]: Datum (latitude, longitude, altitude) is (22.690343, 114.300877, 58.166400)
[ INFO] [1717664903.247466356]: Datum UTM coordinate is (222690.318265, 2511762.443027) zone 50
[ INFO] [1717664903.263823704]: Datum (latitude, longitude, altitude) is (22.690343, 114.300877, 58.165800)
[ INFO] [1717664903.263909727]: Datum UTM coordinate is (222690.318265, 2511762.443027) zone 50
[ INFO] [1717664903.280379873]: Corrected for magnetic declination of 0.000000, user-specified offset of 0.000000 and meridian convergence of -0.018123
4 Transform heading factor is now -0.018293
[ INFO] [1717664903.280485389]: Transform world frame pose is: Origin: (0.32999999804374058332 -3.5932314562114888953e-05 0)
Rotation (RPY): (0.025235594645456410623, -0.003586315846486050167, -0.00010888580191853894992)
[ INFO] [1717664903.280585213]: World frame->cartesian transform is Origin: (-176981.78239128142013 -2515396.3791738608852 -57.665963044585105024)
Rotation (RPY): (0, -0, 0.01818400806000667806)
```

输出这些信息才能开始录制bag

(5)再打开一个终端,进入“/home/yhs/3dbags”目录,执行脚本开始录制 bag,“out1.bag”是 bag 名,随便取,相同则会覆盖掉原来的

```
cd /home/yhs/3dbags/
```

```
./record_bag.sh out1.bag
```

```
yhs@yhs-ros:~$ cd /home/yhs/3dbags/
yhs@yhs-ros:~/3dbags$

yhs@yhs-ros:~/3dbags$ ./record_bag.sh out1.bag
[ INFO] [1717668039.056734961]: Subscribing to /gps/fixed
[ INFO] [1717668039.060420990]: Subscribing to /imu_data
[ INFO] [1717668039.062610744]: Subscribing to /odom
[ INFO] [1717668039.064883671]: Subscribing to /odometry/gps
[ INFO] [1717668039.067177371]: Subscribing to /tmllidar_points
[ INFO] [1717668039.069859308]: Recording to 'out1.bag'.
```

(6) 接下来的操作跟 建图过程不使用 GNSS 中的第 “(5)” 步及后面是一样的了

3、使用 GNSS 给 3D 导航下发初始点

怎么启动 3D 导航，请先看实验九

(1) 假设使用 GNSS 录制好的 bag 是 “out1.bag”，打开一个终端，输入指令

```
roscore
```

```
yhs@yhs-ros:~$ roscore
... logging to /home/yhs/.ros/log/6df31ac2-23ee-11ef-bb0c-fff83ab0b749/roslaunch-yhs-ros-141477.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:41261/
ros_comm version 1.16.0

SUMMARY
=====
PARAMETERS
* /roscdistro: noetic
* /rosversion: 1.16.0

NODES

auto-starting new master
process[master]: started with pid [141485]
ROS_MASTER_URI=http://192.168.1.102:11311/

setting /run_id to 6df31ac2-23ee-11ef-bb0c-fff83ab0b749
process[rosout-1]: started with pid [141495]
started core service [/rosout]
```

(2) 再打开一个终端，播放 “out1.bag”，开始播放后就可以按下 “空格键” 暂停了

```
rosbag play out1.bag
```

```
yhs@yhs-ros:~/3dbags$ rosbag play out1.bag
[ INFO ] [1717671848.946022452]: Opening out1.bag

Waiting 0.2 seconds after advertising topics... done.

Hit space to toggle paused, or 's' to step.
[[PAUSED ] Bag Time: 1717654192.423918 Duration: 0.455917 / 97.643776
```

(3) 打开另外一个终端，打印 “/gps/fixed” 话题数据，在播放的终端再按下 “空格键” 即可恢复播放，记录下这时的经纬度

```
rostopic echo /gps/fixed
```

```
yhs@yhs-ros:~/3dbags$ rostopic echo /gps/fixed
header:
  seq: 1896
  stamp:
    secs: 1717654192
    nsecs: 434709924
  frame_id: "gps_link"
status:
  status: 4
  service: 1
latitude: 22.690343845
longitude: 114.30098262666667
altitude: 58.1653
position_covariance: [0.0001, 0.0, 0.0, 0.0, 0.0001, 0.0, 0.0, 0.0, 0.0016]
position_covariance_type: 1
```



(4) 将记录好的经纬度填入“/home/yhs/catkin_ws/src/robot_localization/params”目录下的“navsat_transform_template.yaml”文件，改完保存后，关闭所有终端运行的程序

```
--
40 # If true, will retrieve the datum from the 'datum' parameter below, if available. If no 'datum' parameter exists,
41 # navsat_transform_node will wait until the user calls the 'datum' service with the SetDatum service message.
42 wait_for_datum: true #建图为 false 导航时为 true 改为 true
43
44 # Instead of using the first GPS location and IMU-based heading for the local-frame origin, users can specify the
45 # origin (datum) using this parameter. The fields in the parameter represent latitude and longitude in decimal degrees,
46 # and heading in radians. As navsat_transform_node assumes an ENU standard, a 0 heading corresponds to east.
47 datum: [22.6903438, 114.30098265, 0.0] #导航时填入刚开始建图时的经纬度
48
49
50
51
52 经度 纬度
53
54
55
```

(5) 启动 3D 导航

(6) 打开另外一个终端，输入指令启动 GNSS 节点

```
roslaunch nmea_ros_driver bringup.launch
```

```
yhs@yhs-ros:~$ roslaunch nmea_ros_driver bringup.launch
. logging to /home/yhs/.ros/log/d3059e9a-23cf-11ef-bb0c-fff83ab0b749/roslaunch-yhs-ros-77033.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:40231/

SUMMARY
=====
PARAMETERS
 * /nmea_analyze_node/enable_custom: False
 * /nmea_analyze_node/enable_gga: False
 * /nmea_analyze_node/enable_gll: False
 * /nmea_analyze_node/enable_gsa: False
 * /nmea_analyze_node/enable_gsv: False
 * /nmea_analyze_node/enable_rmc: False
 * /nmea_analyze_node/enable_vtg: False
 * /nmea_analyze_node/frame_id: gps_link
 * /nmea_analyze_node/nmea_topic_name: /nmea_msg
 * /nmea_ros_driver/baudrate: 115200
 * /nmea_ros_driver/nmea_com: /dev/ttyS0
 * /roscdistro: noetic
 * /rosversion: 1.16.0

NODES
 /
  gnss_to_base_link (tf2_ros/static_transform_publisher)
  nmea_analyze_node (nmea_ros_driver/nmea_analyze_node)
  nmea_ros_driver (nmea_ros_driver/nmea_driver_node)

ROS_MASTER_URI=http://192.168.1.102:11311

process[nmea_ros_driver-1]: started with pid [77057]
process[nmea_analyze_node-2]: started with pid [77058]
process[gnss_to_base_link-3]: started with pid [77060]
[ INFO] [1717656113.551267882]: Open serial port:/dev/ttyS0 successful!!
```

(7) 打开另外一个终端，查看状态

```
rostopic echo /gps/fixd
```

```
yhs@yhs-ros:~$ rostopic echo /gps/fixd
header:
  seq: 340
  stamp:
    secs: 1717664858
    nsecs: 361438302
  frame_id: "gps_link"
status:
  status: 4
  service: 1
latitude: 22.69034314
longitude: 114.300877405
altitude: 58.1671
position_covariance: [0.000144, 0.0, 0.0, 0.0, 0.000144, 0.0, 0.0, 0.0, 0.002304]
position_covariance_type: 1
```

确保 ≥ 4

(8) 打开另外一个终端，输入指令运行 `robot_localization` 节点

```
roslaunch robot_localization navsat_transform_template.launch
```

```
yhs@yhs-ros:~$ roslaunch robot_localization navsat_transform_template.launch
... logging to /home/yhs/.ros/log/d3059e9a-23cf-11ef-bb0c-fff83ab0b749/roslaunch-yhs-ros-77116.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.
```

```
started roslaunch server http://192.168.1.102:33075/
```

```
ROS_MASTER_URI=http://192.168.1.102:11311
```

```
process[navsat_transform_node-1]: started with pid [77142]
```

```
process[odom_to_base_link-2]: started with pid [77143]
```

```
[ INFO] [1717656118.647065610]: Waiting for valid clock time...
```

```
[ INFO] [1717656118.648210183]: Valid clock time received. Starting node.
```

```
现在用的是设定好的位置(22.6903,114.301)，注意是否处于建图。
```

```
[ INFO] [1717656118.674334792]: Datum (latitude, longitude, altitude) is (22.690344, 114.300983, 0.000000)
```

```
[ INFO] [1717656118.674410296]: Datum UTM coordinate is (222701.141565, 2511762.316201) zone 50
```

```
[ INFO] [1717656118.674458860]: Initial odometry pose is Origin: (0 0 0)
```

```
Rotation (RPY): (0, -0, 0)
```

```
[ INFO] [1717656121.715811305]: Corrected for magnetic declination of 0.000000, user-specified offset of 0.000000 and meridian convergence of -0.018183
```

```
3. Transform heading factor is now -0.018183
```

```
[ INFO] [1717656121.715862859]: Transform world frame pose is: Origin: (0 0 0)
```

```
Rotation (RPY): (0, -0, 0)
```

```
[ INFO] [1717656121.715898468]: World frame->cartesian transform is Origin: (-176995.22384625978884 -2515396.2753958939575 0)
```

```
Rotation (RPY): (0, -0, 0.018183097901180639289)
```

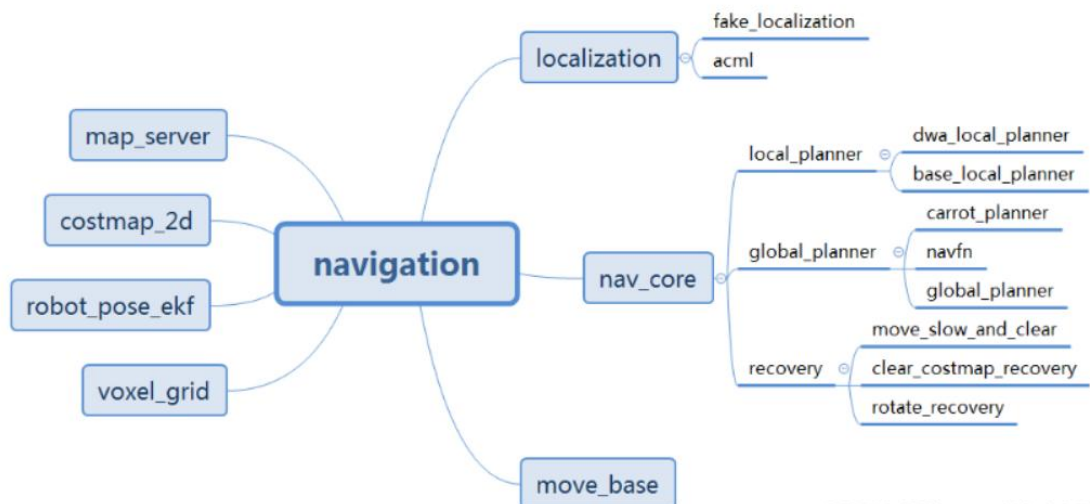
打印此信息

(9) 在 Rviz 上观察，是否定位成功，定位成功关闭 GNSS 节点和 `robot_localization` 节点，至此下发初始位置结束

实验八 Navigation 导航框架简介

将建好的地图进行机器人导航前，先来了解一下机器人导航的原理。导航模块主要是用来做路径规划，这是一个相对比较复杂的模块，因为需要和定位、地图以及控制模块交互，所以涉及到的模块相对比较多。Navigation (<https://github.com/ros-planning/navigation>) 提供了一套框架，我们来了解框架的结构组成。

1、Navigation 里面的功能包如下图：

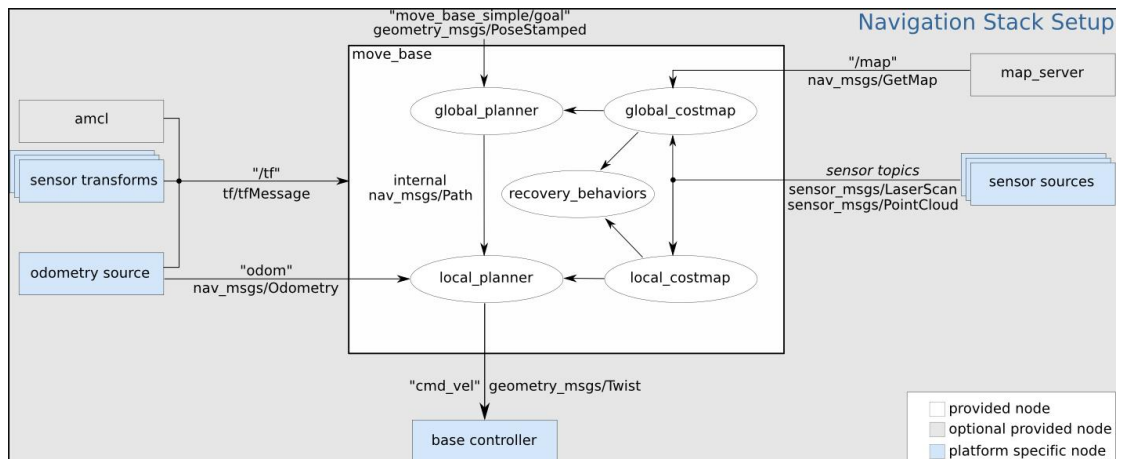


每个功能包的功能如下表：

功能包	作用
amcl	amcl 的作用是根据机器人自身的里程数值以及地图特征，利用粒子滤波修正机器人在已知的地图内的位置
base_local_planner	局部路径规划器
dwa_local_planner	局部路径规划器，使用动态窗口法
carrot_planner	全局路径规划器，生成的路径为目标点到机器人当前点的连线上的点
clear_costmap_recovery	无法规划路径的恢复算法
costmap_2d	代价地图实现
fake_localization	主要用来做定位仿真
global_planner	全局路径规划算法
map_server	提供地图的管理服务
move_base	机器人移动导航框架（导航最主要的逻辑框架）
move_slow_and_clear	一种恢复策略
nav_core	提供接口，能够实现插件更换算法的主要包
nav_fn	全局路径规划算法
robot_pose_ekf	综合里程计、GPS、imu 数据，通过拓展卡尔曼滤波进行位置估计
rotate_recovery	旋转恢复策略实现包
voxel_grid	三维代价地图

2、将 navigation 里面的功能包联通起来，靠的是“move_base”。

(1) 框架如下图：



(2) 根据上面的框架图，我们来看下要提供哪些数据，并且要用到哪些硬件传感器。

map_server	要用到建好的栅格地图
odometry source	要用到融合 IMU 后的里程计
Sensor sources	要用到激光雷达

所有的数据都准备好后，接下来就可以进行导航测试了。

实验九 Rviz 下点自主导航

1、2D 导航

1.1、选择地图并初始化

(1) 到一个新的区域，建好一张新的地图并且保存后，想将此地图用于导航，那么就要在 launch 文件里修改，进入 “/home/yhs/catkin_ws/src/yhs_nav/launch” 目录，可以看到 “yhs_nav_2d.launch.py” 文件

```
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/launch$ ls
cartographer.launch  gmapping.launch  yhs_nav_2d.launch  yhs_nav.launch
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/launch$
```

(2) 用 “Text Editor” 或者 “vim” 都可以打开，打开文件后，找到下图的位置，按照图片上的方式修改完保存即可

```
1 <!-- 2D导航launch文件 -->
2
3 <launch>
4
5 <!-- 载入参数文件 -->
6 <roslaunch file="/home/yhs/catkin_ws/src/yhs_nav/param/yhs_nav_param.yaml" command="load" />
7
8 <!-- 2D导航需要 odom to base_link -->
9 <param name="/yhs_can_control/tfUsed" value="true" />
10
11 <!-- 加载地图节点 -->
12 <arg name="map_name" default="g1" />
13 <arg name="map_file" default="$(find yhs_nav)/map/$(arg map_name).yaml" />
14 <node name="map_server" pkg="map_server" type="map_server" args="$(arg map_file)" />
15
16 <!-- 启动底盘驱动 -->
17 <include file="$(find yhs_can_control)/launch/yhs_can_control.launch" />
18 <arg name="is_load_param" value="false" />
19 </include>
20
21 <!-- 启动IMU节点 -->
22 <include file="$(find serial_imu)/launch/imu_msg.launch" />
23
24 <!-- 启动激光节点 -->
25 <include file="$(find rslidar_sdk)/launch/start.launch" />
26 <!-- <include file="$(find timoo_pointcloud)/launch/TM16.launch" /> -->
27 <!-- <include file="$(find lslidar_cx_driver)/launch/lslidar_cx.launch" /> -->
28
```

保存栅格地图完成，会生成pgm和yaml文件，这里修改为要用于导航的地图名字

(3) 选择完地图后，就可以启动导航了，打开终端，输入指令后敲回车，看到 “odom received!” 后，不一定是最后一行输出，导航启动完成，否则要检查终端输出的信息，查找问题

```
roslaunch yhs_nav yhs_nav_2d.launch
```

```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav_2d.launch
... logging to /home/yhs/.ros/log/17a28978-22e0-11ef-92e5-0b8e29a6dc24/roslaunch-yhs-ros-3832.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

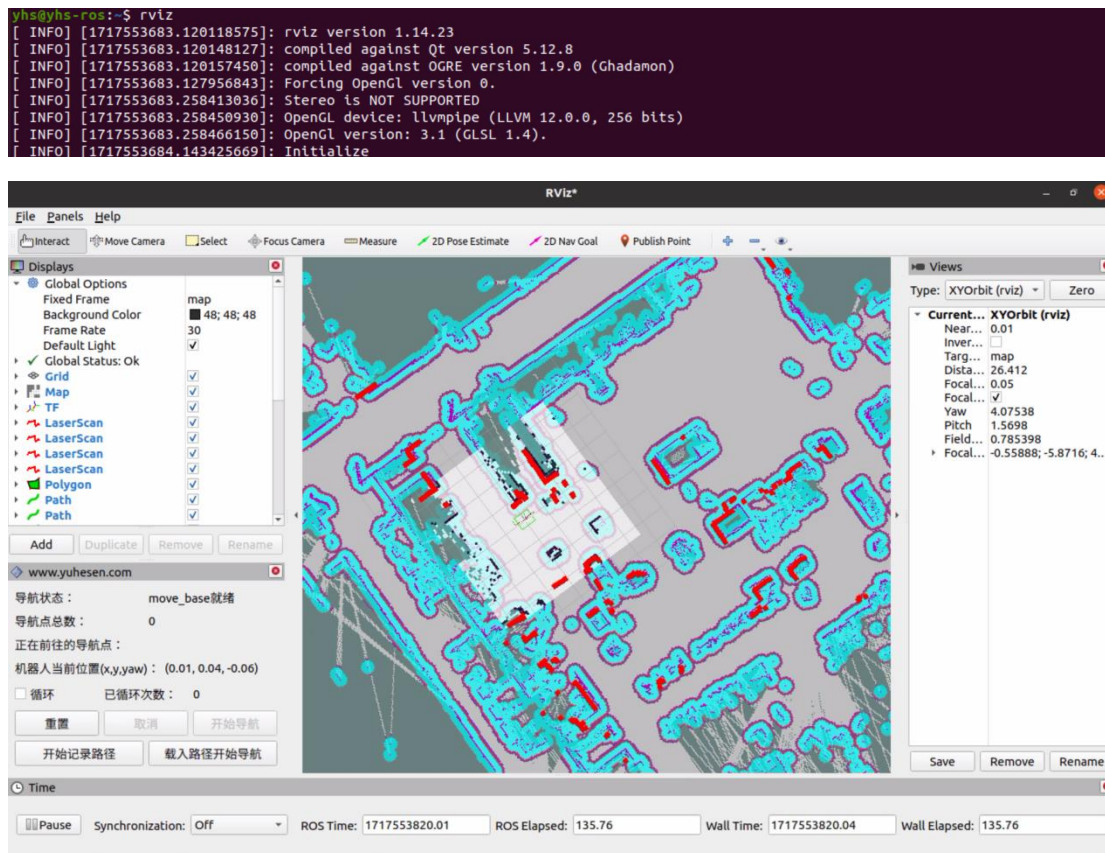
started roslaunch server http://192.168.1.102:45043/

SUMMARY
=====
CLEAR PARAMETERS
* /move_base/

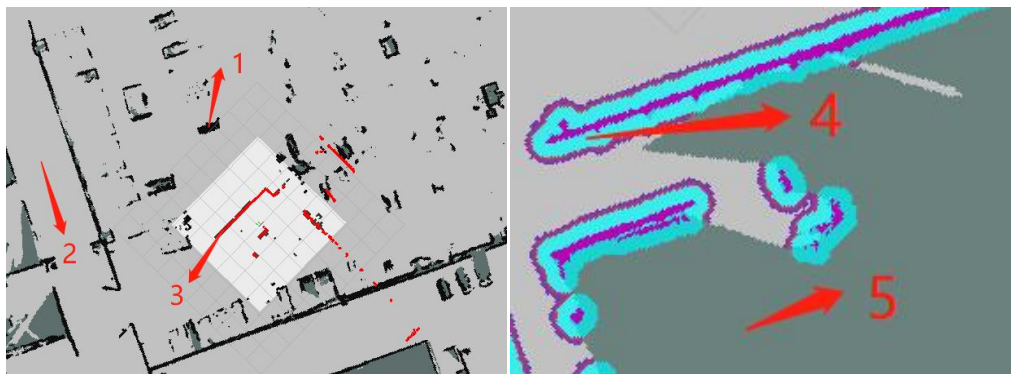
[ INFO] [1717553136.750830953]: Footprint model 'circular' (radius: 0.2m) loaded for trajectory optimization.
[ INFO] [1717553136.853175943]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.858632746]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.877216340]: odom received!
```

(4) 打开另外一个终端，输入指令后敲回车后，弹出 Rviz 界面，可以看到地图、激光等数据

rviz

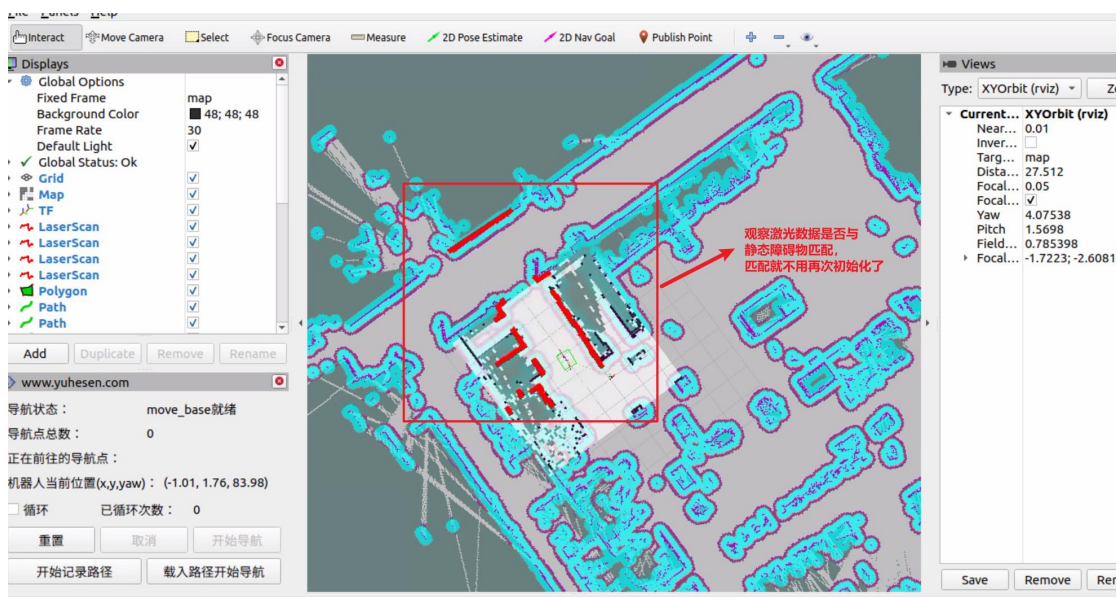
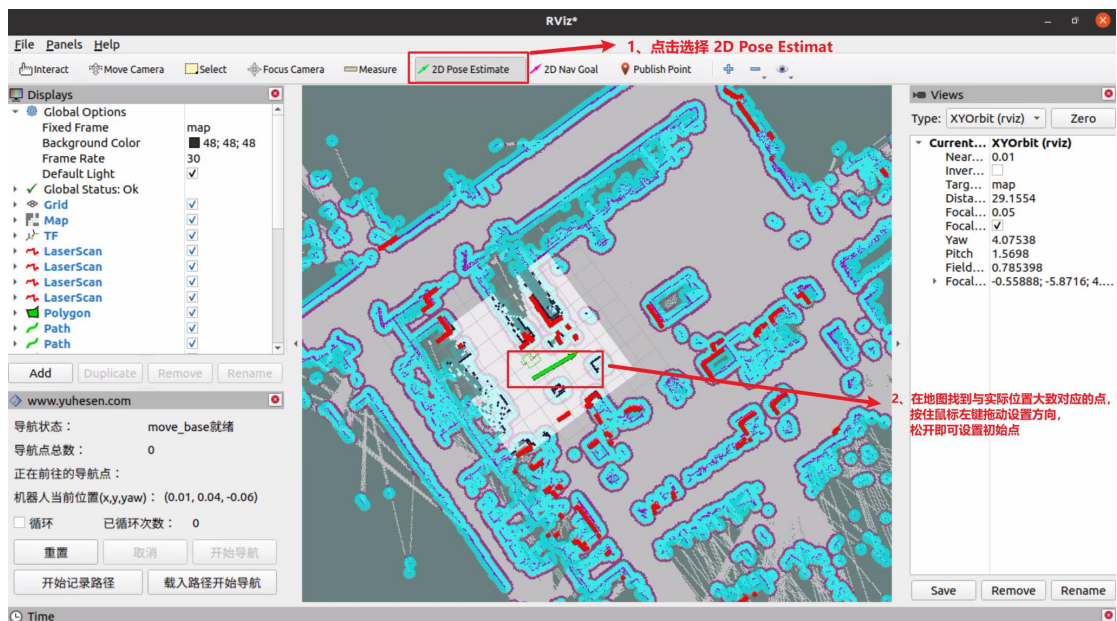


地图上各种颜色代表的含义如下



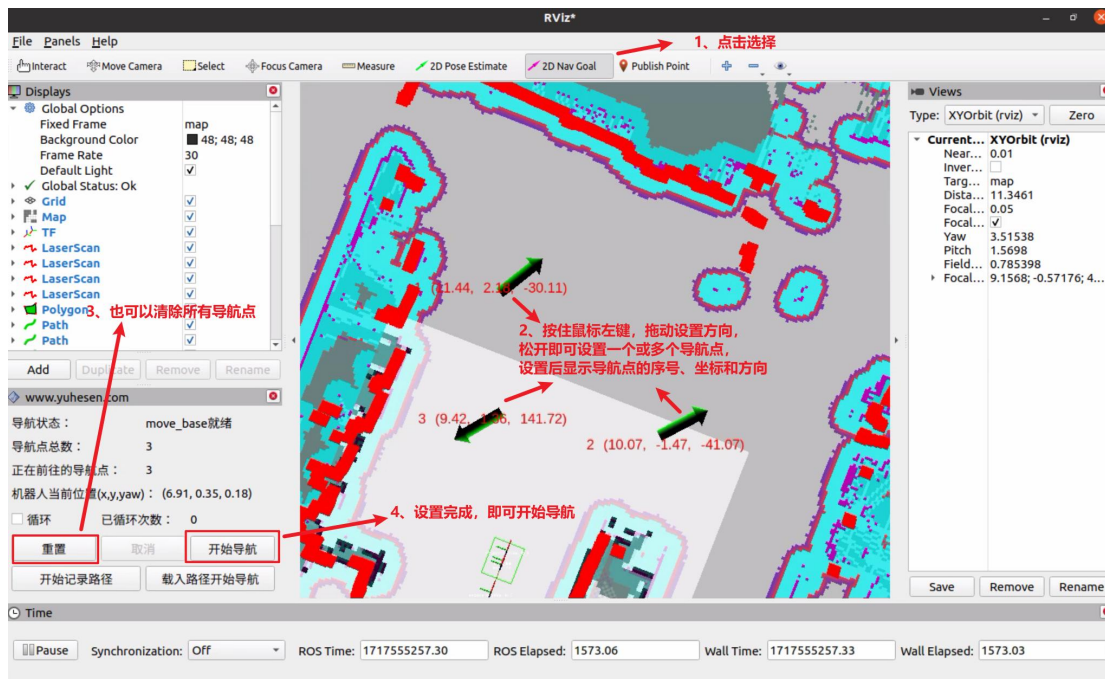
1	黑色区域表示静态障碍物
2	浅灰色区域表示可行区域
3	红色表示激光数据
4	蓝色区域表示膨胀区域
5	深灰色表示未知区域

(5) 设置初始化点，启动导航前，机器人的位置通常不会在建图起点位置，所以要设置初始化点，按照下图的操作设置初始化点，设置后，观察红激光点是否跟周围的黑色静态障碍物大致对应上，如果相差较大，可以再次设置

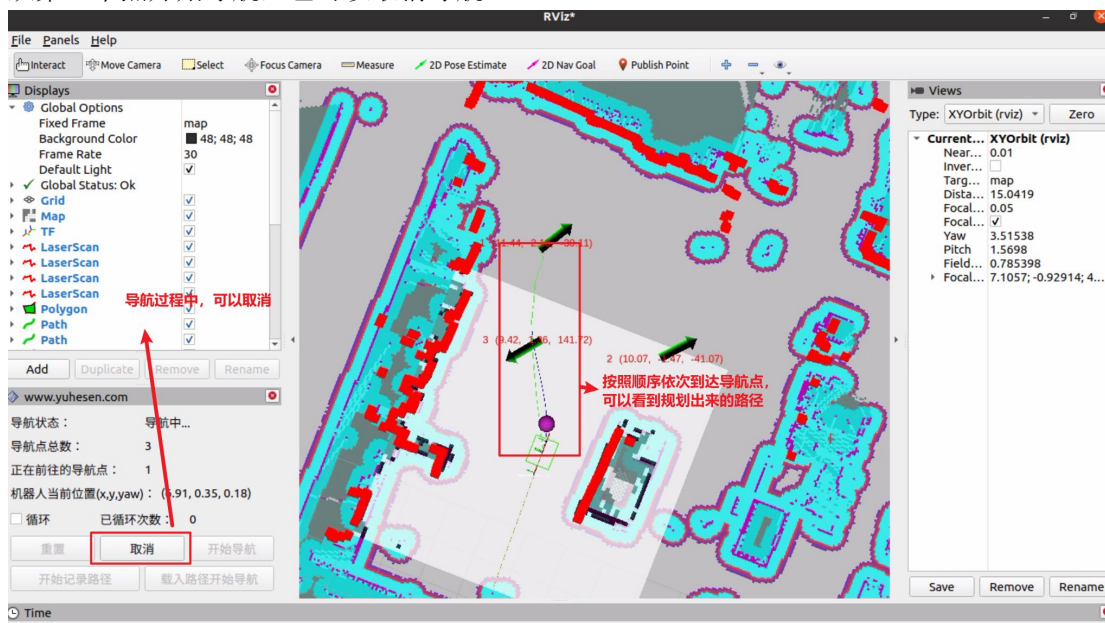


1.2、单点/多点导航

(1) 按照下图的操作

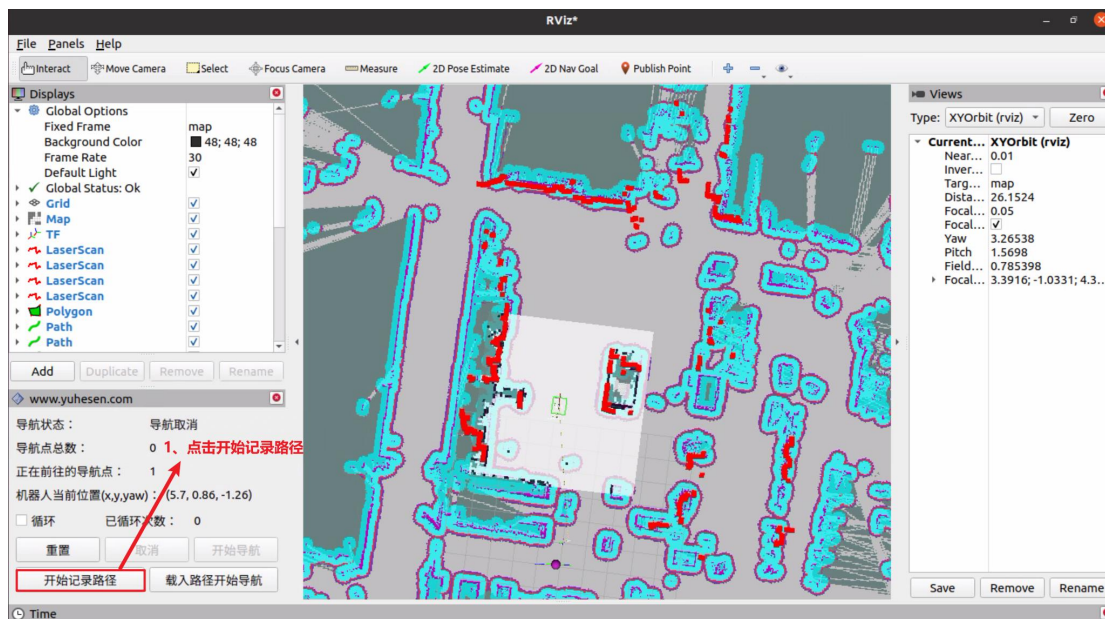


(2) 开始导航后, 可以看到规划出来的路径, 切换遥控器到指令模式, 机器人会按照路径行驶至目标点, 左下角有导航状态反馈, 如果勾选“循环”, 到达最后一个点后, 再次从第一个点开始导航, 也可以取消导航

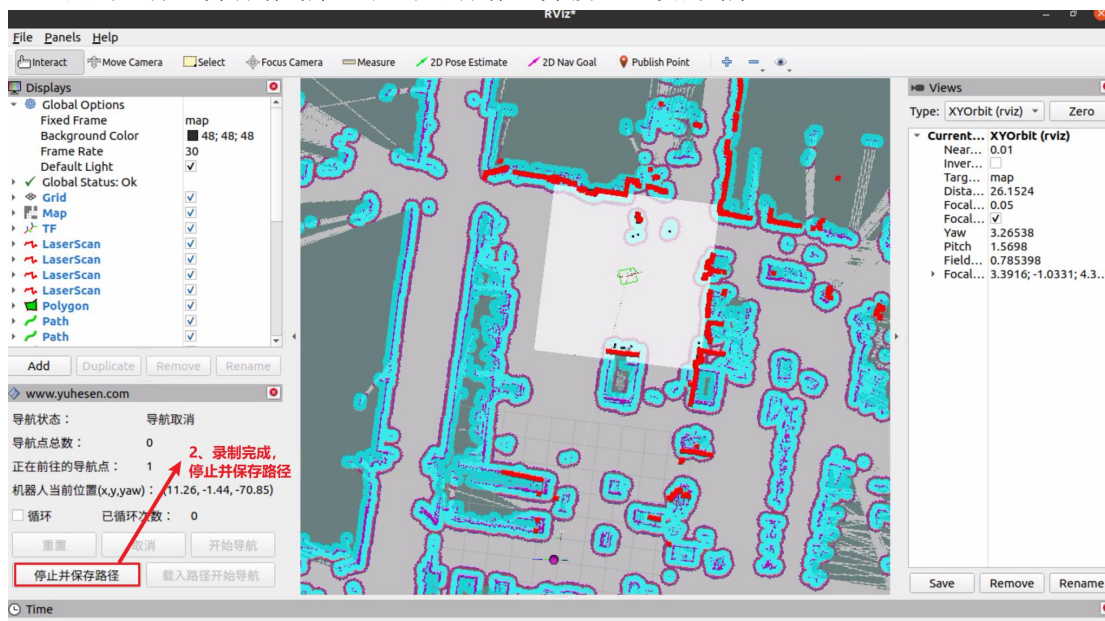


1.3、录制路径导航

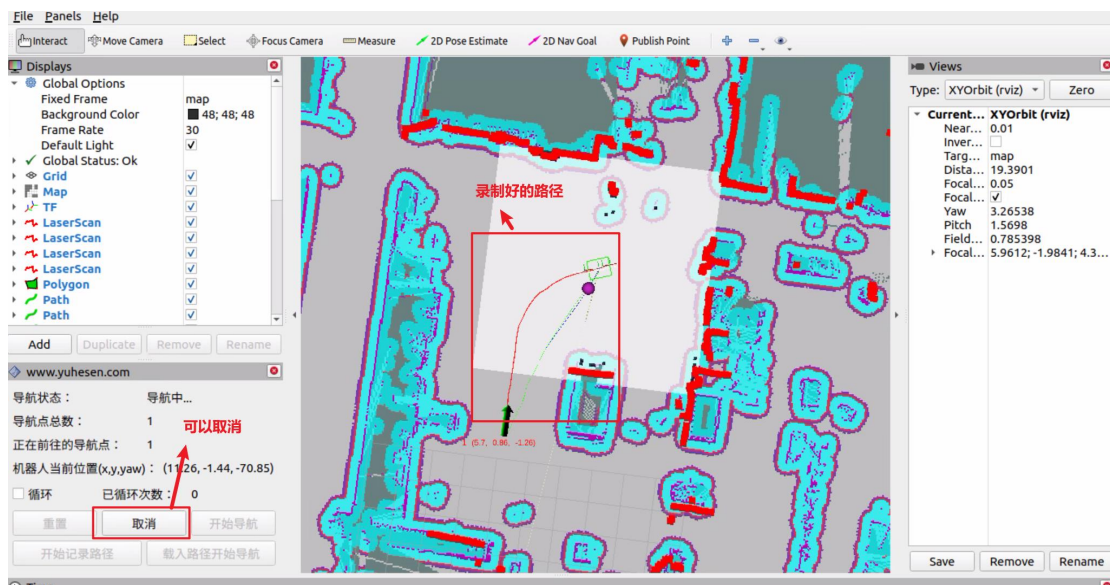
(1) 按照下面图片中的操作



(2) 这时就可以遥控机器人行走了，程序会实时记录路径，路径不能有交叉，不能后退，点击“停止并保存路径”后即可保存，并覆盖上次的路径



(3) 保存完路径后，点击“载入路径并开始导航”，机器人首先会导航到路径的起点，然后把录制好的路径当做全局路径，跟随录制路径到最后一个点，导航过程中，若路径上有障碍物，则会停下来等待，中间可以取消导航



2、3D 导航

2.1、选择地图并初始化

(1) 到一个新的区域，建好一张新的 3D 点云图并且转换得到栅格地图后，想将此地图用于导航，那么就要在 launch 文件里修改，进入“/home/yhs/catkin_ws/src/yhs_nav/launch”目录，可以看到“yhs_nav.launch.py”文件

```
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/launch$ ls
cartographer.launch  gmapping.launch  yhs_nav_2d.launch  yhs_nav.launch
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/launch$
```

(2) 用“Text Editor”或者“vim”都可以打开，打开文件后，找到下图的位置，按照图片上的方式修改完保存即可

```
1 <!-- 3D导航launch文件 -->
2
3 <launch>
4
5 <!-- 载入参数文件 -->
6 <roscpp param file="/home/yhs/catkin_ws/src/yhs_nav/param/yhs_nav_param.yaml" command="load" />
7
8 <!-- 3D导航不需要 odom to base_link -->
9 <param name="/yhs_can_control/tfused" value="false" />
10
11 <!-- 加载地图节点 -->
12 <arg name="map_name" default="ln2" />
13 <arg name="map_file" default="$(find yhs_nav)/map/${arg map_name}.yaml" />
14 <node name="map_server" pkg="map_server" type="map_server" args="$(arg map_file)" />
15
16 <!-- 启动底盘驱动 -->
17 <include file="$(find yhs_can_control)/launch/yhs_can_control.launch" />
18 <arg name="is_load_param" value="false" />
19 </include>
20
```

此处修改为点云图转换得到的栅格地图名字

(3) 选择完地图后，就可以启动导航了，打开终端，输入指令后敲回车，看到“odom received!”后，不一定是最后一行输出，导航启动完成，否则要检查终端输出的信息，查找问题

```
roslaunch yhs_nav yhs_nav.launch
```

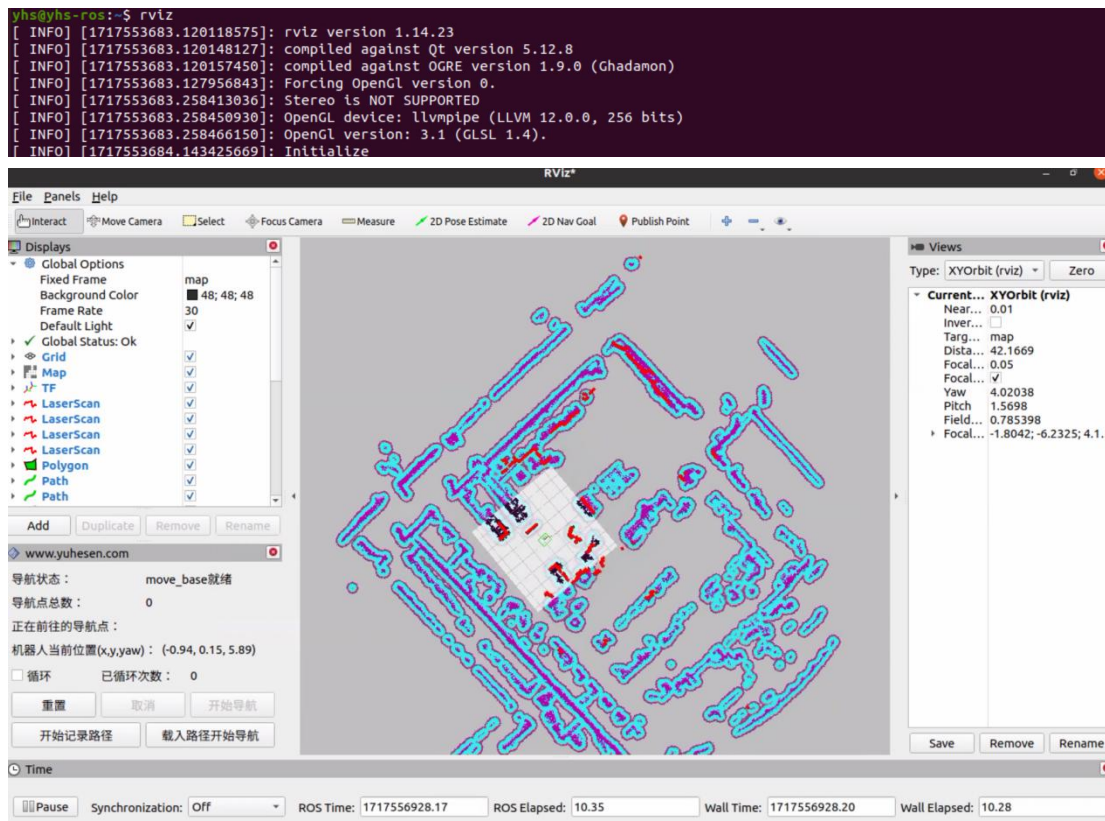
```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav.launch
... logging to /home/yhs/.ros/log/a995d530-22e8-11ef-92e5-0b8e29a6dc24/roslaunch-yhs-ros-5525.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:40445/
```

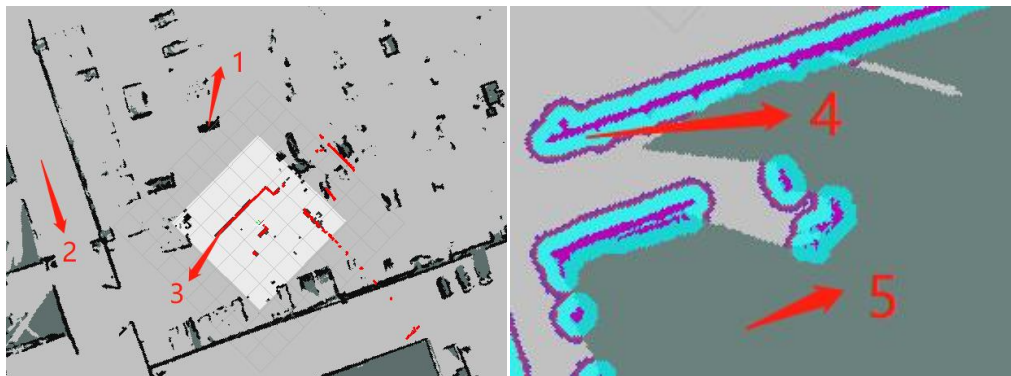
```
[ INFO] [1717553136.750830953]: Footprint model 'circular' (radius: 0.2m) loaded for trajectory optimization.
[ INFO] [1717553136.853175943]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.858632746]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.877216340]: odom received!
```

(4) 打开另外一个终端，输入指令后敲回车后，弹出 **rviz** 界面，可以看到地图、激光等数据

rviz

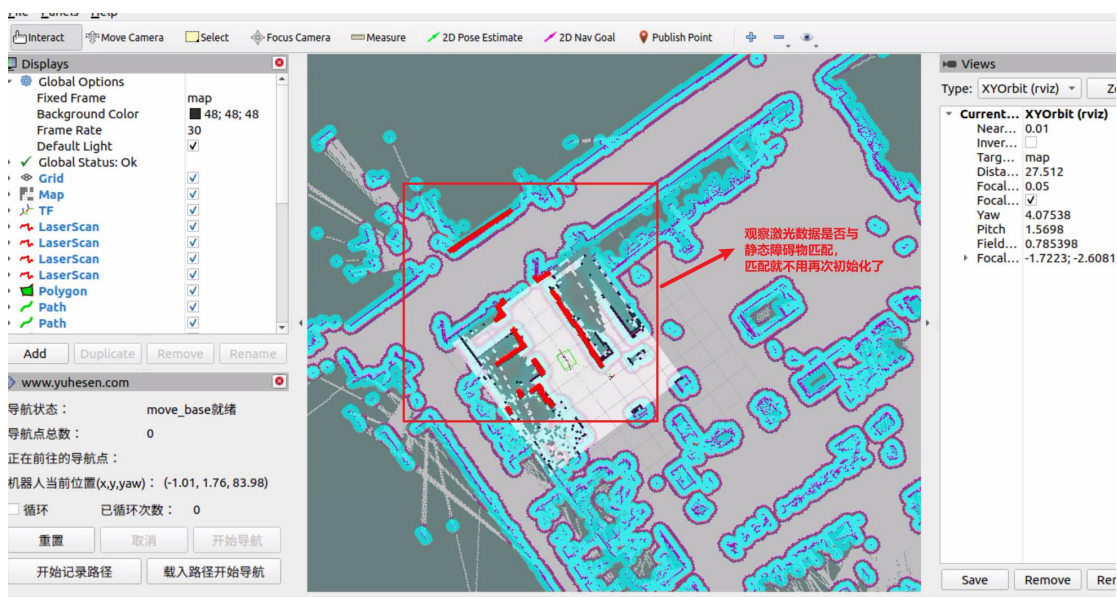
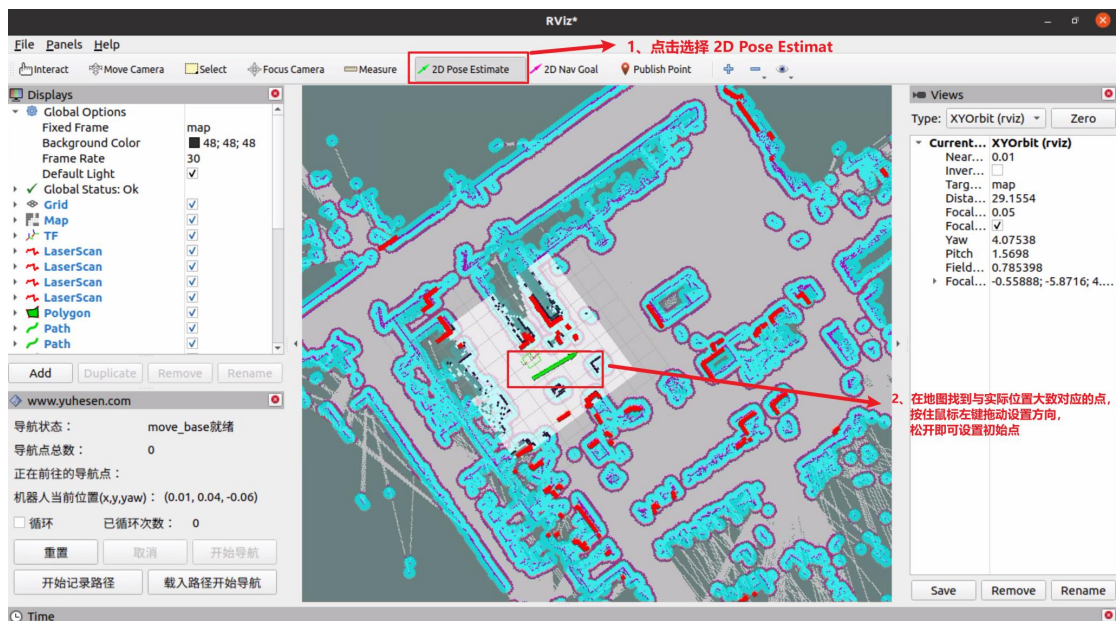


地图上各种颜色代表的含义如下



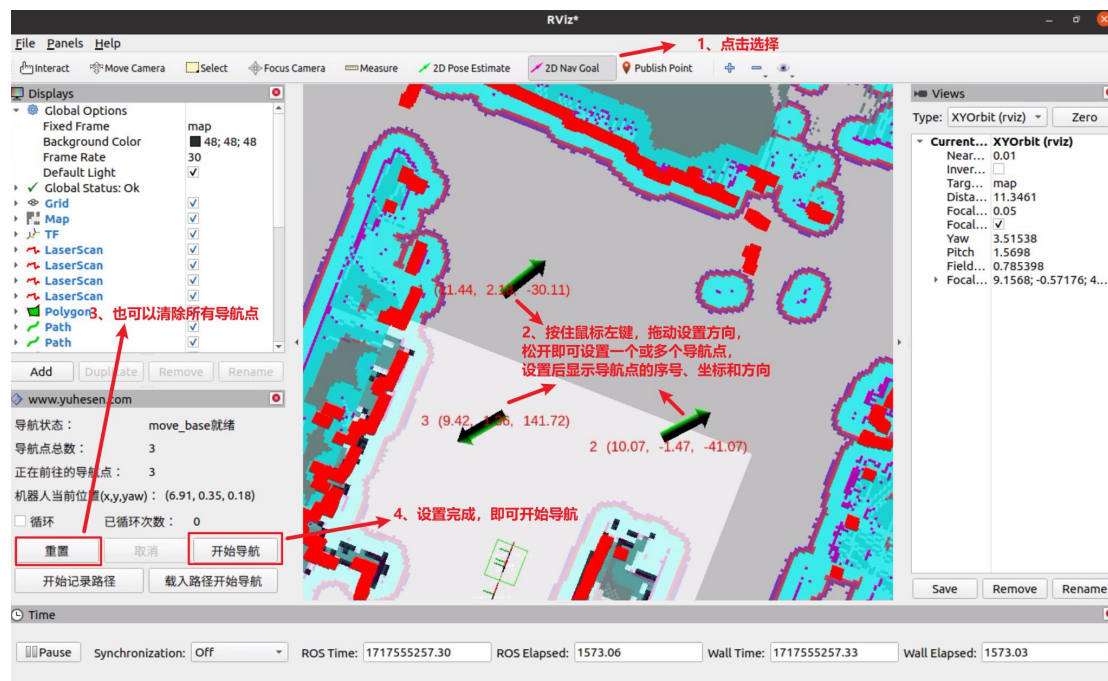
1	黑色区域表示静态障碍物
2	浅灰色区域表示可行区域
3	红色表示激光数据
4	蓝色区域表示膨胀区域
5	深灰色表示未知区域

(5) 设置初始化点，启动导航前，机器人的位置通常不会在建图起点位置，所以要设置初始化点，按照下图的操作设置初始化点，设置后，观察红激光点是否跟周围的黑色静态障碍物大致对应上，如果相差较大，可以再次设置

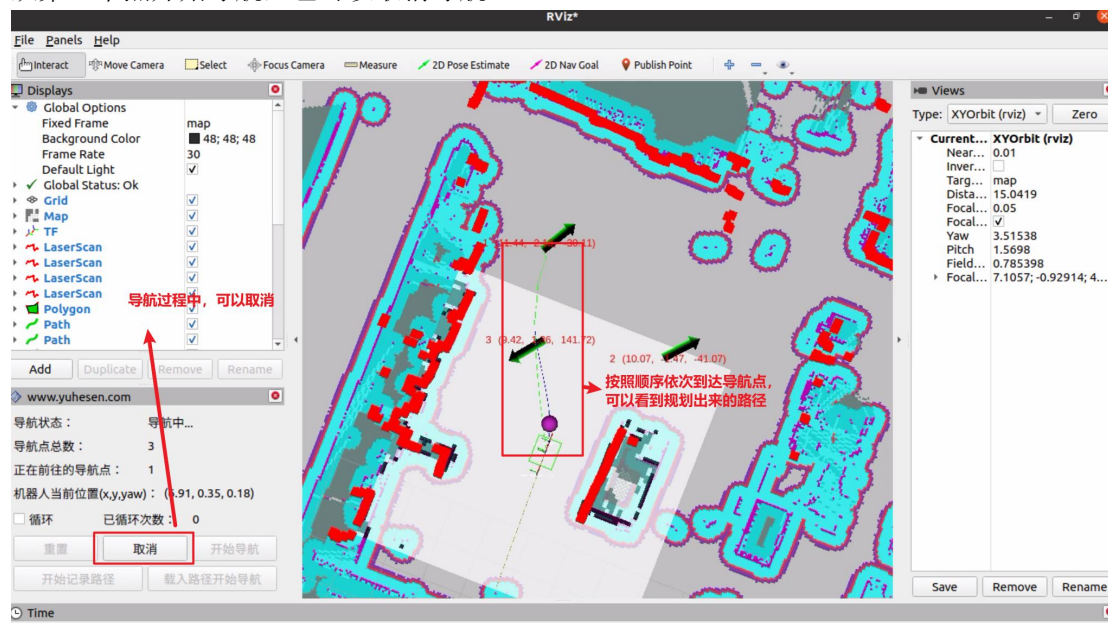


2.2、单点/多点导航

(1) 按照下图的操作

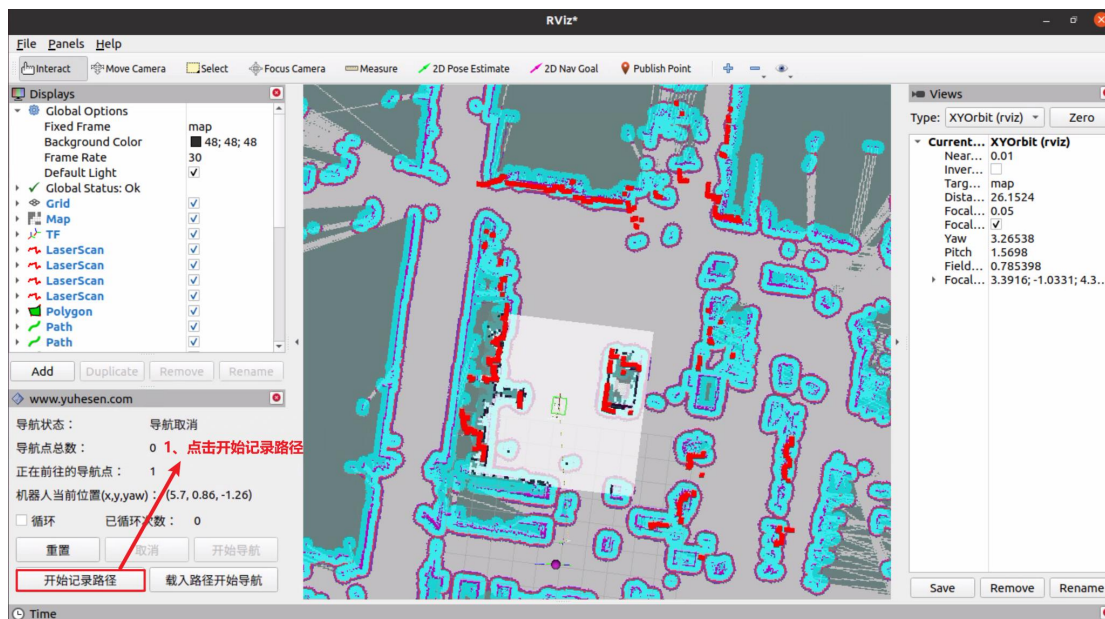


(2) 开始导航后，可以看到规划出来的路径，切换遥控器到指令模式，机器人会按照路径行驶至目标点，左下角有导航状态反馈，如果勾选“循环”，到达最后一个点后，再次从第一个点开始导航，也可以取消导航

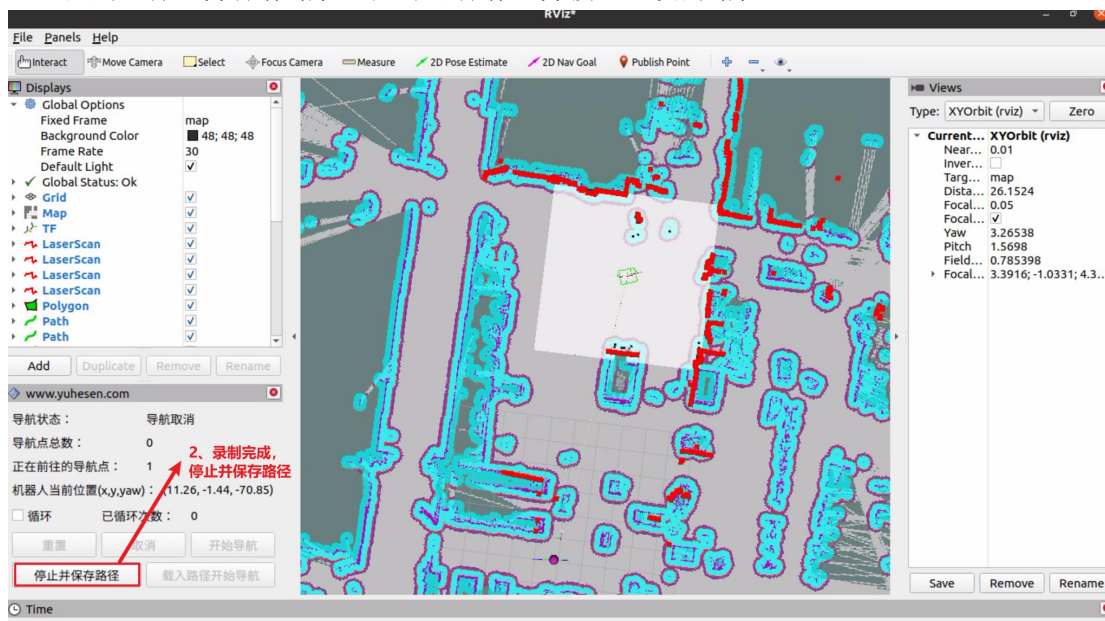


2.3、录制路径导航

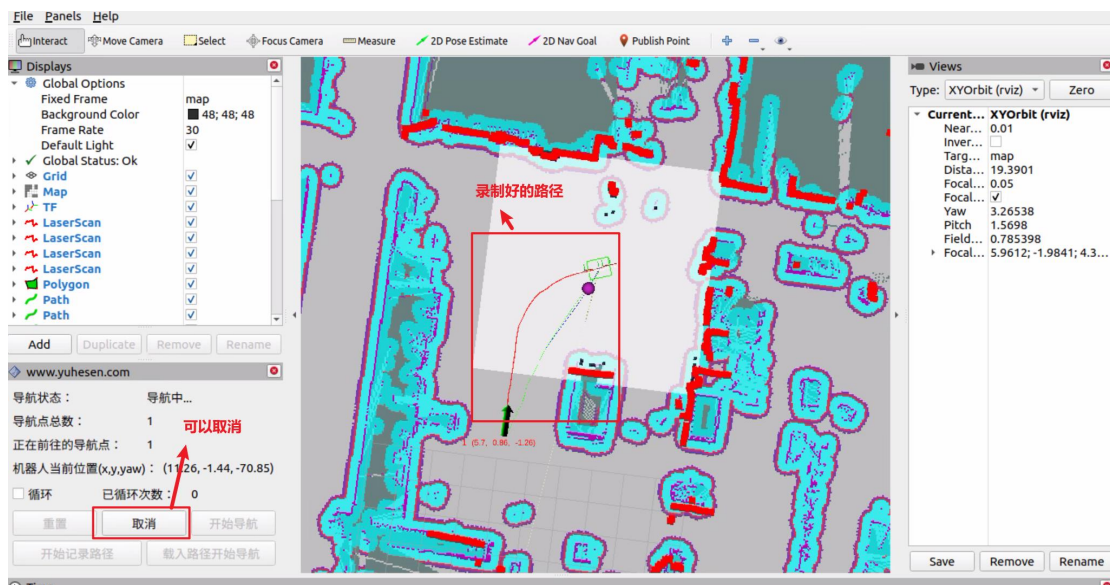
(1) 按照下面图片中的操作



(2) 这时就可以遥控机器人行走了，程序会实时记录路径，路径不能有交叉，不能后退，点击“停止并保存路径”后即可保存，并覆盖上次的路径



(3) 保存完路径后，点击“载入路径并开始导航”，机器人首先会导航到路径的起点，然后把录制好的路径当做全局路径，跟随录制路径到最后一个点，导航过程中，若路径上有障碍物，则会停下来等待，中间可以取消导航



实验十 自主导航 Node 的编程

上一节我们在 Rviz 里通过图形化操作来让机器人完成导航任务，这一节我们将编写代码来实现这个过程。

1、创建 ROS 源码包

(1) 打开一个终端程序，输入指令进入 “cd /home/yhs/catkin_ws/src/yhs_tutorials” 目录

```
cd /home/yhs/catkin_ws/src/yhs_tutorials
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws/src/yhs_tutorials/
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$
```

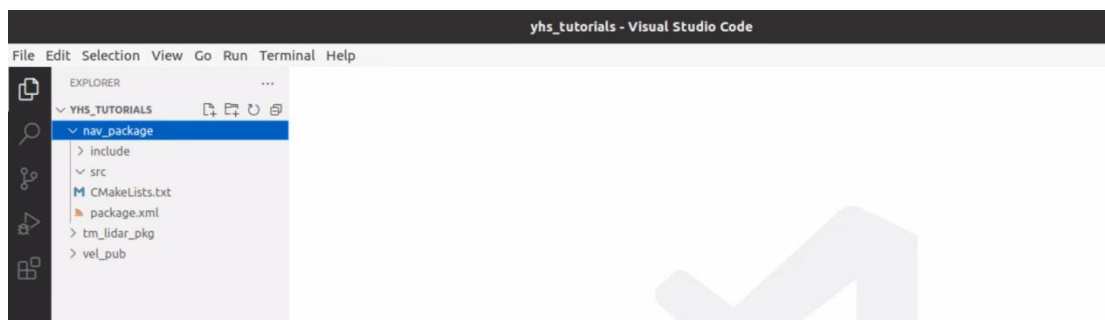
(2) 使用如下指令创建一个新的 ROS 软件包 (Package)

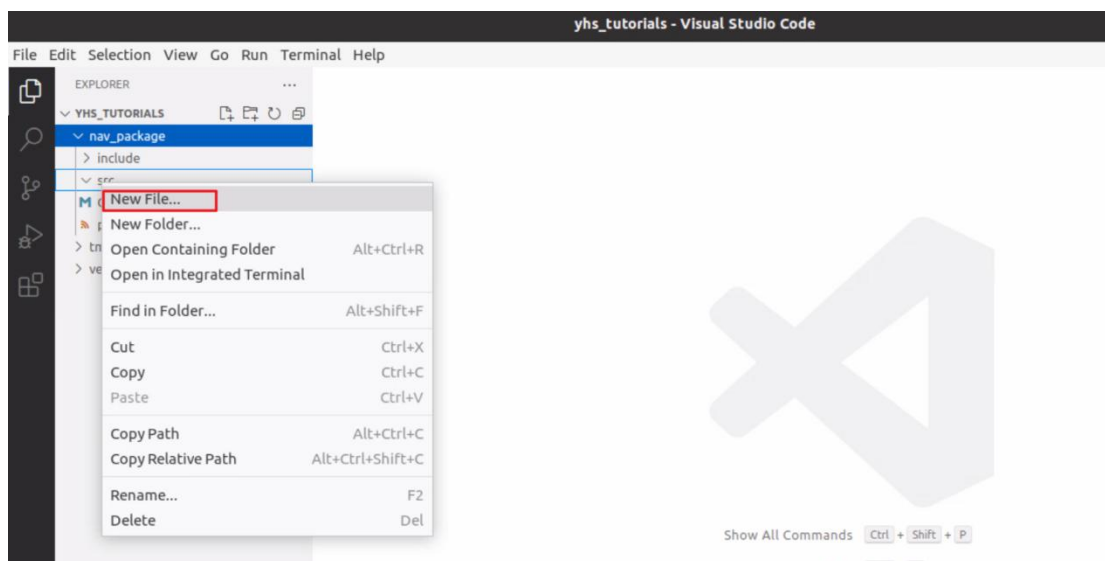
```
catkin_create_pkg nav_package roscpp move_base_msgs actionlib
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$ catkin_create_pkg nav_package roscpp move_base_msgs actionlib
Created file nav_package/package.xml
Created file nav_package/CMakeLists.txt
Created folder nav_package/include/nav_package
Created folder nav_package/src
Successfully created files in /home/yhs/catkin_ws/src/yhs_tutorials/nav_package. Please adjust the values in package.xml.
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$
```

这条指令的具体含义是：

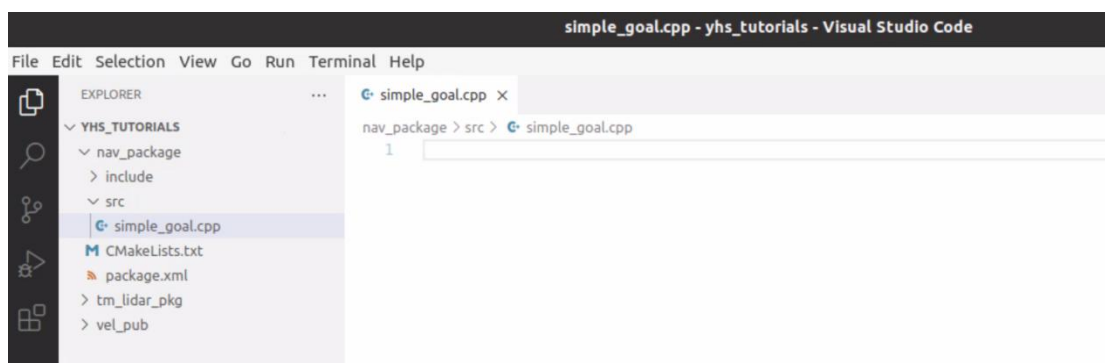
指令	含义
catkin_create_pkg	创建 ROS 源码包 (package) 的指令
nav_package	新建的 ROS 源码包命名
roscpp	C++ 依赖项，本例程使用 C++ 编写，所以需要这个依赖项
move_base_msgs	move_base 导航消息依赖项
actionlib	actionlib 依赖项，Navigation 的编程接口为 actionlib 格式

(3) 创建 “nav_package” 包后，用 IDE 打开 “yhs_tutorials”，可以看到 “yhs_tutorials” 目录下多了一个 “nav_package” 文件夹，在其 “src” 子文件夹上右键点击鼠标，选择 “New File” 新建一个代码文件。（这里我们用的是 visual studio code，也可以使用 vim、Text Editor 等其他文本编辑工具打开或创建）

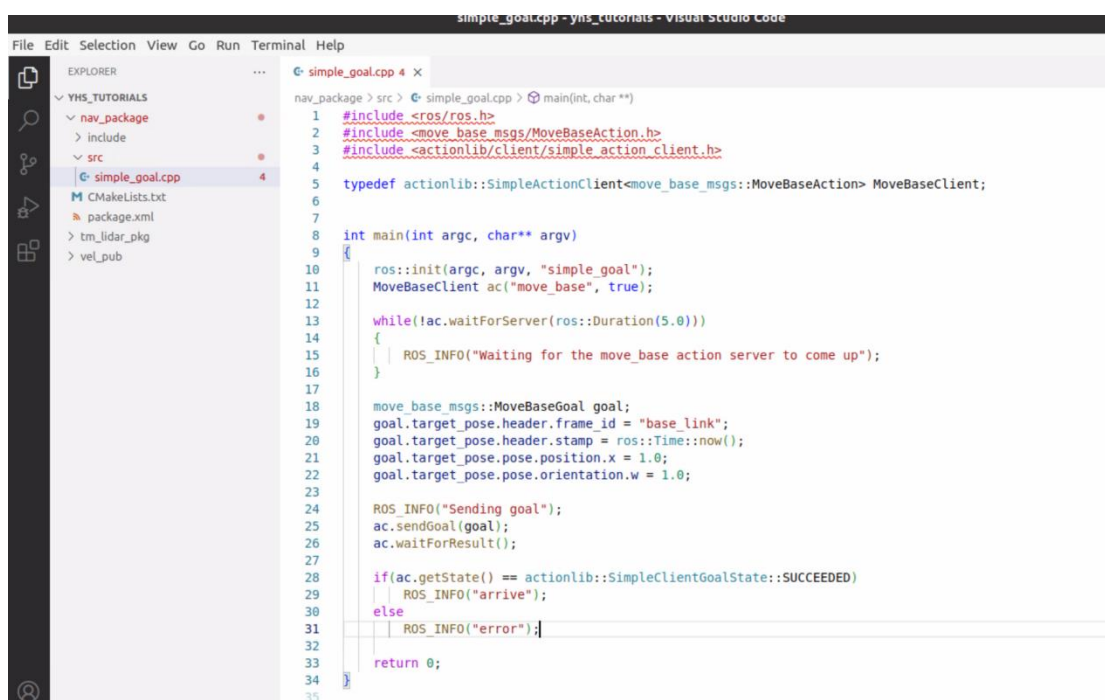




(4) 新建的代码文件，我们为其命名为“simple_goal.cpp”。



(5) 命名完毕后，在 IDE 的右侧可以开始编写“simple_goal.cpp”的代码，其内容如下：



```
#include <ros/ros.h>
#include <move_base_msgs/MoveBaseAction.h>
#include <actionlib/client/simple_action_client.h>

typedef actionlib::SimpleActionClient<move_base_msgs::MoveBaseAction> MoveBaseClient;

int main(int argc, char** argv)
{
    ros::init(argc, argv, "simple_goal");
    MoveBaseClient ac("move_base", true);

    while(!ac.waitForServer(ros::Duration(5.0)))
    {
        ROS_INFO("Waiting for the move_base action server to come up");
    }

    move_base_msgs::MoveBaseGoal goal;
    goal.target_pose.header.frame_id = "base_link";
    goal.target_pose.header.stamp = ros::Time::now();
    goal.target_pose.pose.position.x = 1.0;
    goal.target_pose.pose.orientation.w = 1.0;

    ROS_INFO("Sending goal");
    ac.sendGoal(goal);
    ac.waitForResult();

    if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED)
        ROS_INFO("arrive");
    else
        ROS_INFO("error");

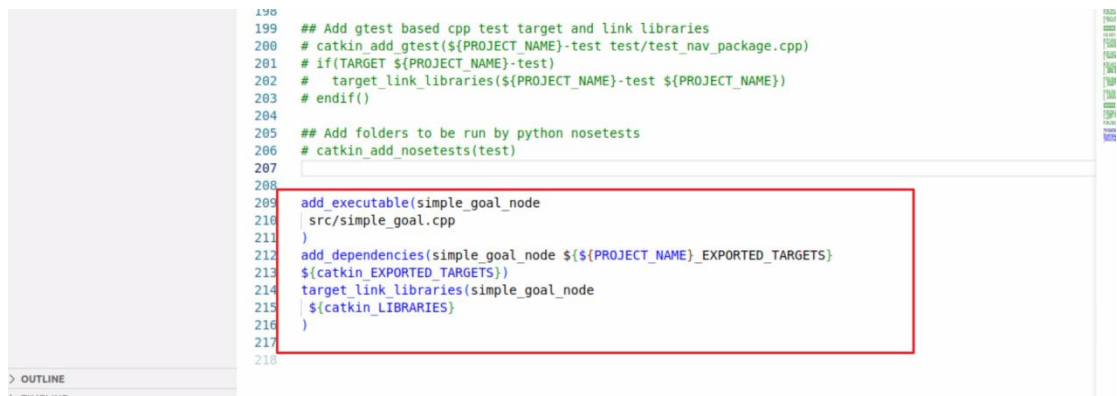
    return 0;
}
```

- 1) 代码的开始部分，先 include 了三个头文件，第一个是 ros 的系统头文件，第二个是导航目标结构体 move_base_msgs::MoveBaseGoal 的定义文件，第三个是 actionlib::SimpleActionClient 的定义文件；
- 2) ROS 节点的主体函数是 int main(int argc, char** argv)，其参数定义和其他 C++ 程序一样；
- 3) main 函数里，首先调用 ros::init(argc, argv, "simple_goal");进行该节点的初始化操作，函数的第三个参数是节点名称；

- 4) 接下来声明一个 `MoveBaseClient` 对象 `ac`，用来调用和主管监控导航功能的服务；
- 5) 在请求导航服务前，需要确认导航服务已经开启，所以这里调用 `ac.waitForServer()` 函数来查询导航服务的状态。`ros::Duration()`是睡眠函数，参数的单位为秒，表示睡眠一段时间，这段时间若被某个信号打断（在这个例程里，这个信号就是导航服务已经启动的信号），则中断睡眠。所以 `ac.waitForServer(ros::Duration(5.0));`的意思就是：休眠 5 秒，若期间导航服务启动了，则中断睡眠，开始后面的操作。用 `while` 循环来嵌套，可以让程序在休眠 5 秒后，若导航服务未启动，则继续进入下一个 5 秒睡眠，直到导航服务启动才中断睡眠；
- 6) 确认导航服务启动后，我们声明一个 `move_base_msgs::MoveBaseGoal` 类型结构体对象 `goal`，用来传递我们要导航去的目标信息。`goal.target_pose.header.frame_id` 表示这个目标位置的坐标是基于哪个坐标系，例程里赋值“`base_link`”表示这是一个基于机器人本体坐标系的导航目标位置。`goal.target_pose.header.stamp` 赋值当前时间戳，`goal.target_pose.pose.position.x` 赋值 1.0，表示本次导航的目的地是以机器人本体坐标系为基础，向 X 轴（机器人正前方）移动 1.0 米。`goal.target_pose.pose.position` 的 `y` 和 `z` 都未赋值，则默认是 0；`goal.target_pose.pose.orientation.w` 赋值 1.0，则导航的目标姿态是机器人面朝 X 轴的正方向（正前方）；
- 7) `ac.sendGoal(goal);`将导航目标信息传递给导航服务的客户端 `ac`，由 `ac` 来监控后面的导航过程；
- 8) `ac.waitForResult();`等待 `MoveBase` 的导航结果，这个函数会保持阻塞，就是卡在这，直到整个导航过程结束，或者导航过程被其他原因中断；
- 9) `ac.waitForResult()`阻塞结束后，调用 `ac.getState()`获取导航服务的结果，如果是“`SUCCEEDED`”说明导航顺利到达目的地；若不是这个结果，则说明导航服务由于某些原因被中断了。

（6）代码编写完毕，需要将文件名添加到编译文件里才能进行编译。编译文件在 `nav_package` 的目录下，文件名为“`CMakeLists.txt`”，在 IDE 界面左侧点击该文件，右侧会显示文件内容。在 `nav_package` 的目录下的“`CMakeLists.txt`”文件末尾，为 `simple_goal.cpp` 添加新的编译规则。内容如下：

```
add_executable(simple_goal_node
  src/simple_goal.cpp
)
add_dependencies(simple_goal_node ${${PROJECT_NAME}_EXPORTED_TARGETS}
  ${catkin_EXPORTED_TARGETS})
target_link_libraries(simple_goal_node
  ${catkin_LIBRARIES}
)
```



```
199 ## Add gtest based cpp test target and link libraries
200 # catkin_add_gtest(${PROJECT_NAME}-test test/test_nav_package.cpp)
201 # if(TARGET ${PROJECT_NAME}-test)
202 #   target_link_libraries(${PROJECT_NAME}-test ${PROJECT_NAME})
203 # endif()
204
205 ## Add folders to be run by python nosetests
206 # catkin_add_nosetests(test)
207
208
209 add_executable(simple_goal_node
210   src/simple_goal.cpp
211 )
212 add_dependencies(simple_goal_node ${${PROJECT_NAME}_EXPORTED_TARGETS}
213   ${catkin_EXPORTED_TARGETS})
214 target_link_libraries(simple_goal_node
215   ${catkin_LIBRARIES}
216 )
217
218
```

(7) 同样，修改完需要按下键盘快捷键 **Ctrl+S** 进行保存，代码上方的文件名右侧的小黑点会变成“X”，说明保存文件成功。下面开始进行代码文件的编译操作，启动一个终端程序，键入如下命令进入 ROS 的工作空间

```
cd /home/yhs/catkin_ws
```

```
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws
yhs@yhs-ros:~/catkin_ws$
```

(8) 然后再执行如下命令开始编译

```
catkin_make
```

```
yhs@yhs-ros:~/catkin_ws$ catkin_make
```

执行这条指令后，会出现滚动的编译信息，直到出现“Linking CXX executable /home/yhs/catkin_ws/devel/lib/nav_package/simple_goal_node”信息，说明新的 nav_pkg_node 节点已经编译成功。

```
[100%] Built target move_base_node
[100%] Linking CXX executable /home/yhs/catkin_ws/devel/lib/nav_package/simple_goal_node
[100%] Built target simple_goal_node
yhs@yhs-ros:~/catkin_ws$
```

2、运行 “simple_goal_node” 节点。

(1) 在启动我们的 `nav_pkg_node` 节点前，需要先启动煜禾森机器人的导航服务，打开终端，输入以下指令后敲回车，看到 “odom received!” 则表示启动导航成功

```
roslaunch yhs_nav yhs_nav.launch
```

```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav.launch
... logging to /home/yhs/.ros/log/a995d530-22e8-11ef-92e5-0b8e29a6dc24/roslaunch-yhs-ros-5525.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:40445/

[ INFO] [1717553136.750830953]: Footprint model 'circular' (radius: 0.2m) loaded for trajectory optimization.
[ INFO] [1717553136.853175943]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.858632746]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.877216340]: odom received!
```

(2) 导航启动成功后，按照上面章节中的方式初始化小车。

(3) 初始化后，打开另外一个终端，输入指令后敲回车

```
roslaunch nav_package simple_goal_node
```

```
yhs@yhs-ros:~$ roslaunch nav_package simple_goal_node
[ INFO] [1717565617.096658934]: Sending goal
[ INFO] [1717565622.252568777]: arrive
yhs@yhs-ros:~$
```

(4) 将遥控器切换到指令模式后，机器人往前导航走 1 米左右

(5) 尝试在代码里给 `pose.pose.position` 的 `x` 和 `y` 赋一些不同的数值，观察机器人的导航行为。

(6) 尝试将 `pose.header.frame_id` 改成其他坐标系（比如 “map” ），观察机器人的导航目的地是否发生变化。

实验十一 全局路径规划

自动驾驶中，全局路径规划算法一般采用 A* 或者 Dijkstra 算法，具体的算法原理及实现可以参考（<https://www.redblobgames.com/pathfinding/a-star/introduction.html>），我们来看下在导航中怎么使用这两种算法。

1、查看导航配置文件

（1）打开终端，进入 “/home/yhs/catkin_ws/src/yhs_nav/param” 目录下，可以看到 “yhs_nav_param.yaml” 文件，所有导航相关的参数都在这里设置修改

```
cd /home/yhs/catkin_ws/src/yhs_nav/param
```

```
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/param$ ls
yhs_nav_param.yaml
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/param$
```

（2）打开该文件，找全局路径规划相关的内容

```
176 # move_base参数
177 move_base:
178   base_global_planner: navfn/NavfnROS
179   base_local_planner: teb_local_planner/TebLocalPlannerROS
180   planner_frequency: 5
181   planner_patience: 5
182
183   controller_frequency: 20
184   controller_patience: 5
185   max_planning_retries: 5
186   oscillation_timeout: 0.0
187   oscillation_distance: 5
188
189 #全局路径规划参数
190 NavfnROS:
191   allow_unknown: false
192   use_astar: true
193
```

（3）可能要修改调试的参数

use_astar	规划算法选择，设置为 true 使用 A*，设置为 false 使用 Dijkstra
allow_unknown	是否能在在地图未知区域规划路径，默认不能

2、查看全局规划路径

（1）使用默认的全局规划参数，启动导航后，进行机器人位置初始化

```
roslaunch yhs_nav yhs_nav.launch
```

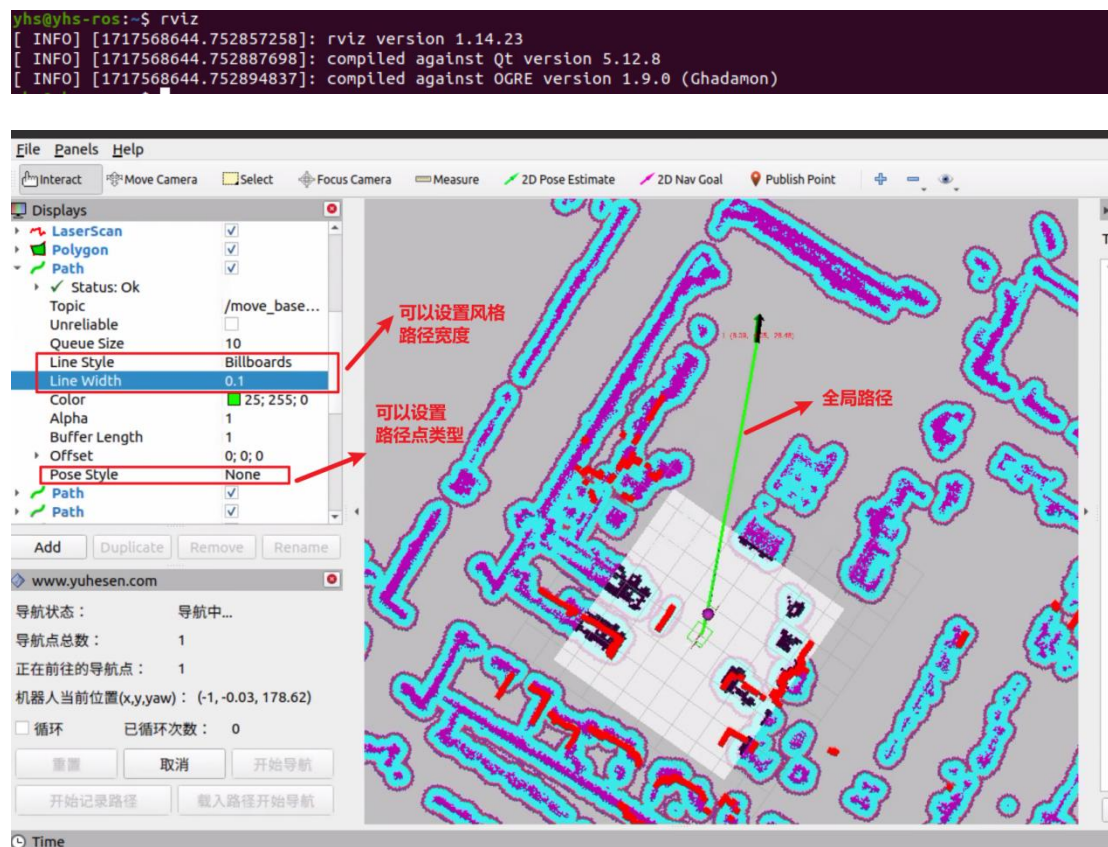
```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav.launch
... logging to /home/yhs/.ros/log/a995d530-22e8-11ef-92e5-0b8e29a6dc24/roslaunch-yhs-ros-5525.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:40445/

[ INFO] [1717553136.750830953]: Footprint model 'circular' (radius: 0.2m) loaded for trajectory optimization.
[ INFO] [1717553136.853175943]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.858632746]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.877216340]: odom received!
```

(2) 打开 Rviz, 下发导航点, 查看全局路径, 路径的大小颜色都是可以设置的

rviz



实验十二 局部路径规划

上一节实验中，我们选择一种全局路径规划算法，并且下发导航点，规划出了全局路径，然后调用局部路径规划器根据这条路线及 `costmap` 的信息规划出机器人在局部时做出具体行动策略。常用的局部路径规划算法有动态窗口法(DWA)和时间弹性带(TEB)算法。

动态窗口法(DWA): DWA 算法全称为 `dynamic window approach`，其原理主要是在速度空间 (v, w) 中采样多组速度，并模拟出这些速度在一定时间内的运动轨迹，并通过评价函数对这些轨迹进行评价，选取最优轨迹对应的速度驱动机器人运动。

时间弹性带(TEB): 全称为 `Time Elastic Band`，TEB 算法是一种非线性最优化算法，它首先利用全局路径作为初始轨迹得到局部范围内离散的一系列机器人位姿(`pose`)，以及这些位姿对应的时间差(`timediff`)变量。轨迹上的位姿和时间差需要满足速度约束、加速度约束、机器人运动学约束和障碍物之间的距离约束等约束，这些约束可以建模为非线性最优化问题，从而得到优化后的轨迹，通过位姿和时间差可以推算出速度，进而下发给机器人进行控制。

优缺点对比：

	优点	缺点
DWA	计算简单	前瞻性不足
	适用于差分和全向车模	动态效果差
		不适用于阿克曼模型车模
TEB	适用于各种常见车模	计算复杂度较大
	对前方一段轨迹进行优化	速度和角度波动较大、控制不稳定
	对动态障碍有较好的避障效果	非全局最优

1、查看导航配置文件

(1) 打开终端，进入 “/home/yhs/catkin_ws/src/yhs_nav/param” 目录下，可以看到 “yhs_nav_param.yaml” 文件，所有导航相关的参数都在这里设置修改

```
cd /home/yhs/catkin_ws/src/yhs_nav/param
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws/src/yhs_nav/param
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/param$ ls
yhs_nav_param.yaml
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/param$
```

(2) 打开该文件，找局部路径规划相关的内容

```
330 #TEB规划参数
331 TebLocalPlannerROS:
332   odom_topic: /odom
333
334   # Custom
335   desired_linear_vel : 0.5
336   back_dist : -0.08
337   use_rotate_to_heading : true
338   goal_dist_tol : 0.1
339   goal_yaw_tol: 0.15
340   goal_dist_tol_error : 0.3
341   rotate_to_heading_angular_vel : 0.3
342   min_approach_linear_velocity : 0.2
343   approach_velocity_scaling_dist : 1.5
344   max_speed_limit : 1.0
345   max_angular_vel : 0.5
346   self_rotate_angular : 0.75
347
348   # Trajectory
349   teb_autosize: True
350   dt_ref: 0.3 #0.45
351   dt_hysteresis: 0.1
352   max_samples: 500
353   global_plan_overwrite_orientation: True
354   allow_init_with_backwards_motion: False
355   max_global_plan_lookahead_dist: 6.0
356   global_plan_viapoint_sep: -0.1 #0.3
357   global_plan_prune_distance: 1
358   exact_arc_length: False
359   feasibility_check_no_poses: 1
360   publish_feedback: False
361   control_look_ahead_poses: 1
362
363   . . .
```

(3) 可能要修改调试的参数（参数已调到较优，这里只是作调试说明）

dt_ref	期望的轨迹时间分辨率
dt_hysteresis	根据当前时间分辨率自动调整大小的滞后现象，通常约为。建议使用 dt_ref 的 10%
max_vel_x	机器人最大线速度
max_vel_x_backwards	机器人最大后退线速度
acc_lim_x	机器人线加速度
max_vel_theta	机器人角速度
acc_lim_theta	机器人角加速度
weight_kinematics_forward_drive	强制机器人仅选择正向（正的平移速度）的优化权重
weight_viapoint	跟踪全据路径的权重
min_obstacle_dist	与障碍物的最小期望距离

2、查看调试局部规划路径

(1) 使用默认的局部规划参数，启动导航后，进行机器人位置初始化

```
roslaunch yhs_nav yhs_nav.launch
```

```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav.launch
... logging to /home/yhs/.ros/log/a995d530-22e8-11ef-92e5-0b8e29a6dc24/roslaunch-yhs-ros-5525.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

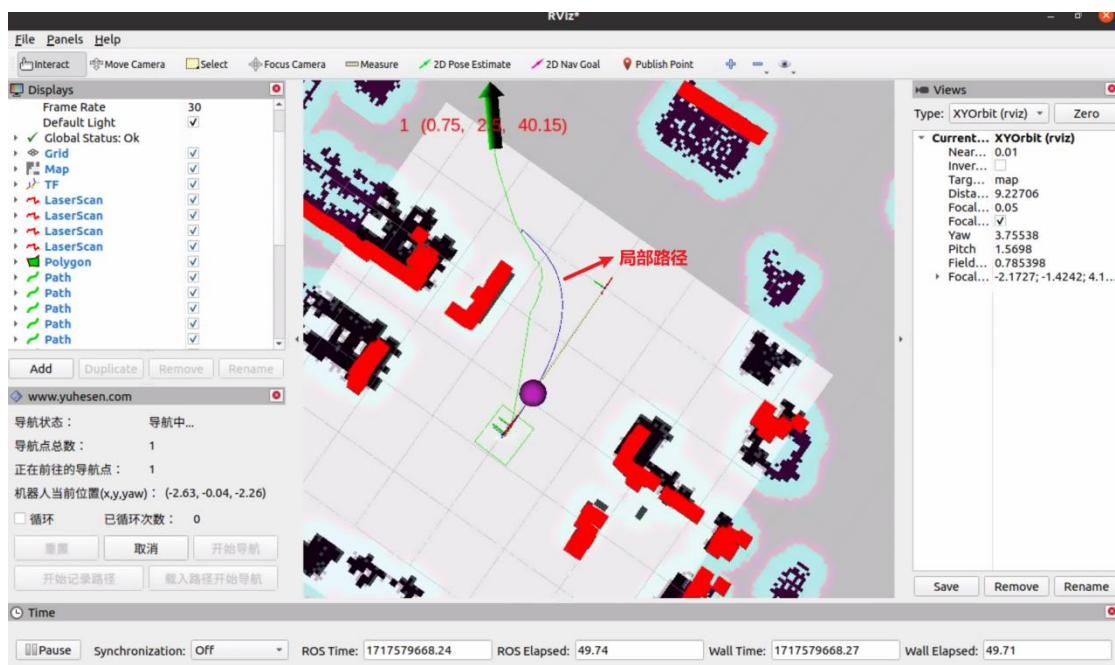
started roslaunch server http://192.168.1.102:40445/
```

```
[ INFO] [1717553136.750830953]: Footprint model 'circular' (radius: 0.2m) loaded for trajectory optimization.
[ INFO] [1717553136.853175943]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.858632746]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.877216340]: odon received!
```

(2) 打开 Rviz，下发导航点，查看全局路径，路径的大小颜色都是可以设置的

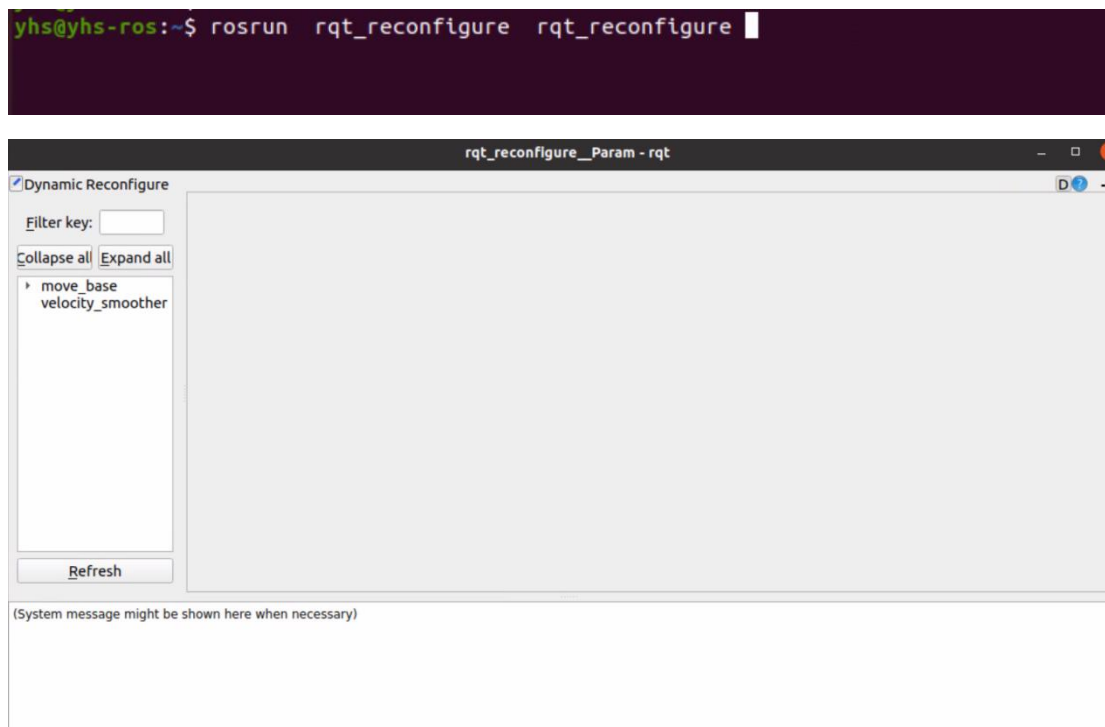
```
rviz
```

```
yhs@yhs-ros:~$ rviz
[ INFO] [1717568644.752857258]: rviz version 1.14.23
[ INFO] [1717568644.752887698]: compiled against Qt version 5.12.8
[ INFO] [1717568644.752894837]: compiled against OGRE version 1.9.0 (Chadamon)
```

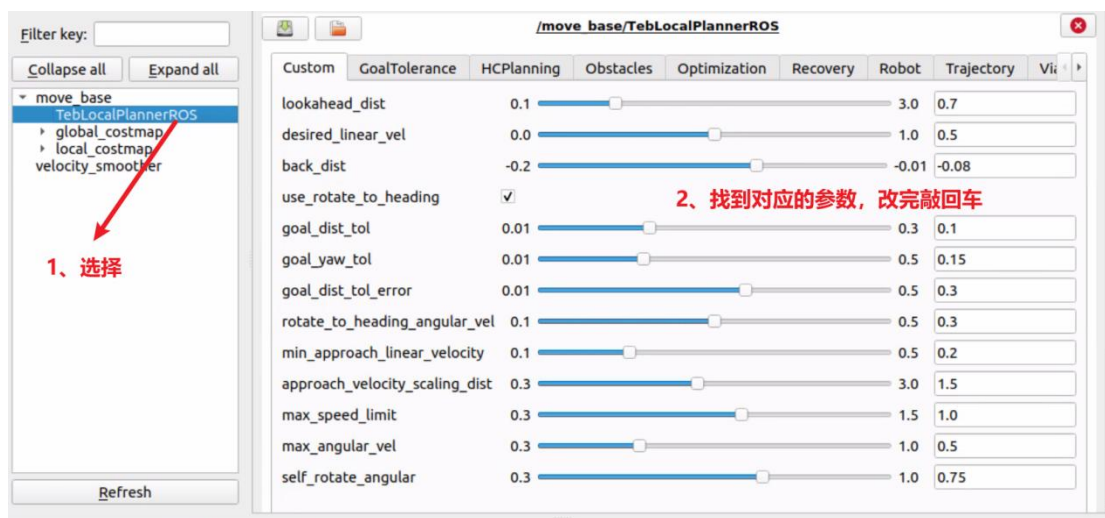


(3) 在导航过程中, 可以使用“rqt_reconfigure”动态调节局部规划参数, 便于观察, 不用每次修改参数重启导航, 提高效率, 打开另外一个终端, 输入下面指令敲回车后弹出“rqt_reconfigure”界面

```
roslaunch rqt_reconfigure rqt_reconfigure
```

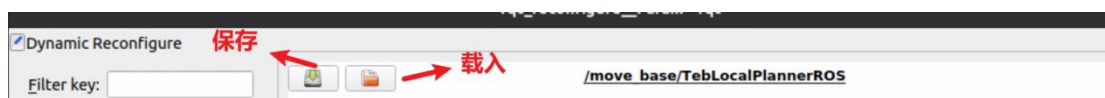


(4) 按照下图中的步骤进行调试



(5) 改完参数后, 可以直接在 Rviz 界面看规划效果

(6) 参数保存和加载, 右侧参数界面左上角可以加载和保存参数



实验十三 RGB-D 相机

煜禾森 ROS 机器人的车头装备一台视觉传感器，这是一款典型的 RGB-D 相机。RGB-D 的全称是“RGB-Depth”，其中“RGB”是“红绿蓝”的意思，表示的是彩色图像；“Depth”是深度的意思，表示的是深度（距离）图像。所以 RGB-D 相机是一种即能感知物体颜色，又能探测到物体距离的复合型传感器。在获取的数据形式上，就是一幅彩色图像，和一个包含距离信息的点阵也就是三维点云。在这节实验中，我们将会大致了解这个相机的组成原理以及采集到的数据形态。

首先我们看看深度相机的外观组成，可以看到其主体是一个长方体外形。主体的正面面板排布着几个主要的传感器：



在这一节实验中，我们主要使用彩色相机。

1、启动相机

(1) 打开终端，输入指令敲回车，启动相机节点

单相机: `roslaunch ascamera_listener hp60c.launch`

双相机: `roslaunch ascamera_listener hp60c_multiple.launch`

```
yhs@yhs-ros:~$ roslaunch ascamera_listener hp60c_multiple.launch
... logging to /home/yhs/.ros/log/8767c1bc-2315-11ef-b239-7b54079e3601/roslaunch-yhs-ros-7888.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:44977/
```

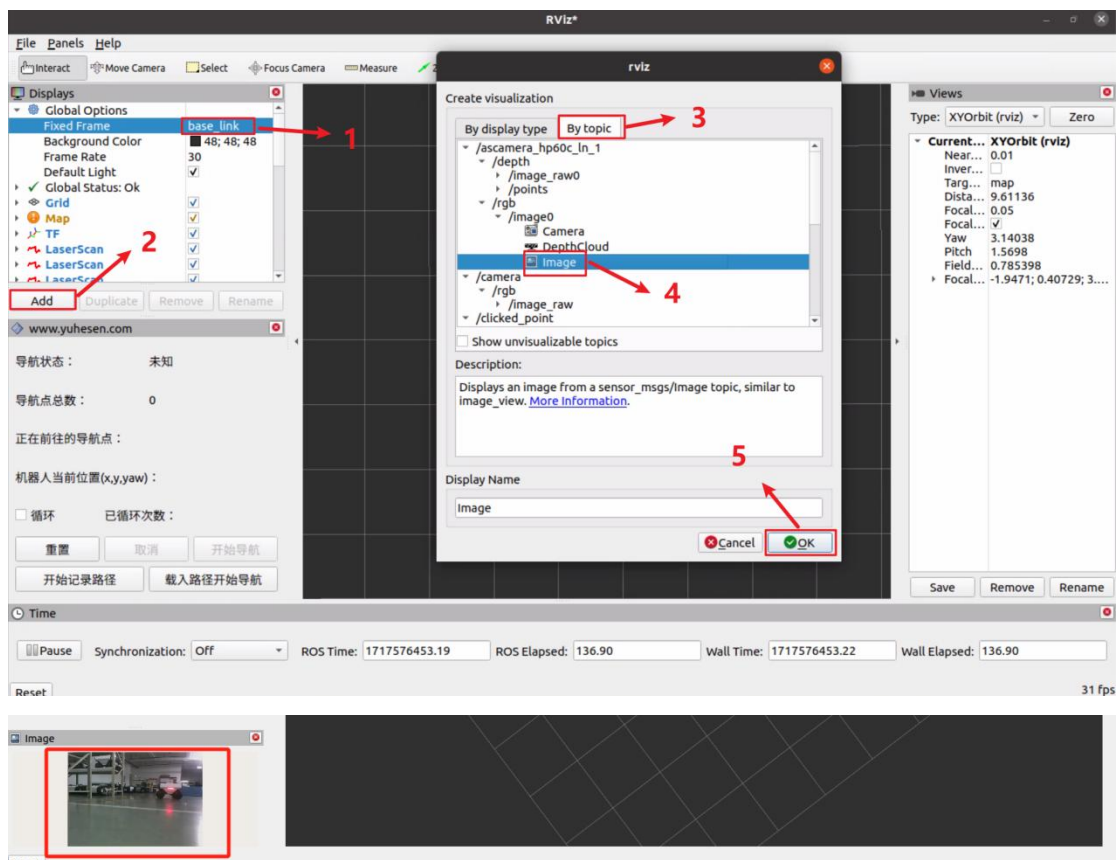
```
2024-06-05 16:31:52[INFO] [Camera.cpp] [120] [backgroundThread] SN [ ASC60CE03000061 ]'s parameter:
2024-06-05 16:31:52[INFO] [Camera.cpp] [121] [backgroundThread] trfx: 443.799
2024-06-05 16:31:52[INFO] [Camera.cpp] [122] [backgroundThread] trfy: 443.505
2024-06-05 16:31:52[INFO] [Camera.cpp] [123] [backgroundThread] trcx: 315.656
2024-06-05 16:31:52[INFO] [Camera.cpp] [124] [backgroundThread] trcy: 236.051
2024-06-05 16:31:52[INFO] [Camera.cpp] [125] [backgroundThread] rgbfx: 577.274
2024-06-05 16:31:52[INFO] [Camera.cpp] [126] [backgroundThread] rgbfy: 576.765
2024-06-05 16:31:52[INFO] [Camera.cpp] [127] [backgroundThread] rgbcx: 338.312
2024-06-05 16:31:52[INFO] [Camera.cpp] [128] [backgroundThread] rgbcy: 242.838
```

(2) 打开另外一个终端，输入指令打开 Rviz

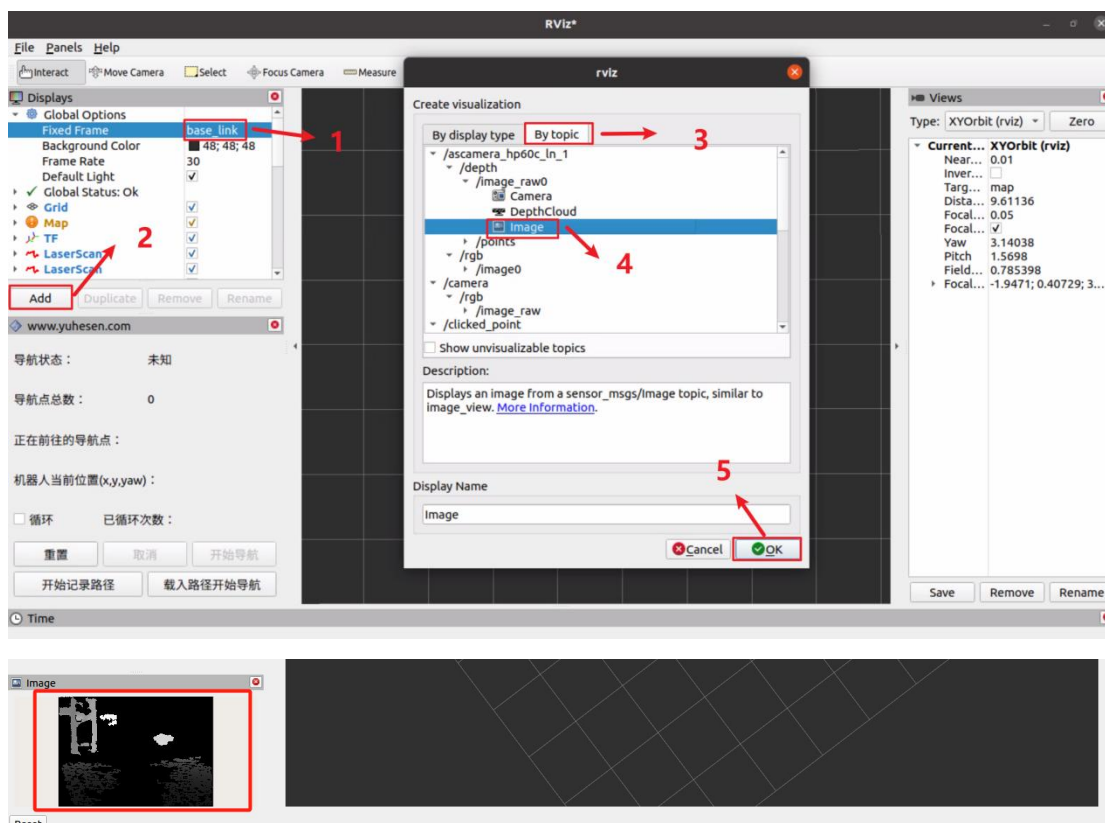
`rviz`

```
yhs@yhs-ros:~$ rviz
```

(3) 按照以下方式在 Rviz 上增加彩色图像显示



(4) 按照以下方式在 rviz2 上增加深度图像显示



实验十四 RGB-D 相机的三维点云

上一节实验我们已经获取相机拍摄到的彩色图像，这一节实验我们将学习如何在代码中获取 RGB-D 相机拍摄到的三维点云。

1、创建 ROS 源码包

(1) 打开一个终端程序，输入指令进入 “cd /home/yhs/catkin_ws/src/yhs_tutorials” 目录

```
cd /home/yhs/catkin_ws/src/yhs_tutorials/
```

```
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws/src/yhs_tutorials/
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$
```

(2) 使用如下指令创建一个新的 ROS 软件包 (Package)

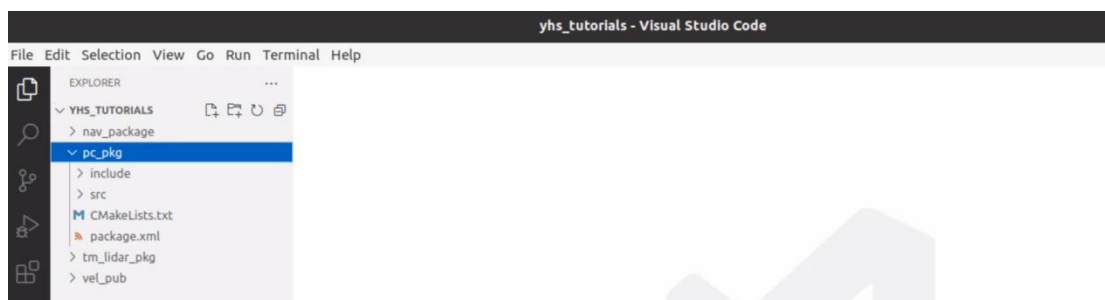
```
catkin_create_pkg pc_pkg roscpp std_msgs sensor_msgs pcl_ros
```

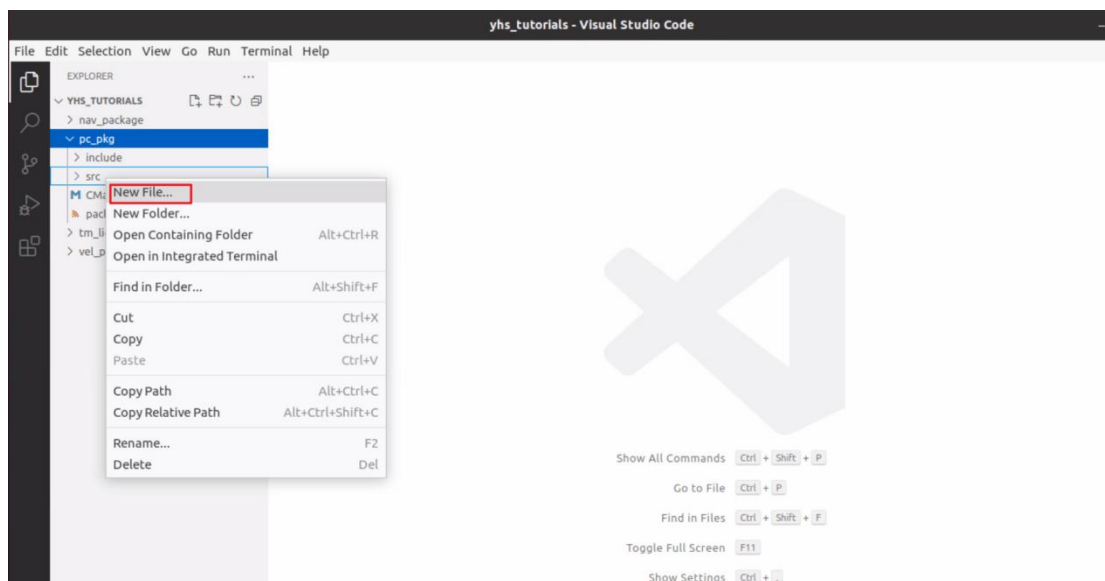
```
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$ catkin_create_pkg pc_pkg roscpp std_msgs sensor_msgs pcl_ros
Created file pc_pkg/package.xml
Created file pc_pkg/CMakeLists.txt
Created folder pc_pkg/include/pc_pkg
Created folder pc_pkg/src
Successfully created files in /home/yhs/catkin_ws/src/yhs_tutorials/pc_pkg. Please adjust the values in package.xml.
yhs@yhs-ros:~/catkin_ws/src/yhs_tutorials$
```

这条指令的具体含义是：

指令	含义
catkin_create_pkg	创建 ROS 源码包 (package) 的指令
pc_pkg	新建的 ROS 源码包命名
roscpp	C++依赖项，本例程使用 C++编写，所以需要这个依赖项
std_msgs	标准消息依赖项，需要里面的 String 格式做文字输出
sensor_msgs	传感器消息依赖项，需要里面的图像数据格式
pcl_ros	ROS 里的开源点云库 PCL 的依赖项

(3) 创建 “pc_pkg” 包后，用 IDE 打开 “yhs_tutorials”，可以看到 “yhs_tutorials” 目录下多了一个 “pc_pkg” 文件夹，在其 “src” 子文件夹上右键点击鼠标，选择 “New File” 新建一个代码文件。（这里我们用的是 visual studio code，也可以使用 vim、Text Editor 等其他文本编辑工具打开或创建）

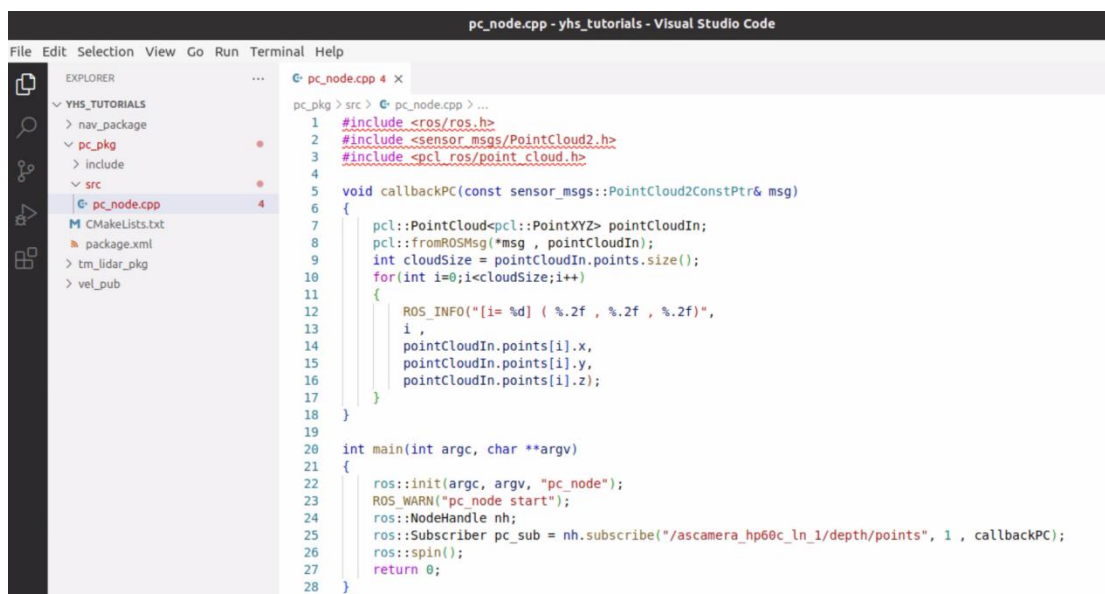




(4) 新建的代码文件，我们为其命名为“pc_node.cpp”



(5) 命名完毕后，在 IDE 的右侧可以开始编写“pc_node.cpp”的代码，其内容如下：



```
#include <ros/ros.h>
#include <sensor_msgs/PointCloud2.h>
#include <pcl_ros/point_cloud.h>

void callbackPC(const sensor_msgs::PointCloud2ConstPtr& msg)
{
    pcl::PointCloud<pcl::PointXYZ> pointCloudIn;
    pcl::fromROSMsg(*msg, pointCloudIn);
    int cloudSize = pointCloudIn.points.size();
    for(int i=0;i<cloudSize;i++)
    {
        ROS_INFO("[i= %d] ( %.2f , %.2f , %.2f)",
            i,
            pointCloudIn.points[i].x,
            pointCloudIn.points[i].y,
            pointCloudIn.points[i].z);
    }
}

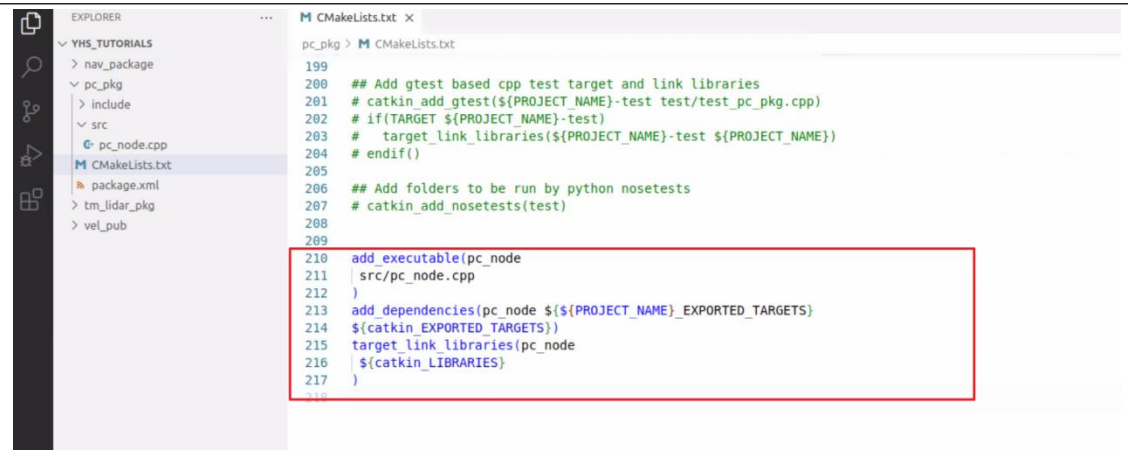
int main(int argc, char **argv)
{
    ros::init(argc, argv, "pc_node");
    ROS_WARN("pc_node start");
    ros::NodeHandle nh;
    ros::Subscriber pc_sub = nh.subscribe("/ascamera_hp60c_ln_1/depth/points", 1,
callbackPC);
    ros::spin();
    return 0;
}
```

- 1) 代码的开头 `include` 了三个头文件：`ros.h` 是 ROS 系统头文件；`sensor_msgs/PointCloud2.h` 是 ROS 里的点云类型头文件；`pcl_ros/point_cloud.h` 是 PCL 里的点云类型头文件；
- 2) 定义一个回调函数 `void callbackPC()`，用来处理三维点云数据。ROS 每接收到一帧深度图像，就会转换成三维点云，自动调用一次这个回调函数。三维点云数据会以参数的形式传递到这个回调函数里；
- 3) 回调函数 `void callbackPC()`的参数 `msg` 是一个 `sensor_msgs::PointCloud2` 格式指针，其指向的内存区域就是存放三维点云的内存空间。在实际开发中，通常不会直接使用这个格式的点云，而是将其转换成 PCL 的点云格式，这样就可以使用丰富的 PCL 函数来处理点云数据；

- 4) 在回调函数 `void callbackPC()` 中, 定义一个 `pcl::PointXYZ` 格式的点云容器 `pointCloudIn`, 调用 `pcl::fromROSMsg()` 函数将参数里的 ROS 格式点云数据转换成 PCL 格式的点云数据, 存放在容器 `pointCloudIn` 里;
- 5) 获取转换后的点云数组 `pointCloudIn.points` 的三维点数量, 存储在一个变量 `cloudSize` 里。使用一个 `for` 循环, 把的 `pointCloudIn.points` 里所有点的 `x`、`y` 和 `z` 三个值, 通过 `ROS_INFO()` 显示在终端程序里。通常来说, `pointCloudIn.points` 里的原始坐标值不会直接拿来使用, 而是需要转换到机器人坐标系之后, 再用 PCL 点云库的函数来进行处理;
- 6) 在主函数 `main()` 中, 调用 `ros::init()`, 对这个节点进行初始化;
- 7) 调用 `ROS_WARN()` 向终端程序输出字符串信息, 以表明节点正常启动了;
- 8) 定义一个 `ros::NodeHandle` 节点句柄 `nh`, 并使用这个句柄向 ROS 核心节点订阅 Topic 主题 “`/ascamera_hp60c_in_1/depth/points`” 的数据, 回调函数设置为之前定义的 `callbackPC()`。这个 “`/ascamera_hp60c_in_1/depth/points`” 是相机的 ROS 节点发布三维点云的主题名, 相机采集到的三维点云会以 ROS 图像数据包格式发送到这个主题里;
- 9) 调用 `ros::spin()` 对 `main()` 函数进行阻塞, 保持这个节点程序不会结束退出。

(6) 代码编写完毕, 需要将文件名添加到编译文件里才能进行编译。编译文件在 `pc_pkg` 的目录下, 文件名为 “`CMakeLists.txt`”, 在 IDE 界面左侧点击该文件, 右侧会显示文件内容。在 “`CMakeLists.txt`” 文件末尾, 为 `pc_node.cpp` 添加新的编译规则。内容如下:

```
add_executable(pc_node
  src/pc_node.cpp
)
add_dependencies(pc_node ${PROJECT_NAME}_EXPORTED_TARGETS
${catkin_EXPORTED_TARGETS})
target_link_libraries(pc_node
  ${catkin_LIBRARIES}
)
```



```
199
200 ## Add gtest based cpp test target and link libraries
201 # catkin_add_gtest(${PROJECT_NAME}-test test/test_pc_pkg.cpp)
202 # if(TARGET ${PROJECT_NAME}-test)
203 #   target_link_libraries(${PROJECT_NAME}-test ${PROJECT_NAME})
204 # endif()
205
206 ## Add folders to be run by python nosetests
207 # catkin_add_nosetests(test)
208
209
210 add_executable(pc_node
211   src/pc_node.cpp
212 )
213 add_dependencies(pc_node ${PROJECT_NAME}_EXPORTED_TARGETS
214 ${catkin_EXPORTED_TARGETS})
215 target_link_libraries(pc_node
216   ${catkin_LIBRARIES}
217 )
```

(7) 同样, 修改完需要按下键盘快捷键 **Ctrl+S** 进行保存, 代码上方的文件名右侧的小黑点会变成“X”, 说明保存文件成功。下面开始进行代码文件的编译操作, 启动一个终端程序, 键入如下命令进入 ROS 的工作空间

```
cd /home/yhs/catkin_ws
```

```
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws
yhs@yhs-ros:~/catkin_ws$
```

(8) 然后再执行如下命令开始编译

```
catkin_make
```

```
yhs@yhs-ros:~/catkin_ws$ catkin_make
Base path: /home/yhs/catkin_ws
Source space: /home/yhs/catkin_ws/src
Build space: /home/yhs/catkin_ws/build
Devel space: /home/yhs/catkin_ws/devel
Install space: /home/yhs/catkin_ws/install
```

执行这条指令后, 会出现滚动的编译信息, 直到出现 “Linking CXX executable /home/yhs/catkin_ws/devel/lib/pc_pkg/pc_node” 信息, 说明新的 pc_node 节点已经编译成功

```
[ 98%] Built target test_optim_node
[100%] Built target move_base
[100%] Built target move_base_node
[100%] Linking CXX executable /home/yhs/catkin_ws/devel/lib/pc_pkg/pc_node
[100%] Built target pc_node
yhs@yhs-ros:~/catkin_ws$
```

2、运行 “pc_node” 节点

(1) 先运行相机节点, 打开终端, 输入指令敲回车, 启动相机节点

```
单相机: roslaunch ascamera_listener hp60c.launch
```

```
双相机: roslaunch ascamera_listener hp60c_multiple.launch
```

```
yhs@yhs-ros:~$ roslaunch ascamera_listener hp60c_multiple.launch
... logging to /home/yhs/.ros/log/8767c1bc-2315-11ef-b239-7b54079e3601/roslaunch-yhs-ros-7888.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:44977/
```

```
2024-06-05 16:31:52[INFO] [Camera.cpp] [120] [backgroundThread] SN [ ASC60CE03000061 ]'s parameter:
2024-06-05 16:31:52[INFO] [Camera.cpp] [121] [backgroundThread] trfx: 443.799
2024-06-05 16:31:52[INFO] [Camera.cpp] [122] [backgroundThread] trfy: 443.505
2024-06-05 16:31:52[INFO] [Camera.cpp] [123] [backgroundThread] trcx: 315.656
2024-06-05 16:31:52[INFO] [Camera.cpp] [124] [backgroundThread] trcy: 236.051
2024-06-05 16:31:52[INFO] [Camera.cpp] [125] [backgroundThread] rgbfx: 577.274
2024-06-05 16:31:52[INFO] [Camera.cpp] [126] [backgroundThread] rgbfy: 576.765
2024-06-05 16:31:52[INFO] [Camera.cpp] [127] [backgroundThread] rgbcx: 338.312
2024-06-05 16:31:52[INFO] [Camera.cpp] [128] [backgroundThread] rgbcy: 242.838
```

(2) 此时我们可以运行刚才编写的 pc_node, 打开另外一个终端, 输入指令, 敲回车

```
roslaunch pc_pkg pc_node
```

```
yhs@yhs-ros:~$ roslaunch pc_pkg pc_node
```

这条指令会启动我们编写的 `pc_node`。按照程序逻辑，会从相机的三维点云主题“`/ascamera_hp60c_1/depth/points`”主题里不断获取点云数据包，并把 ROS 格式的三维点云转换成 PCL 格式，然后把所有点的 `x`、`y`、`z` 坐标值显示在终端上

```
[ INFO] [1717578045.780385580]: [i= 29248] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780395730]: [i= 29249] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780405716]: [i= 29250] ( 1.63 , -0.20 , -0.38)
[ INFO] [1717578045.780413196]: [i= 29251] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780420583]: [i= 29252] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780430110]: [i= 29253] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780439715]: [i= 29254] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780448593]: [i= 29255] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780457360]: [i= 29256] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780466495]: [i= 29257] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780475269]: [i= 29258] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780486373]: [i= 29259] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780496261]: [i= 29260] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780504464]: [i= 29261] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780511935]: [i= 29262] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780519126]: [i= 29263] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780526425]: [i= 29264] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780538233]: [i= 29265] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780545902]: [i= 29266] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780552591]: [i= 29267] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780560459]: [i= 29268] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780567746]: [i= 29269] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780574853]: [i= 29270] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780583644]: [i= 29271] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780597804]: [i= 29272] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780606325]: [i= 29273] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780614923]: [i= 29274] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780624527]: [i= 29275] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780631819]: [i= 29276] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780640765]: [i= 29277] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780650163]: [i= 29278] ( 0.00 , 0.00 , 0.00)
[ INFO] [1717578045.780659073]: [i= 29279] ( 0.00 , 0.00 , 0.00)
```

实验十五 障碍图层加入相机 LaserScan 数据

机器人在导航过程中，会利用 3D 激光数据进行动态的全局和局部路径规划，从而绕开路径上的障碍物。但是 3D 激光的扫描范围是有盲区的，较低的地方，激光是扫描不到的，所以我们需要把深度相机的点云数据，也添加到导航障碍图层中。

1、点云数据转换为 LaserScan 数据

(1) 将点云数据转换为 LaserScan 数据，用到“pointcloud_to_laserscan”功能包，我们来看一下运行的“hp60c.launch”文件，存放于“/home/yhs/catkin_ws/src/hardware/ascamera_listener/launch”目录下

```
yhs@yhs-ros:~/catkin_ws/src/hardware/ascamera_listener/launch$ ls
hp60c.launch  hp60c_multiple.launch  pointcloud_to_laserscan.launch
yhs@yhs-ros:~/catkin_ws/src/hardware/ascamera_listener/launch$
```

文件中调用了“pointcloud_to_laserscan.launch”

```
41 <!-- 将相机点云转化为scan -->
42 <include file="$(find ascamera_listener)/launch/pointcloud_to_laserscan.launch">
43   <arg name="double_camera" value="$(arg double_camera)" />
44
```

“pointcloud_to_laserscan.launch”其中我们可能要修改的内容如下：

/ascamera_hp60c_1/depth/points	订阅的点云话题，相机发布的点云话题就是这个，如果是别的话题，更改此项即可
scan1	点云转化为 LaserScan 后发布的话题，因为 3D 激光的话题为“scan”，所以不能与激光的话题名相同，这里改为“scan1”
min_height	截取点云数据的最低高度，最低高度过小，可能会将地面的点云数据转化为 LaserScan 数据，影响路径规划，可以反复修改查看，找到合适的值
max_height	截取点云数据的最高高度，最高高度过大，可能会影响路径规划，可以反复修改查看，找到合适的值

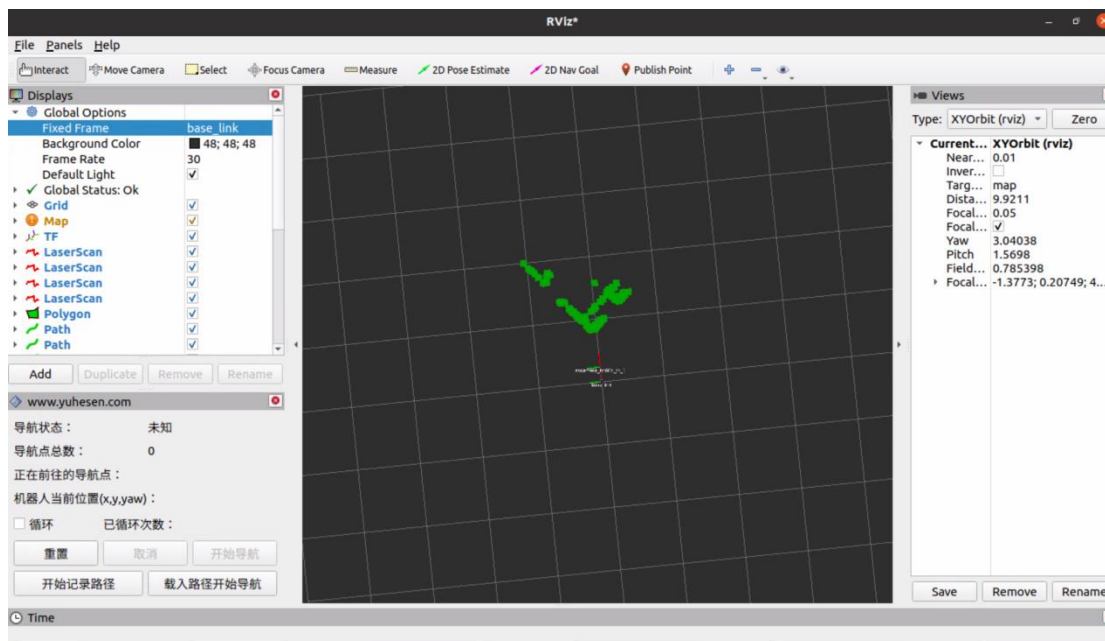
(2) 打开一个终端，输入命令，启动相机节点

单相机: <code>ros2 launch ascamera_listener hp60c.launch.py</code>
双相机: <code>ros2 launch ascamera_listener hp60c_multiple.launch.py</code>

```
yhs@yhs-ros:~$ roslaunch ascamera_listener hp60c_multiple.launch
... logging to /home/yhs/.ros/log/8767c1bc-2315-11ef-b239-7b54079e3601/roslaunch-yhs-ros-7888.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:44977/
```

(3) 在 Rviz 上查看将深度相机点云转化为 LaserScan 的数据，我们将“scan1”颜色设置成绿色

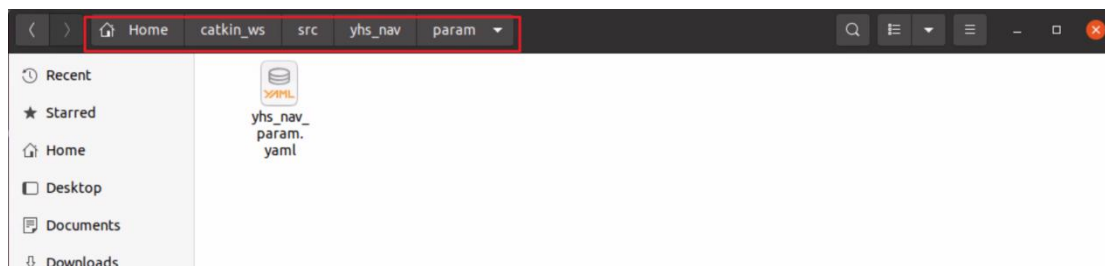


2、将“scan1”话题数据添加到障碍图层

(1) 进入“/home/yhs/catkin_ws/src/yhs_nav/param”目录，可以看到“yhs_nav_param.yaml”文件

```
cd /home/yhs/catkin_ws/src/yhs_nav/param
```

```
yhs@yhs-ros:~$ cd /home/yhs/catkin_ws/src/yhs_nav/param/
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/param$ ls
yhs_nav_param.yaml
yhs@yhs-ros:~/catkin_ws/src/yhs_nav/param$
```



(2) 打开“yhs_nav_param.yaml”文件，按照下图的方式给全局代价地图障碍物图层添加“scan1”数据

```

194 #全局代价地图参数
195 global_costmap:
196   robot_base_frame: base_link
197   global_frame: map
198   update_frequency: 1.0
199   publish_frequency: 2
200   static_map: true
201   transform_tolerance: 0.5
202
203 plugins:
204   - { name: static_layer, type: "costmap_2d::StaticLayer" }
205   - { name: obstacle_layer, type: "costmap_2d::ObstacleLayer" }
206   - { name: inflation_layer, type: "costmap_2d::InflationLayer" }
207
208 footprint: [[0.33,0.28],[0.33,-0.28],[-0.33,-0.28],[-0.33,0.28]]
209
210 static_layer:
211   enabled: true
212   map_topic: "/map"
213
214 obstacle_layer:
215   enabled: true
216   obstacle_range: 3.0
217   raytrace_range: 8.0
218   inflation_radius: 0.2
219   track_unknown_space: true
220   combination_method: 1
221
222 observation_sources: laser_scan_sensor laser_scan_sensor_1 laser_scan_sensor_2 laser_scan_sensor_3
223 laser_scan_sensor:
224   {
225     data_type: LaserScan,
226     topic: scan,
227     marking: true,
228     inf_is_valid: false,
229     clearing: true,
230   }
231 laser_scan_sensor_1:
232   {
233     data_type: LaserScan,
234     topic: scan1,
235     marking: true,
236     inf_is_valid: false,
237     clearing: true,
238   }
239 laser_scan_sensor_2:
240   {
241     data_type: LaserScan,
242     topic: scan2,
243     marking: true,
244     inf_is_valid: false,
245     clearing: true,

```

(3) 打开“yhs_nav_param.yaml”按照下图的方式给局部代价地图障碍物图层添加“scan1”数据

```

261 #局部代价地图参数
262 local_costmap:
263   global_frame: map
264   robot_base_frame: base_link
265   update_frequency: 5.0
266   publish_frequency: 5.0
267   static_map: true
268   rolling_window: true
269   width: 6.6
270   height: 6.6
271   resolution: 0.1
272   transform_tolerance: 0.5
273
274 plugins:
275   - { name: static_layer, type: "costmap_2d::StaticLayer" }
276   - { name: obstacle_layer, type: "costmap_2d::ObstacleLayer" }
277
278 footprint: [[0.33,0.28],[0.33,-0.28],[-0.33,-0.28],[-0.33,0.28]]
279
280 static_layer:
281   enabled: true
282   map_topic: "/map"
283
284 obstacle_layer:
285   enabled: true
286   obstacle_range: 3.0
287   raytrace_range: 8.0
288   inflation_radius: 0.2
289   track_unknown_space: true
290   combination_method: 1
291
292 observation_sources: laser_scan_sensor laser_scan_sensor_1 laser_scan_sensor_2 laser_scan_sensor_3
293 laser_scan_sensor:
294   {
295     data_type: LaserScan,
296     topic: scan,
297     marking: true,
298     inf_is_valid: false,
299     clearing: true,
300   }
301 laser_scan_sensor_1:
302   {
303     data_type: LaserScan,
304     topic: scan1,
305     marking: true,
306     inf_is_valid: false,
307     clearing: true,
308   }
309 laser_scan_sensor_2:
310   {
311     data_type: LaserScan,

```

(4) 打开终端，输入以下指令后敲回车，看到“odom received!”则表示启动导航成功

```
roslaunch yhs_nav yhs_nav.launch
```

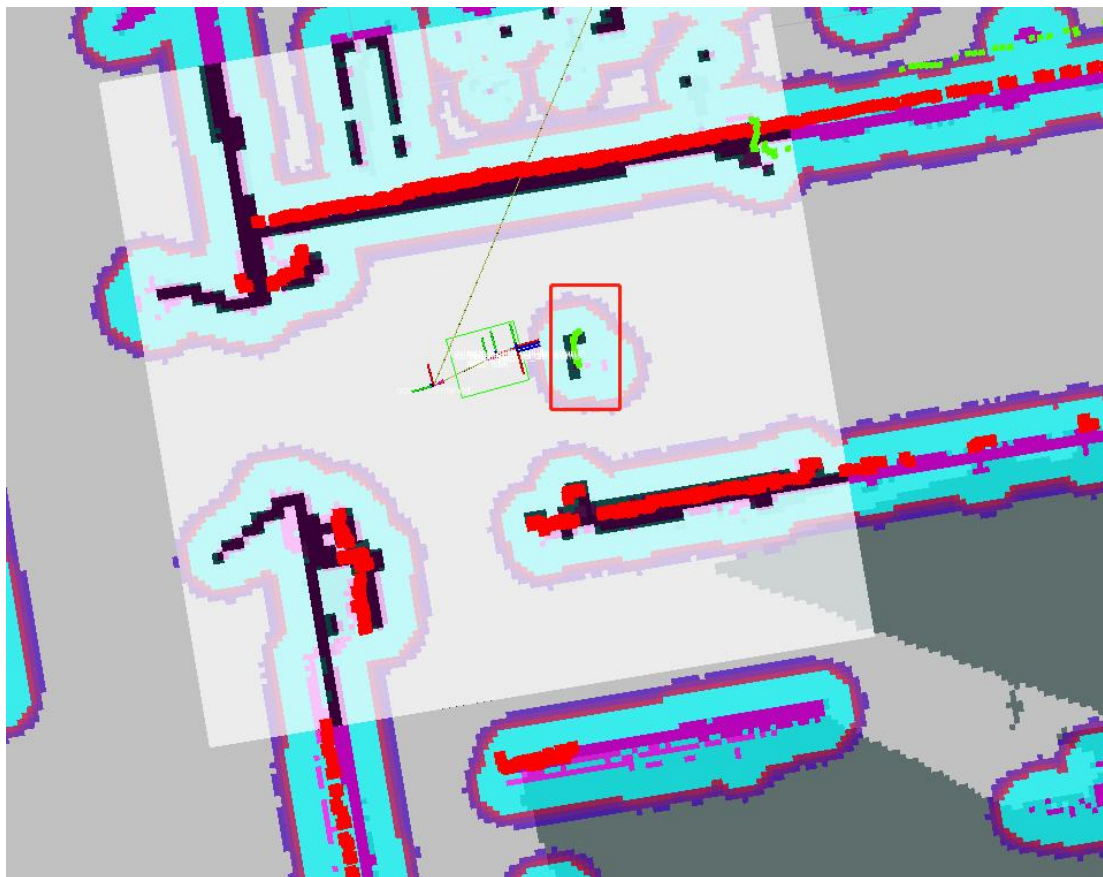
```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav.launch
... logging to /home/yhs/.ros/log/a995d530-22e8-11ef-92e5-0b8e29a6dc24/roslaunch-yhs-ros-5525.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:40445/

[ INFO] [1717553136.750830953]: Footprint model 'circular' (radius: 0.2m) loaded for trajectory optimization.
[ INFO] [1717553136.853175943]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.858632746]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.877216340]: odom received!
```

(5) 导航启动成功后，按照上面章节中的方式初始化小车。

(6) 如下图，激光安装的位置要比相机的位置高，相机能检测到较为低矮的物体，而激光检测不到



实验十六 回充

配套充电桩才有此功能，只能使用 3D 导航进行回充。

1、充电

1.1、启动 3D 导航

(1) 打开终端，输入指令，启动导航，看到“odom received!”则表示启动导航成功

```
roslaunch yhs_nav yhs_nav.launch
```

```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav.launch
... logging to /home/yhs/.ros/log/a995d530-22e8-11ef-92e5-0b8e29a6dc24/roslaunch-yhs-ros-5525.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

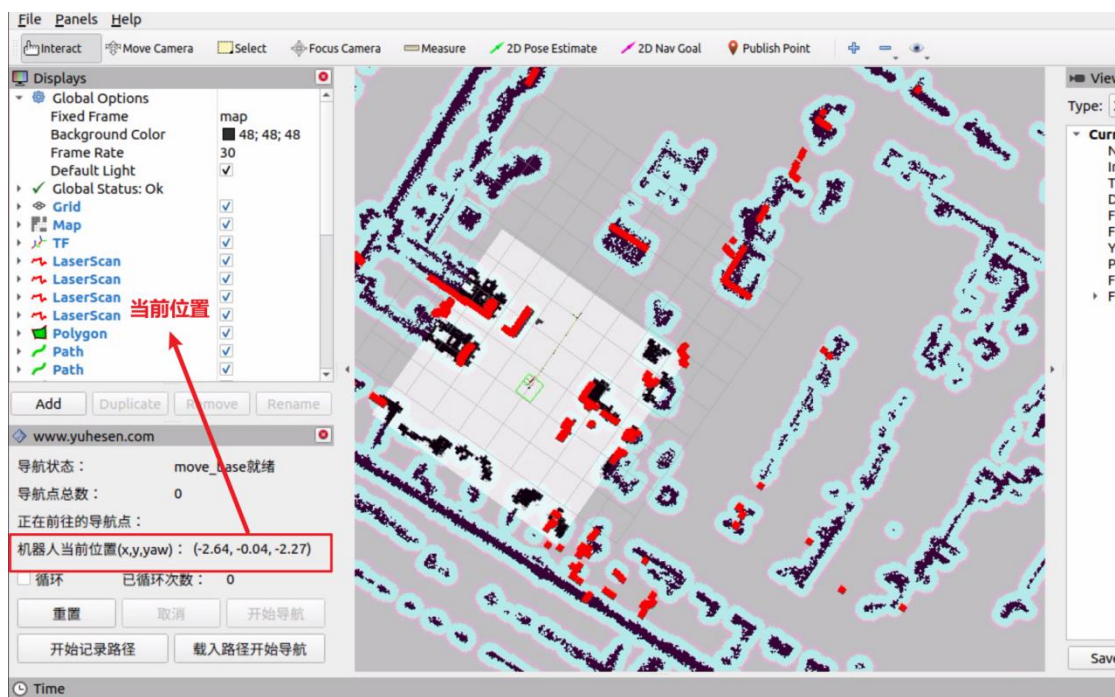
started roslaunch server http://192.168.1.102:40445/

[ INFO] [1717553136.750830953]: Footprint model 'circular' (radius: 0.2m) loaded for trajectory optimization.
[ INFO] [1717553136.853175943]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.858632746]: Recovery behavior will clear layer 'obstacles'
[ INFO] [1717553136.877216340]: odom received!
```

(2) 根据上面章节的说明，进行初始化，初始化成功才能进行下一步

(3) 遥控底盘与充电桩对准，底盘电片与充电桩电片接触位置应当在中间，方向垂直，观察充电桩上的信号灯，确保充上电即可，不用怼到底

(4) 此时，不要移动底盘和充电桩，观察 Rviz 上机器人的当前坐标 (x,y,yaw)，记录下来



(5) 记录好位置后，遥控底盘脱离充电桩，打开另外一个终端，输入“rosservice call /recharge ”指令后按“Tab”键补全，按左右箭头填入数据



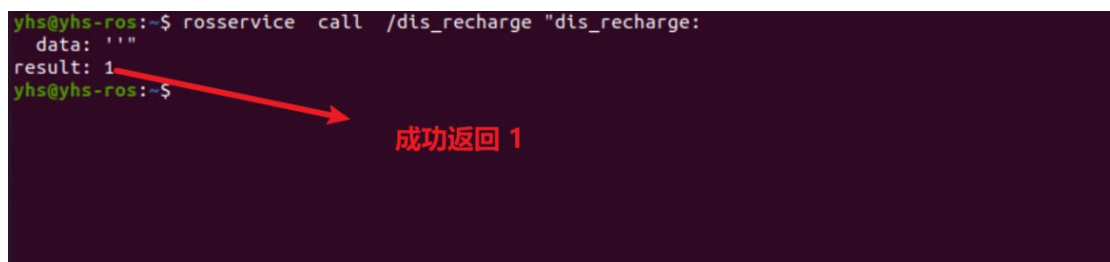
A terminal window showing the execution of the `rosservice call /recharge` command. The command is partially entered as `rosservice call /recharge "recharge_x: -0.38`. Red arrows point from the text labels to the corresponding parts of the command:
 - "成功返回 1" points to the `result: 1` output.
 - "填入记录好的 Y 坐标" points to the `recharge_y: -1.57` parameter.
 - "填入记录好的 X 坐标" points to the `recharge_x: -0.38` parameter.
 - "填入记录好的 YAW 角度" points to the `recharge_yaw: 93` parameter.
 The full command shown is `rosservice call /recharge "recharge_x: -0.38 recharge_y: -1.57 recharge_yaw: 93"` and the output is `result: 1`.

```
yhs@yhs-ros:~$ rosservice call /recharge "recharge_x: -0.38
recharge_y: -1.57
recharge_yaw: 93"
result: 1
yhs@yhs-ros:~$
```

(6) 敲回车，机器人会先导航到充电点的正前方几十厘米处，接着后退进行充电，后退过程中不能有障碍物

2、脱离充电桩

(1) 打开另外一个终端，输入“rosservice call /dis_recharge ”指令后按“Tab”键补全，敲回车，机器人往前走几十厘米脱离充电桩，前面不能有障碍物



A terminal window showing the execution of the `rosservice call /dis_recharge` command. The command is partially entered as `rosservice call /dis_recharge "dis_recharge:`. A red arrow points from the text label "成功返回 1" to the `result: 1` output.
 The full command shown is `rosservice call /dis_recharge "dis_recharge: data: ''"` and the output is `result: 1`.

```
yhs@yhs-ros:~$ rosservice call /dis_recharge "dis_recharge:
data: ''"
result: 1
yhs@yhs-ros:~$
```

常见问题解决方法

首先要查看终端上打印的错误和警告信息、Rviz 上机器人定位状态和障碍物的位置，绝大部分问题都能找到原因。

问题 1：建图、导航启动不了

检查命令是否输入正确

问题 2：保存的地图找不到

检查保存地图的目录是否正确

问题 3：3D 定位不成功

检查保存的 2D 地图是否与 3D 点云地图匹配

问题 4：下发导航点，机器人不动

检查遥控器是否切换到指令模式，检查终端输出信息，检查 Rviz 上机器人周边的障碍物情况

问题 5：重启工控机后，启动建图或者导航失败

是否使用“reboot”命令重启，使用“reboot”命令重启导致 CAN 卡不能初始化，无法获取或发送数据，要断电重启

附录一 远程控制

推荐使用 ssh，必须要用界面时可以用 NOMACHINE。

1、ssh 远程连接服务器

前提：

- 一台笔记本电脑
- 安装好 ubuntu20.04
- 安装好 noetic 版本 ros

(1) 安装 ssh

```
sudo apt install ssh
```

(2) 连接上小车的 wifi，打开终端，输入命令，敲回车，输入密码“123456”，密码输入过程无显示，密码输入正确后即可远程登录。

```
ssh yhs@192.168.1.102
```

```
xwqf@ros1:~$ ssh yhs@192.168.1.102
```

```
xwqf@ros1:~$ ssh yhs@192.168.1.102
yhs@192.168.1.102's password:
```

```
xwqf@ros1:~$ ssh yhs@192.168.1.102
yhs@192.168.1.102's password:
Welcome to Ubuntu 20.04.5 LTS (GNU/Linux 5.15.0-46-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

327 updates can be applied immediately.
263 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Your Hardware Enablement Stack (HWE) is supported until April 2025.
Last login: Wed Jun  5 15:40:18 2024 from 192.168.1.101
yhs@yhs-ros:~$
```

(3) 有时候需要小车上多个终端来输入命令，可以在笔记本上再打开终端，远程登录。

(4) 远程登录后，打开小车工控机上的 Rviz 时，是会报错的，例如

```
roslaunch yhs_chassis_description robot.launch
```

```
yhs@yhs-ros:~$ roslaunch yhs_chassis_description robot.launch
... logging to /home/yhs/.ros/log/67ee0d3a-230d-11ef-b239-7b54079e3601/roslaunch-yhs-ros-4330.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.1.102:35553/

SUMMARY
=====
PARAMETERS
 * /robot_description: <...>
 * /roscdistro: noetic
 * /rosversion: 1.16.0
 * /use_gui: False

NODES
 /
   joint_state_publisher (joint_state_publisher/joint_state_publisher)
   robot_state_publisher (robot_state_publisher/robot_state_publisher)
   rviz (rviz/rviz)

auto-starting new master
process[master]: started with pid [4340]
ROS_MASTER_URI=http://192.168.1.102:11311

setting /run_id to 67ee0d3a-230d-11ef-b239-7b54079e3601
process[rosout-1]: started with pid [4350]
started core service [/rosout]
process[joint_state_publisher-2]: started with pid [4353]
process[robot_state_publisher-3]: started with pid [4354]
process[rviz-4]: started with pid [4355]
qt.qpa.xcb: could not connect to display
qt.qpa.plugin: Could not load the Qt platform plugin "xcb" in "" even though it was found.
This application failed to start because no Qt platform plugin could be initialized. Reinstalling the application may fix this problem.

Available platform plugins are: eglfs, linuxfb, minimal, minimalegl, offscreen, vnc, wayland-egl, wayland, wayland-xcomposite-egl, wayland-xcomposite-glx, xcb.

[rviz-4] process has died [pid 4355, exit code -6, cmd /opt/ros/noetic/lib/rviz/rviz -d /home/yhs/catkin_ws/src/yhs_chassis_description/rviz/urdf.rviz
_name=rviz _log=/home/yhs/.ros/log/67ee0d3a-230d-11ef-b239-7b54079e3601/rviz-4.log].
log file: /home/yhs/.ros/log/67ee0d3a-230d-11ef-b239-7b54079e3601/rviz-4*.log
```

(5) 在 ros 中，连上同一个局域网，通过设置环境变量，那么就可以在笔记本上打开 Rviz，在此之前，先把小车工控机上 Rviz 的默认配置文件复制到笔记本上

```
scp yhs@192.168.1.102:~/rviz/default.rviz ~/rviz/
```

```
xwqf@ros1:~$ scp yhs@192.168.1.102:~/rviz/default.rviz ~/rviz/
yhs@192.168.1.102's password:
default.rviz 100% 19KB 1.5MB/s 00:00
```

(6) 假设笔记本上有“rviz_tool/src”目录，将小车工控机上的文件夹“yhs_nav_rviz_plugin”复制到笔记本上的“rviz_tool/src”目录下

```
xwqf@ros1:~$ cd rviz_tool/src/
xwqf@ros1:~/rviz_tool/src$
xwqf@ros1:~/rviz_tool/src$
xwqf@ros1:~/rviz_tool/src$ ls
xwqf@ros1:~/rviz_tool/src$
```

```
scp -r yhs@192.168.1.102:~/catkin_ws/src/yhs_nav_rviz_plugin .
```

```
xwqf@ros1:~/rviz_tool/src$ scp -r yhs@192.168.1.102:~/catkin_ws/src/yhs_nav_rviz_plugin .
yhs@192.168.1.102's password:
path.txt 100% 802 232.2KB/s 00:00
multi_navi_goal_panel.h 100% 4144 1.3MB/s 00:00
multi_navi_goal_panel.cpp 100% 23KB 3.9MB/s 00:00
plugin_description.xml 100% 291 113.4KB/s 00:00
CMakeLists.txt 100% 1282 495.4KB/s 00:00
package.xml 100% 1106 451.6KB/s 00:00
xwqf@ros1:~/rviz_tool/src$ ls
yhs_nav_rviz_plugin
xwqf@ros1:~/rviz_tool/src$
```

(7) 将所需文件复制到笔记本上后，进行编译，编译过程中可能会报缺少某个依赖包的错误，这时切换可以联网的 wifi（安装完成再切回小车上的 wifi）逐一安装依赖即可，安装完所有依赖再编译

```
xwqf@ros1:~/rviz_tool$ catkin_make
```

```
-- traversing 1 packages in topological order:
--   - navi_tools_rviz_plugin
--
-- *** processing catkin package: 'navi_tools_rviz_plugin'
-- ==> add_subdirectory(yhs_nav_rviz_plugin)
-- Using these message generators: gencpp;geneus;genlisp;gennodejs;genpy
-- Using Qt5 based on the rviz_QT_VERSION: 5.12.8
-- Configuring done
-- Generating done
-- Build files have been written to: /home/xwqf/rviz_tool/build
####
#### Running command: "make -j16 -l16" in "/home/xwqf/rviz_tool/build"
####
Scanning dependencies of target navi_tools_rviz_plugin_autogen
[ 25%] Automatic MOC for target navi_tools_rviz_plugin
[ 25%] Built target navi_tools_rviz_plugin_autogen
Scanning dependencies of target navi_tools_rviz_plugin
[ 75%] Building CXX object yhs_nav_rviz_plugin/CMakeFiles/navi_tools_rviz_plugin.dir/navi_tools_rviz_plugin_autogen/mocs_compilation.cpp.o
[ 75%] Building CXX object yhs_nav_rviz_plugin/CMakeFiles/navi_tools_rviz_plugin.dir/src/multi_navi_goal_panel.cpp.o
[100%] Linking CXX shared library /home/xwqf/rviz_tool/devel/lib/libnavi_tools_rviz_plugin.so
[100%] Built target navi_tools_rviz_plugin
xwqf@ros1:~/rviz_tool$
```

(8) 编译成功后，在远程登录工控机的终端运行导航程序，请一定要按照下图的步骤来操作（注意图片左上角绿色字体，不要弄错终端）

```
yhs@yhs-ros:~$ roslaunch yhs_nav yhs_nav.launch
... logging to /home/yhs/.ros/log/a2a0c700-2313-11ef-b239-7b54079e3601/roslaunch-yhs-ros-7531.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

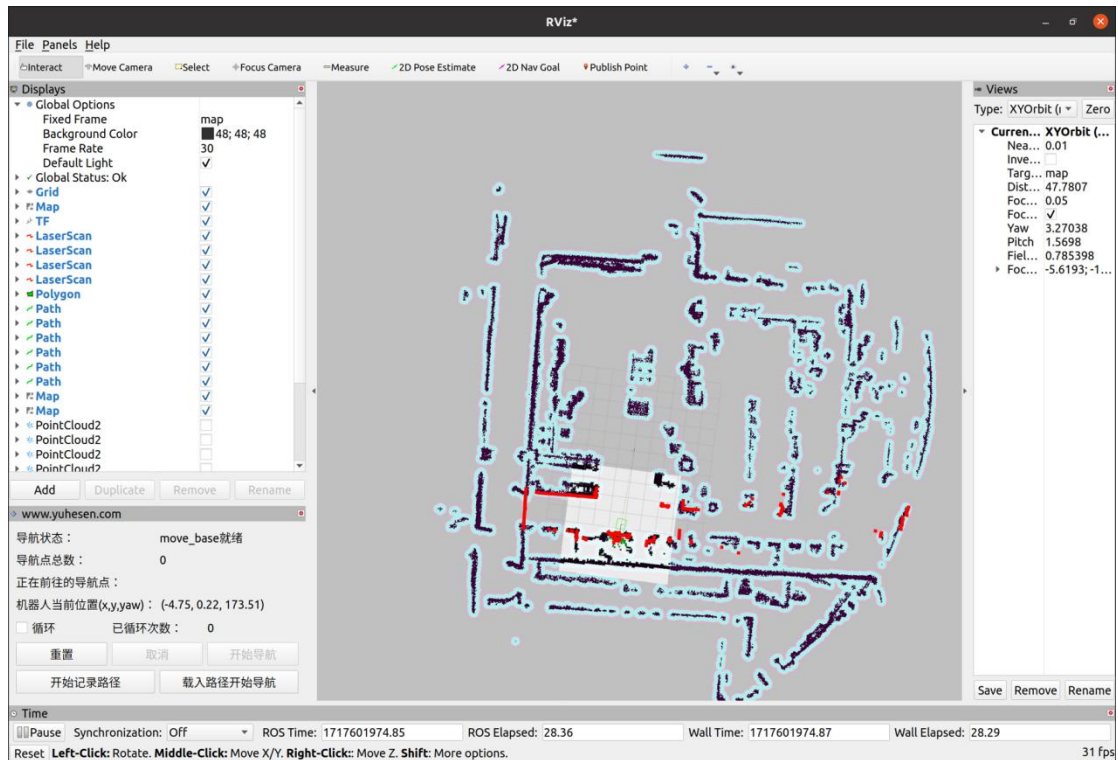
started roslaunch server http://192.168.1.102:38091/
```

在笔记要打开 Rviz 的终端执行以下命令

cd ~/rviz_tool
source devel/setup.bash
export ROS_HOSTNAME=192.168.1.101 （192.168.1.101 是我笔记本的 IP，要改为你的 IP）
export ROS_MASTER_URI=http://192.168.1.102:11311

```
xwqf@ros1:~/rviz_tool$ source devel/setup.bash
xwqf@ros1:~/rviz_tool$
xwqf@ros1:~/rviz_tool$
xwqf@ros1:~/rviz_tool$
xwqf@ros1:~/rviz_tool$ export ROS_HOSTNAME=192.168.1.101
xwqf@ros1:~/rviz_tool$
xwqf@ros1:~/rviz_tool$ export ROS_MASTER_URI=http://192.168.1.102:11311
xwqf@ros1:~/rviz_tool$
```

```
xwqf@ros1:~/rviz_tool$ rviz
```



(9) 在笔记本上打开 Rviz 后, 就可以进行导航的各种操作了, 具体操作看上面的导航章节。

2、NOMACHINE 远程软件

前提:

- 一台笔记本
- 下载 NOMACHINE 软件，安装好，windows 和 ubuntu 系统都有相应的版本

(1) 连接小车上的 wifi，先在笔记本上 ssh 连上工控机，输入以下两条指令后，才能进行下面的步骤

```
sudo systemctl stop gdm3.service
```

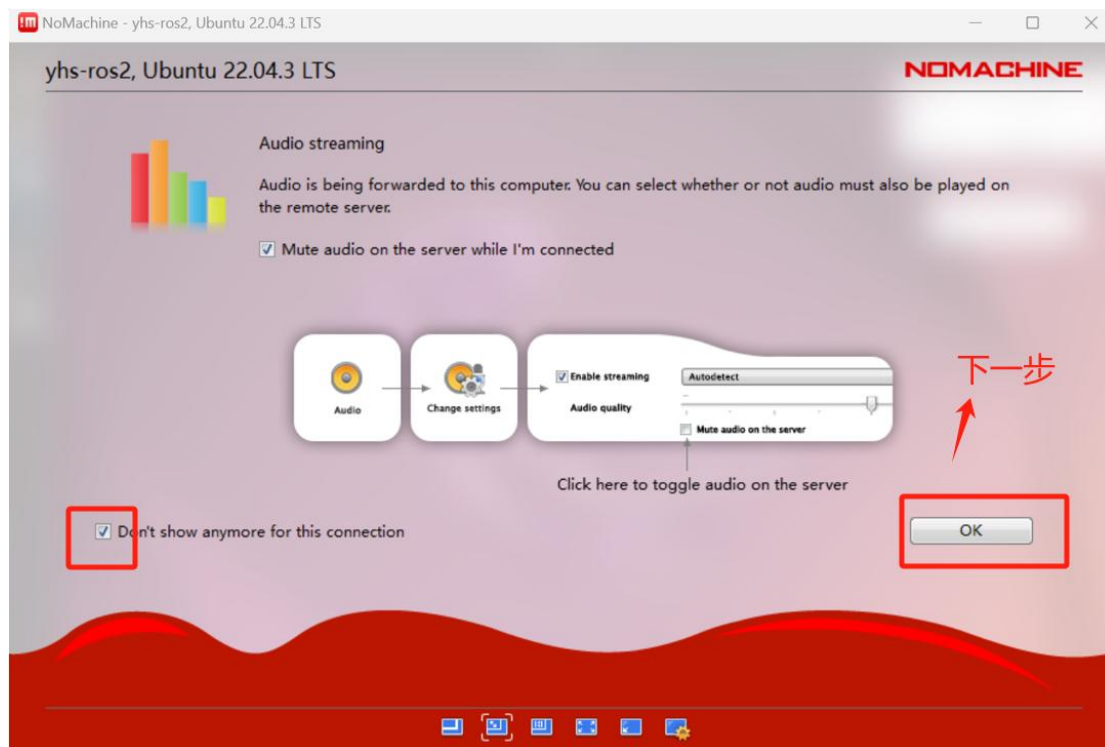
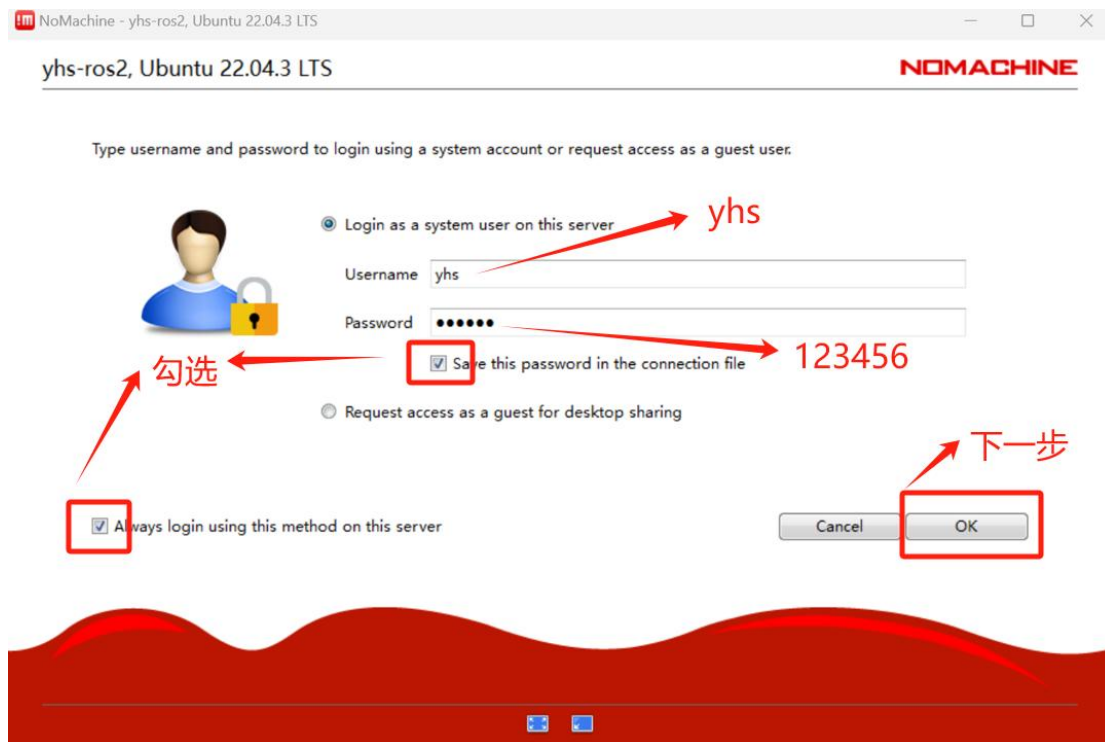
```
sudo systemctl restart nxserver.service
```

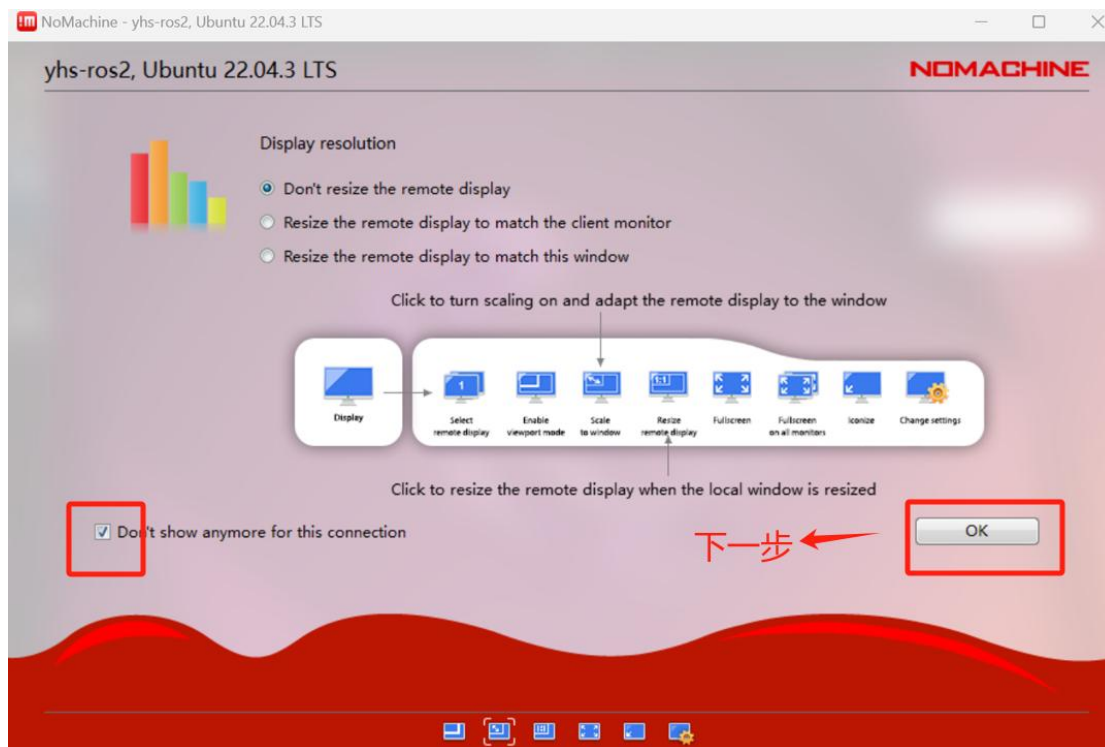
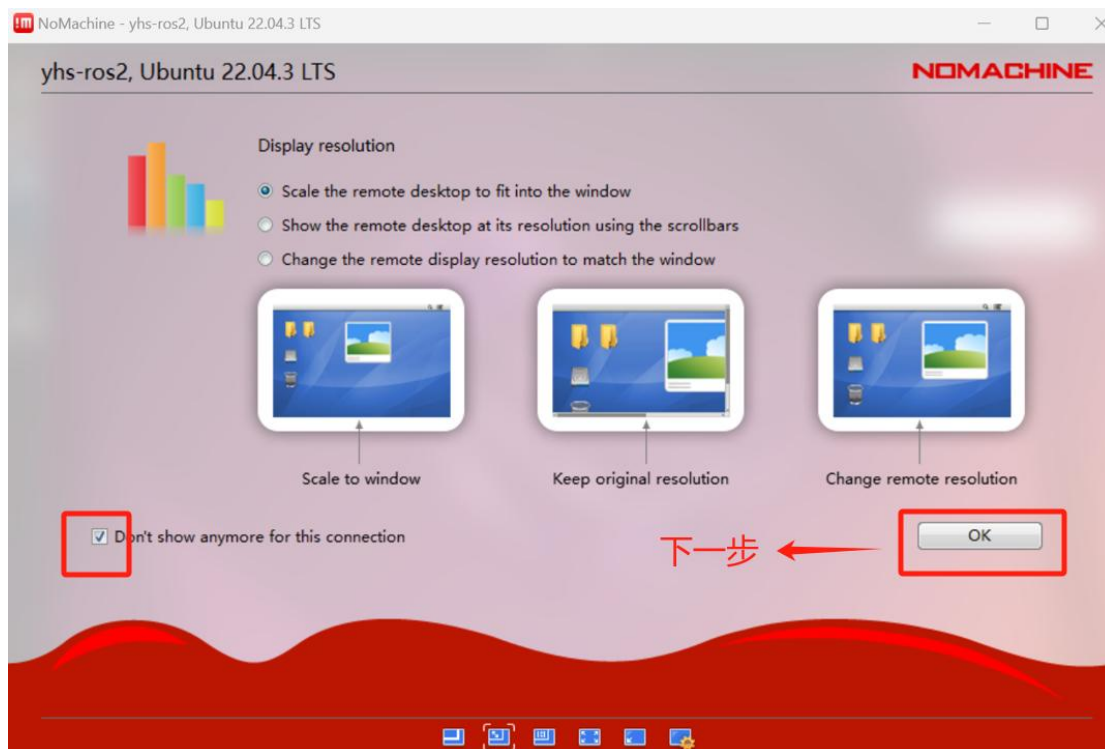
```
yhs@yhs-ros:~$ sudo systemctl stop gdm3.service
[sudo] password for yhs:
yhs@yhs-ros:~$
yhs@yhs-ros:~$
yhs@yhs-ros:~$ sudo systemctl restart nxserver.service
yhs@yhs-ros:~$
```

(2) 打开 NOMACHINE 软件，进入主界面后可以看到，小车上工控机安装的 ubuntu 系统信息

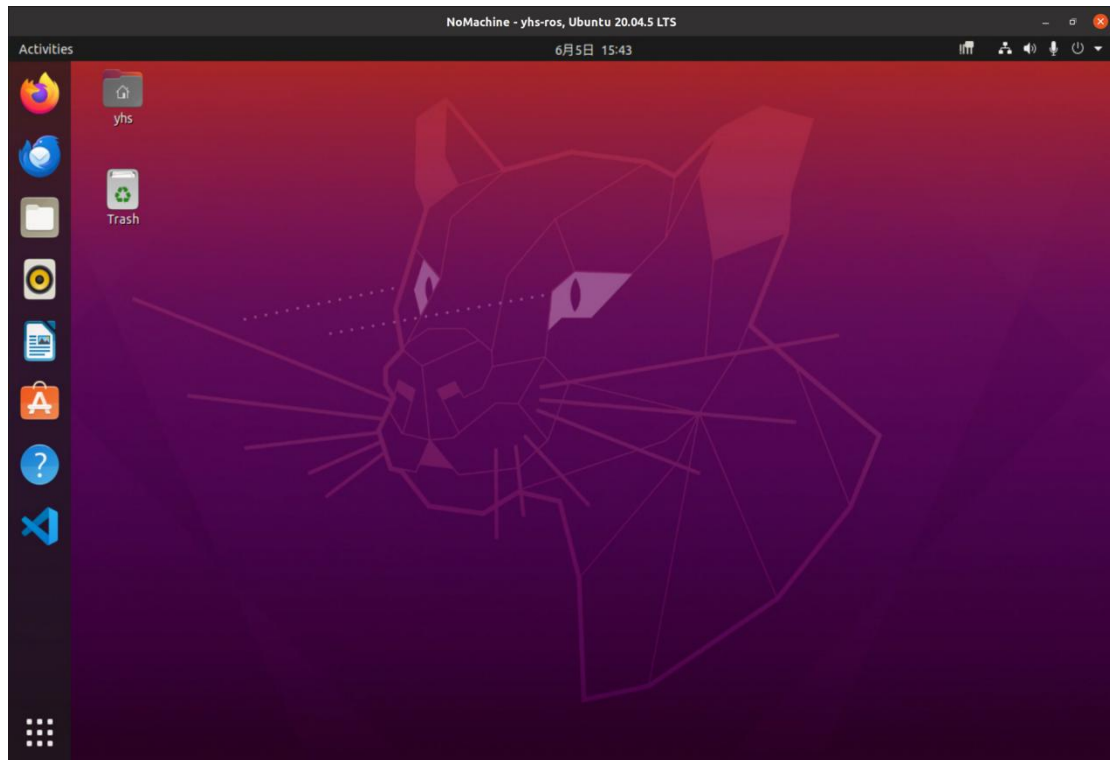


(3) 双击图标开始连接，过程如下





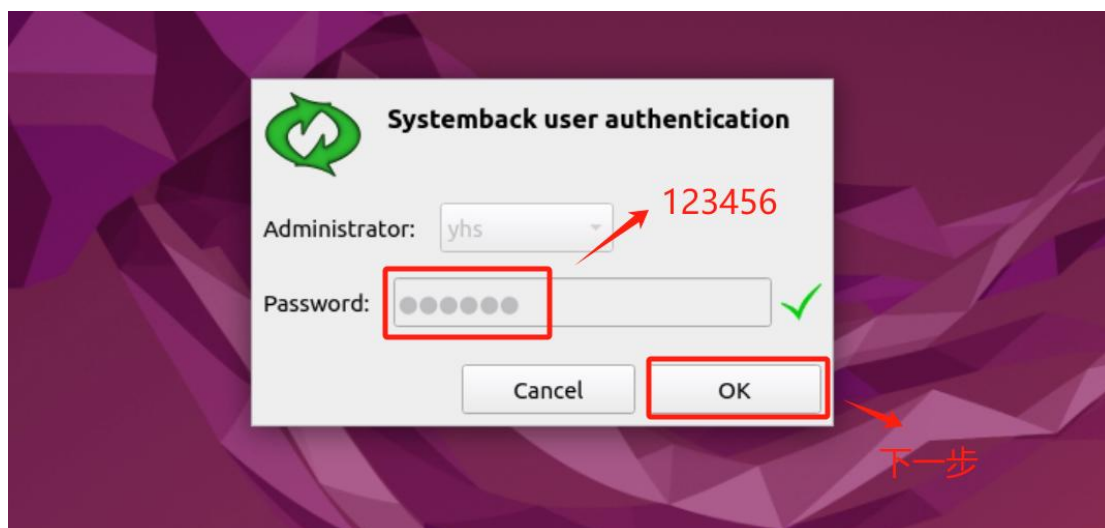
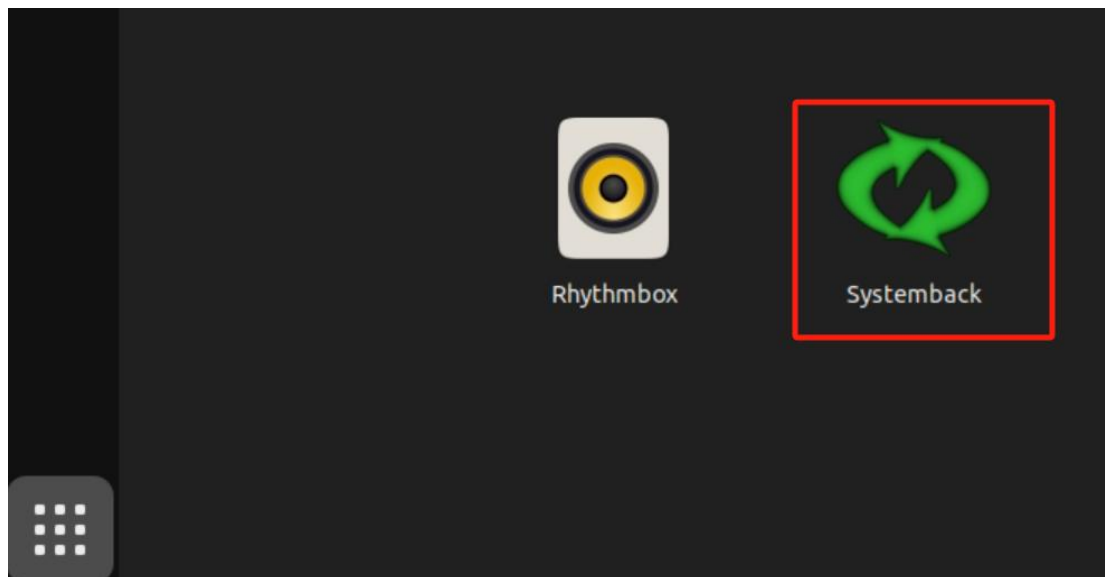
(4) 进入小车工控机桌面

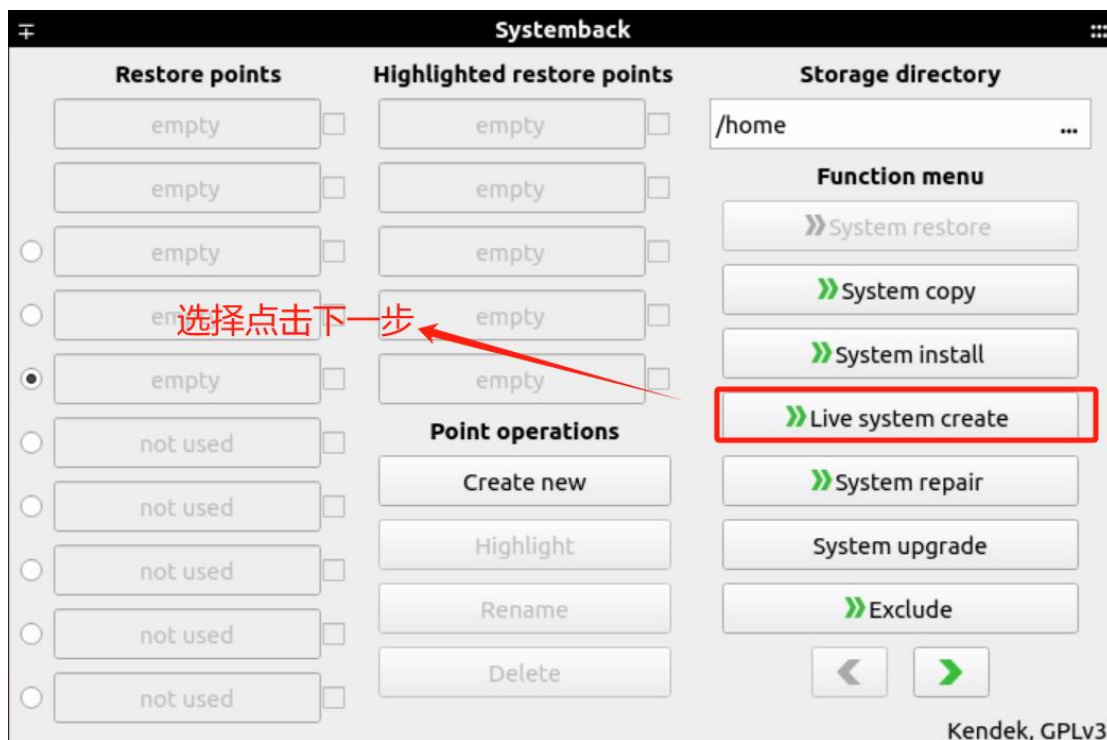


(5) 进入桌面后，操作跟工控机接屏幕的操作是一样的，某些快捷键可能用不了。

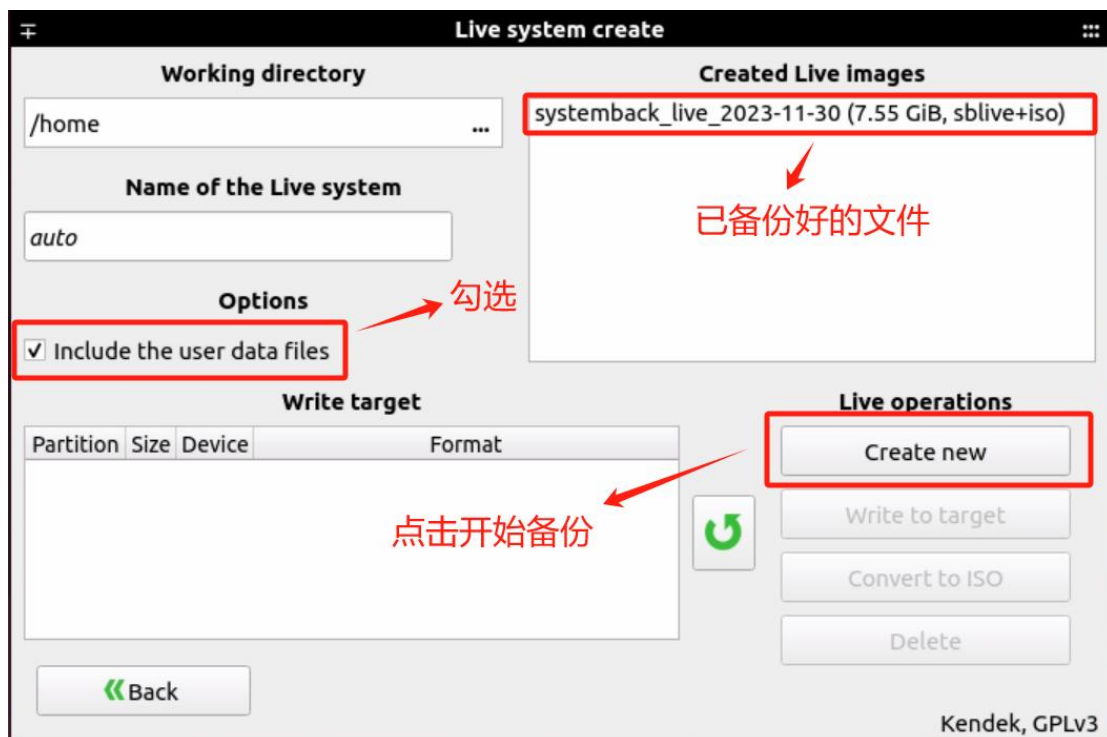
附录二 系统备份

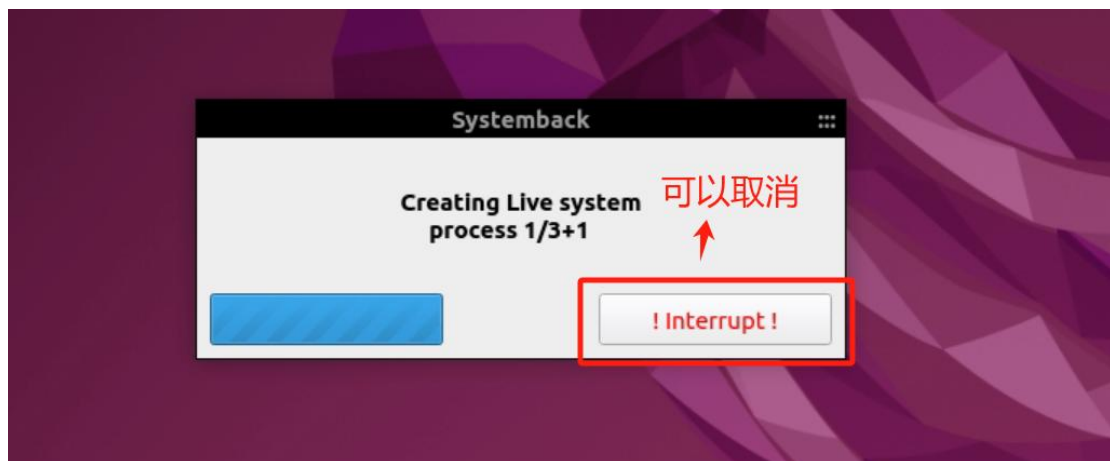
(1) 在桌面左下角搜索选择打开“Systemback”，输入密码，选择进入“Live system create”



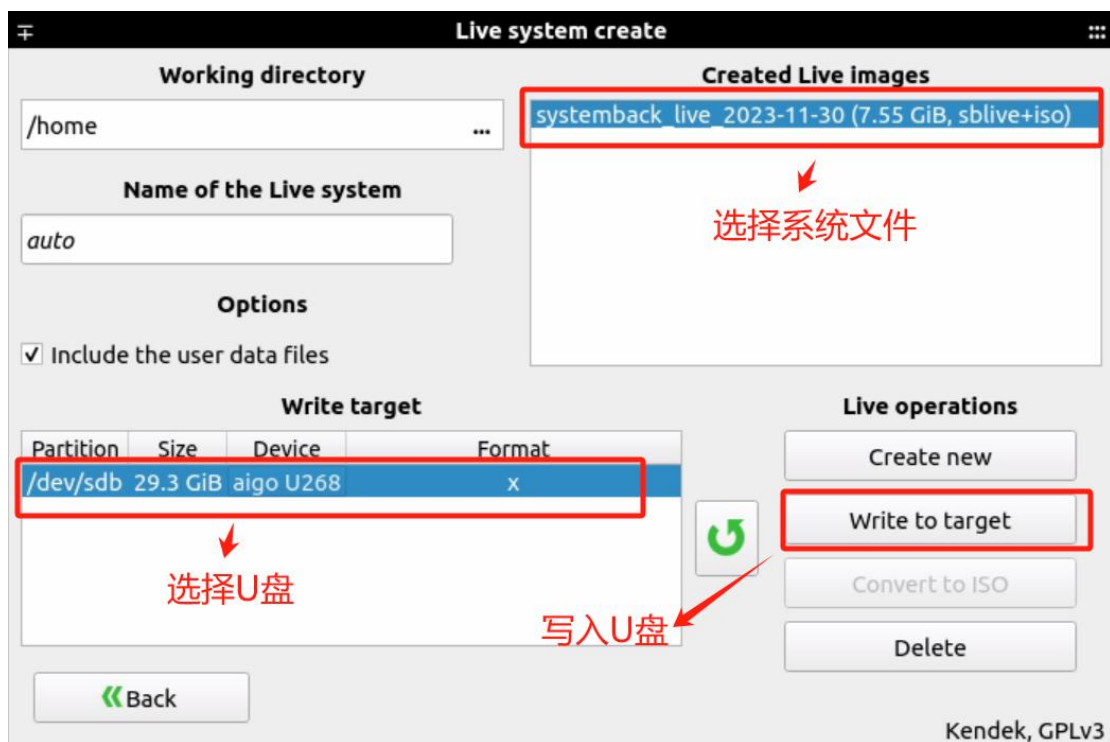


(2) 开始备份系统（非必要文件可以删除再备份），备份时间长，等待完成，备份过程不要运行程序和进行其他操作





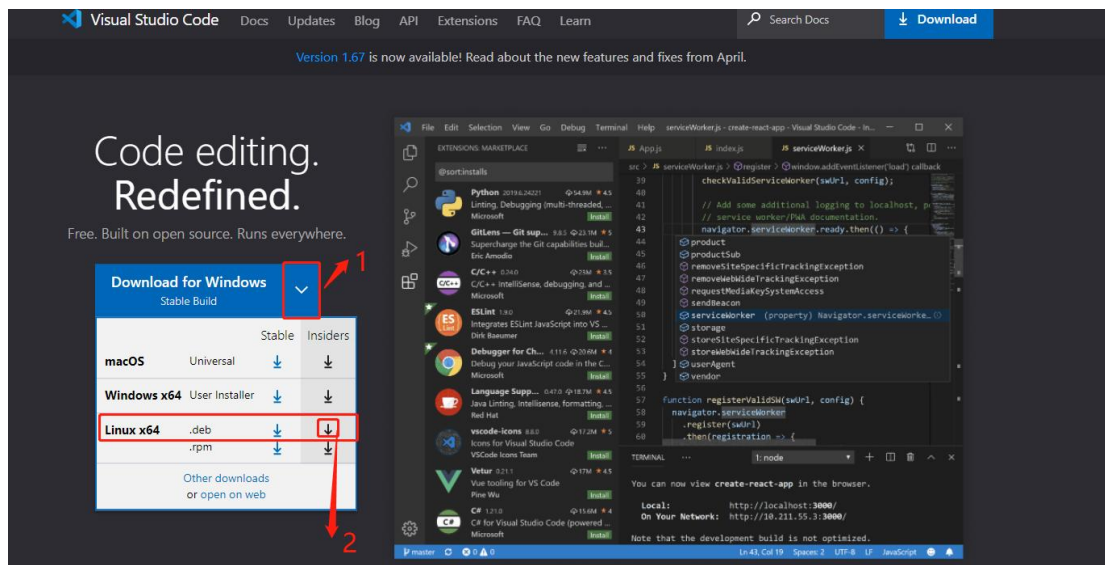
(3) 备份完成后，插入 U 盘到工控机，把系统写入 U 盘（备份前会清空 U 盘），写入时间长，等待写入完成



附录三 安装 vscode

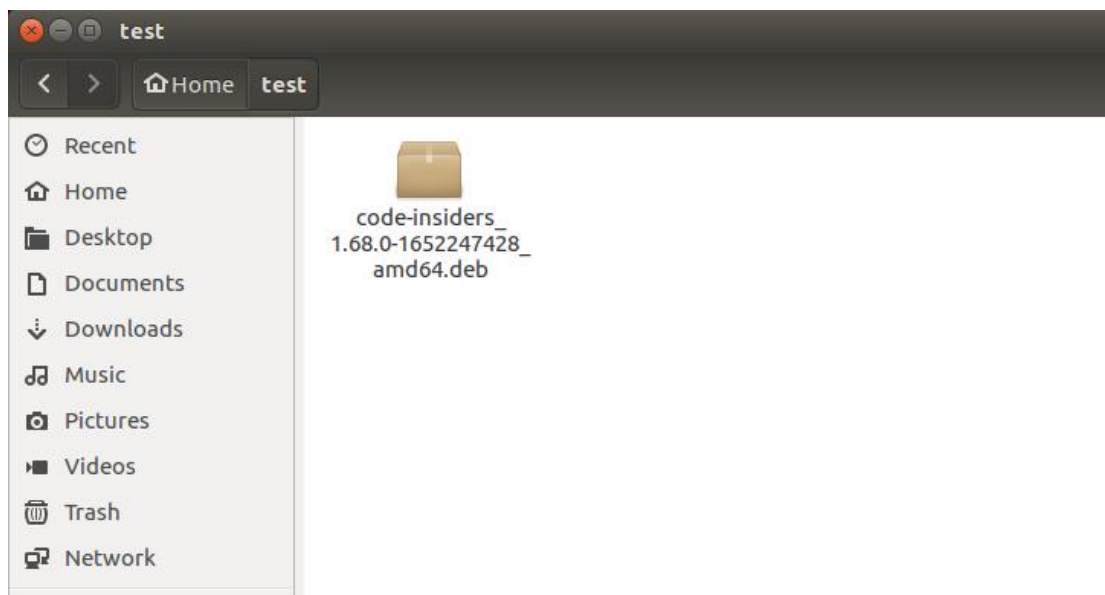
工控机已安装好，下面只是做演示。

- 1、进入官网（<https://code.visualstudio.com/>）下载 Linux x64 deb 安装版。



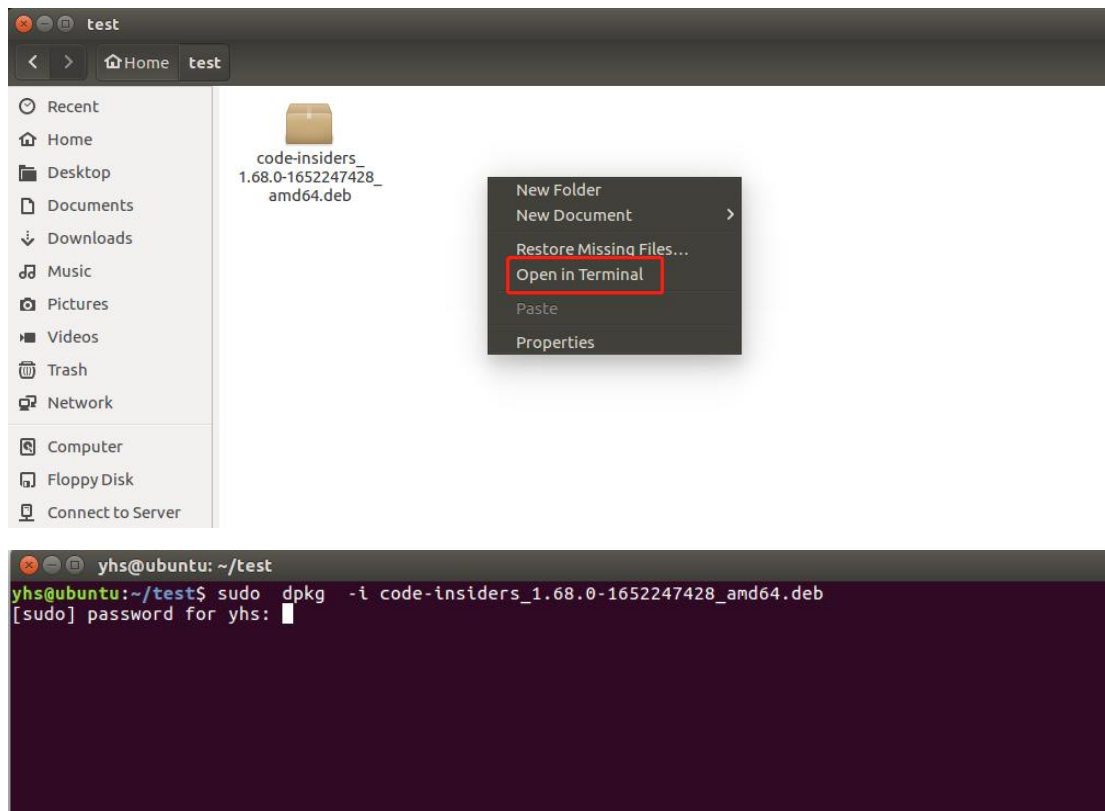
- 2、假设我们下载到的是 “code-insiders_1.68.0-1652247428_amd64.deb”

cc



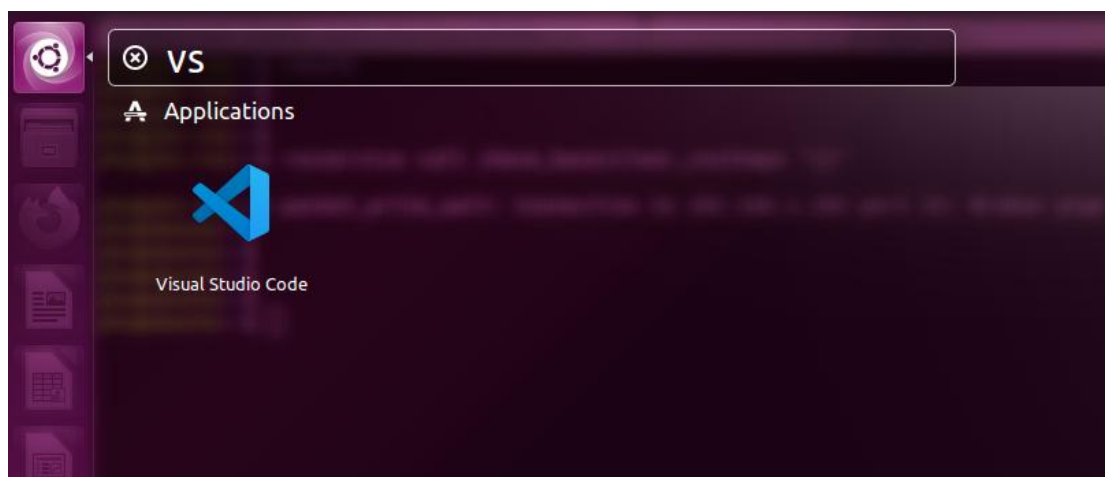
3、在安装包的目录下右键打开终端，输入指令后敲回车

```
sudo dpkg -i code-insiders_1.68.0-1652247428_amd64.deb
```



4、输入 root 密码，等待安装完成即可。

5、打开 vscode，点击搜索，输入“vs”后就可以看到软件图标了，点击图标就可以打开软件。至此，软件安装完成。



电话: 0755-28468956

传真: 0755-28468956

网址 www.yuhesen.com 邮件: sales01@yuhesen.com

地址: 深圳市龙岗区宝龙六路 1 号创维群欣科技园 1 栋 5 楼

YUHESEN

深圳煜禾森科技有限公司
Shenzhen yuhesen Technology Co., Ltd.